Podstawy Inżynierii Oprogramowania

Ćwiczenia

Opracowania tematów

Spis treści

1. Diagram klas *3*

C02 1.1-1.4 3

1.1 Klasa, obiekt 3

1.2 Relacja generalizacji 7

1.3 Asocjacja: podstawowe właściwości 13

1.4 Asocjacja: zaawansowane aspekty (asocjacja rekurencyjna, asocjacja n-arna) 18

C03 1.5-1.7 22

1.5 Typ danych 22

1.6 Właściwość statyczna (w tym *read only*) 25

1.7 Właściwość wnioskowana (*derived property*) 27

C04 1.8-1.10 29

1.8 Krotności i liczności 29

1.9 Właściwość identyfikująca obiekty klasy (ograniczenie ID) 31

1.10 Wartość domyślna właściwości 33

C05 1.11-1.13 35

1.11 Właściwość wielowartościowa (ograniczenie ORDERED, NOUNIQUE) 35

1.12 Klasa asocjacji 36

1.13 Agregacja, Kompozycja 37

2. Diagram przypadków użycia *41*

C06 2.1-2.3 41

2.1 Aktor 41

2.2 Przypadek użycia 43

2.3 Asocjacja 47

C07 2.4-2.5 49

2.4 Relacja zawierania (*include dependency*) 49

2.5 Relacja rozszerzania (*extend dependency*) 51

C08 2.6-2.7 53

2.6 Relacja generalizacji: aktorzy 53

2.7 Relacja generalizacji: przypadki użycia 53

3. Diagram aktywności *55*

C09 3.1-3.5 55

3.1 Aktywność, a czynność (*Activity* vs *Action*) 55

3.2 Aktywność: przepływ obiektów, węzły obiektów 61

3.3 Aktywność: przepływ sterowania, węzły sterowania 64

3.4 Aktywność: węzły wykonawcze (*executable nodes*) 70

3.5 Aktywność: grupy aktywności (*Activity Groups*), partycje aktywności (*Activity Partitions*) 73

C10 3.6-3.8 79

3.6 Akcja: podział na rodzaje, funkcja przekaźników (*Pins*) 79

3.7 Akcja: akcje wywołania (*invocation actions*) 85

3.8 Akcja: akcje obiektów (*object actions*) 89

C11 3.9-3.11 92

3.9 Akcja: akcje cech strukturalnych (*Structural Feature Action*), akcje zmiennych (*Variable Actions*), akcje odpowiedzi na zdarzenia (*Accept Actions*), pozostałe akcje (*Other Actions*) 92

3.10 Akcja: akcje strukturalne (*Structured Actions*) 97

3.11 Akcja: obszary rozszerzenia (*Expansion Regions*) 100

4. Ograniczenia i reguły biznesowe w modelowaniu systemów w UML *102*

C12 4.1-4.3 102

4.1 Reguły biznesowe i ich kategorie (definicja terminów biznesowych; fakty pokazujące relacje między terminami; ograniczenia; wnioskowania) 102

4.2 Ograniczenia na elementy modelu systemu (niezmienniki dla atrybutów klasy; niezmienniki dla asocjacji klasy; określenia warunków początkowych i końcowych realizacji operacji; ograniczenia dla zbiorów generalizacji) 103

4.3 Stereotypy związane z ograniczeniami w UML: `<<invariant>>`, `<<precondition>>`, `<<postcondition>>` 105

5. Wymagania *107*

C13 5.1-5.8 107

5.1 Wymagania wg UML (diagram przypadków użycia) i wg sysML (diagram wymagań) 107

5.2 Elementy składni konkretnej diagramu wymagań wg sysML 108

5.3 Wymagania wg RUP 109

5.4 Specyfikowanie treści przypadków użycia wg RUP (przejście od dyscypliny *Requirements* do dyscypliny *Analisys & Design*) 111

5.6 Klasyfikacja FURPS+ wg RUP w odniesieniu do wymagań 113

5.7 Klasyfikacja FURPS+ w odniesieniu do testów (przez analogię do klasyfikacji wymagań wg RUP) 114

5.8 Wybrane klasyfikacje i metodyki opisu wymagań 115

1. Diagram klas

C02 1.1-1.4

1.1 Klasa, obiekt

Klasa (`Class`) – jest to podstawowy element modelowania w UML, który reprezentuje zbiór obiektów o wspólnych cechach (a także ograniczeniach i znaczeniu) (*features, constraints, semantics*). Definiuje ona zarówno właściwości (*properties*), jak i zachowania (*behaviors*) tych obiektów.
Klasa może zawierać:

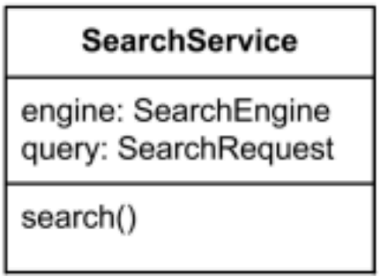
a) **Atrybuty (*attributes*)** – cechy lub właściwości opisujące stan obiektów należących do klasy.
b) **Operacje (*operations*)** – funkcje lub metody, które mogą być wykonywanie przez obiekty klasy.

W UML klasa jest przedstawiana jako prostokąt podzielony na trzy sekcje:

a) Nazwa klasy – wyśrodkowana i pogrubiona, nazwa pisana wielką literą



b) Atrybuty – wyjustowane do lewej, rozpoczynane małą literą (środkowa sekcja)
c) Operacje – analogicznie do atrybutów (dolna sekcja)

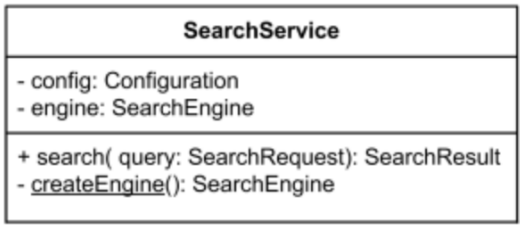


Cechy klasy mogą być:

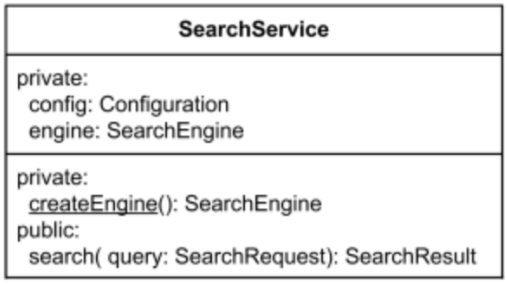
a) Dynamiczne (instancyjne) (*dynamic*) – czyli dotyczące pojedynczych instancji klasy
b) Statyczne (*static*) – czyli dotyczące całego klasyfikatora.

Ta sama cecha nie może jednocześnie być statyczna i niestatyczna w różnych kontekstach!

Cechy statyczne oznaczane są poprzez podkreślenie nazwy. Ponadto, symbol `(...)` na końcu listy cech wskazuje, że istnieją dodatkowe cechy, ale nie zostały ujęte w diagramie.



Atrybuty i operacje mogą być grupowane przez kryterium widoczności (*visibility*). Widoczność może być określona raz dla całej grupy cech z tą samą widocznością.
Słowa kluczowe: publiczny, prywatny, chroniony (`+public, -private, #protected`).



W UML można dodawać dodatkowe sekcje / przedziały, które mogą zawierać, np.:

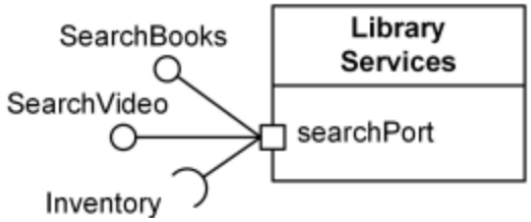
- Ograniczenia (*constraints*)
- Inne szczegóły klasyfikatora
- Podział cech na kategorie

Klasa jako przestrzeń nazw (*namespace*) – dla innych klasyfikatorów, klasa w UML może działać jako przestrzeń nazw (np. dla innych klas, interfejsów czy przypadków użycia (*use cases*)).
Klasyfikatory zagnieżdżone są widoczne tylko wewnątrz przestrzeni nazw klasy nadrzędnej.

Klasa ustrukturyzowana i hermetyzacja – od UML 2.5, klasa może być:

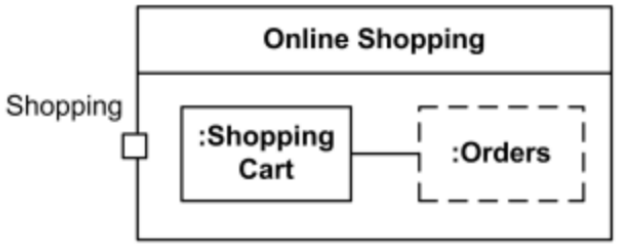
a) Ustrukturyzowana (*structured*)
b) Hermetyzowana (*encapsulated*)
c) Zachowująca się (*behaviored*).

Dzięki temu, każda klasa może posiadać wewnętrzną strukturę i porty.



Zagnieżdżone klasyfikatory są ograniczone do przestrzeni nazw klasy nadrzędnej, co wspiera hermetyzację (*information hiding*) poprzez porty.

Przykład klasy z portem i strukturą wewnętrzną:



Port – (własność klasyfikatora) jest specjalnym punktem interakcji na granicy klasy lub komponentu, który umożliwia komunikację między obiektem a jego otoczeniem. Reprezentuje punkt dostępu do funkcjonalności obiektu, ale nie określa szczegółowo jego wewnętrznej implementacji.

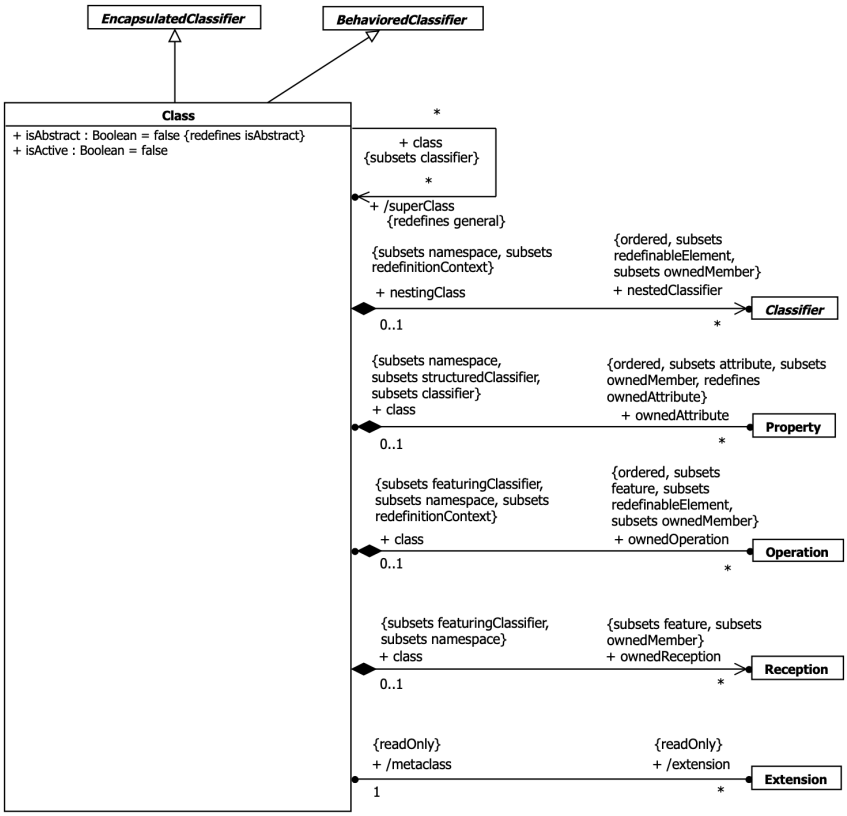
Struktura wewnętrzna – jest to organizacja wewnętrzna klasy, która określa komponenty, ich relacje i sposób interakcji. Umożliwia kompozycję obiektów i opisuje, jak różne części współpracują w ramach jednej klasy.

Obiekt (*Object*) – jest to konkretna instancja klasy, posiadająca rzeczywiste wartości atrybutów i mogąca wykonywać operacje zdefiniowane w klasie. Obiekt reprezentuje jednostkowy element systemu z unikalnym stanem i zachowaniem. W diagramach UML obiekt jest przedstawiany jako prostokąt z podkreśloną nazwą w formacie `nazwaObiektu: Klasa`.

Przykład:
Rozważmy klasę `Samochód` z atrybutami marka i model oraz operacją `uruchom()`. Obiekt `mójSamochód` będący instancją tej klasy może mieć przypisane wartości `marka = "Toyota"` i `model = "Corolla"`, a wywołanie `mójSamochód.uruchom()` spowoduje uruchomienie samochodu.

Podsumowując, klasa definiuje strukturę i zachowanie grupy obiektów, podczas gdy obiekt jest konkretną instancją tej klasy z określonym stanem.

Składnia abstrakcyjna (*abstract syntax*):



Powiązania z innymi kategoriami UML

`Class` jest szczególnym przypadkiem `Classifier`, co oznacza, że może mieć atrybuty (`ownedAttribute`), mieć operacje (`ownedOperation`) oraz być częścią hierarchii dziedziczenia (`superClass`). Każda klasa może posiadać relację dziedziczenia do innej klasy (`superClass {redefines general}`).

Klasa może być powiązana z:

- `EncapsulatedClassifier` – może zawierać zamknięte struktury (elementy dostępne wyłącznie wewnątrz tej klasy, niewidoczne na zewnątrz)
- `BehavioredClassifier` – może definiować zachowania
- `Property` – może zawierać atrybuty (`ownedAttribute`)
- `Operation` – może posiadać operacje (`ownedOperation`)
- `Reception` – może obsługiwać sygnały (`ownedReception`) (komunikaty, które mogą być wysyłane między obiektami w sposób asynchroniczny)
- `Extension` – może być rozszerzona w ramach mechanizmu metamodelowania UML (dostosowana i wzbogacona o dodatkowe informacje poprzez: stereotypy, profile, metaklasy lub rozszerzenia).

Własności klasy w UML

Klasa posiada następujące właściwości:

a) `isAbstract` (`Boolean`, domyślnie `false`):
   - Określa, czy klasa jest abstrakcyjna (czyli nie można utworzyć jej instancji)
   - Jeśli `true`, to klasa musi być dziedziczona przez inne klasy.
b) `isActive` (`Boolean`, domyślnie `false`):
   - Określa, czy klasa jest aktywna, tzn. czy ma własny wątek wykonywania
   - Klasy aktywne mogą definiować własne zachowanie (np. w systemach wielowątkowych).
c) Atrybuty – klasa może posiadać zestaw atrybutów, które są instancjami `Property`.
d) Operacje – klasa może zawierać metody (operacje) opisujące jej zachowanie.
e) Dziedziczenie (`superClass`) – klasa może dziedziczyć po innych klasach, redefiniując ich własności.
f) Zagnieżdżone klasy (`nestedClassifier`) – klasa może zawierać inne klasy jako swoje zagnieżdżone elementy.
g) Metaklasa (`metaclass`) – klasa może należeć do systemu metamodelowania UML.

1.2 Relacja generalizacji

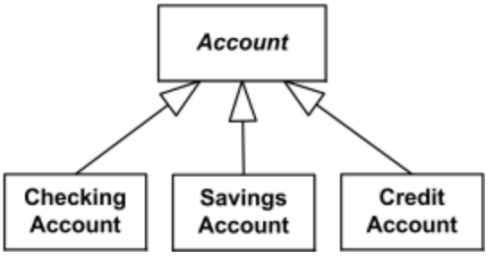
Generalizacja – jest to binarna, taksonomiczna (czyli związana z klasyfikacją) relacja skierowana pomiędzy bardziej ogólnym klasyfikatorem (superklasą - *superclass*) a bardziej szczegółowym klasyfikatorem (podklasą / subklasą - *subclass*).

Każda instancja klasyfikatora szczegółowego jest również pośrednią instancją klasyfikatora ogólnego, (czyli jeśli `B` jest podklasą `A`, to każdy obiekt `B` można traktować jako obiekt `A`) co oznacza, że możemy powiedzieć:
a) „Pacjent jest Osobą.” („*Patient is a Person.”)*
b) „Konto oszczędnościowe jest Kontem.” („*Savings account is an Account.”)*

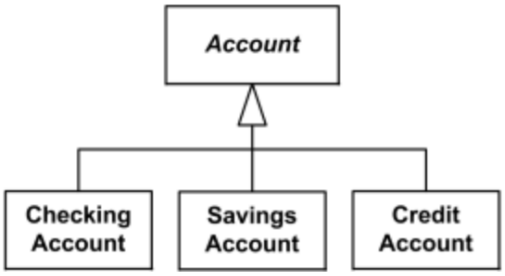
Z tego powodu **relacja generalizacji** jest również nieformalnie nazywana relacją „Jest A” (*Is-A relationship*).

Generalizacja jest własnością klasyfikatora szczegółowego.

Reprezentacja na diagramie UML
Generalizacja jest przedstawiana jako linia z pustym trójkątem jako grotem strzałki, łącząca symbole reprezentujące klasyfikatory – grot strzałki wskazuje na klasyfikator ogólny. Taka notacja nazywana jest „**stylem oddzielnego celu**” (*separate target style*).



Relacje generalizacji, które odnoszą się do tego samego klasyfikatora ogólnego, mogą być również połączone w „**stylu wspólnego celu**" (*shared target style*).



Właściwości generalizacji:

a) Dziedziczenie:
   - Klasa podrzędna **przejmuje** wszystkie nieprywatne atrybuty i operacje klasy nadrzędnej.
   - Może również dodać własne cechy lub **przesłonić (*override*)** metody klasy nadrzędnej.

b) Polimorfizm:
   - Instancja klasy podrzędnej może być traktowana jako instancja klasy nadrzędnej.
   - Przykład: Obiekt `Sedan` można przypisać do zmiennej typu `Samochód`.

c) Generalizacja statyczna i dynamiczna:
   - UML nie wymusza konkretnego sposobu dziedziczenia – niektóre systemu mogą obsługiwać statyczne dziedziczeni, inne mogą dynamiczne przypisywanie klasy.
   - Statyczne dziedziczenie (*Static inheritance):*
     - Określane na etapie kompilacji
     - Struktura dziedziczenia jest ustalona w kodzie źródłowym i nie może być zmieniona w czasie działania programu
     - Klasa podrzędna dziedziczy wszystkie publiczne i chronione cechy klasy nadrzędnej
     - Typ obiektu jest znany w czasie kompilacji i nie można go zmienić w czasie działania programu.
   - Dynamiczne przypisywanie klasy (*Dynamic Reclassification / Dynamic Typing*):
     - Klasa obiektu może zmieniać się w czasie działania programu
     - Wykorzystywane w językach dynamicznych (np. Python) lub poprzez mechanizmy polimorfizmu i rzutowania w językach statycznych (np. C++ z `dynamic_cast`)
     - Nie występuje w UML jako mechanizm dziedziczenia, ale można go modelować poprzez strategię lub dynamiczne zmiany klasyfikatora.

Dziedziczenie

W **OOAD** (*Object-Oriented Analisys and Design* – analiza i projektowanie obiektowe) dziedziczenie jest zazwyczaj definiowane jako mechanizm, dzięki któremu bardziej szczegółowe klasy (podklasy *subclasses* lub klasy pochodne *derived classes*) przejmują strukturę i zachowanie bardziej ogólnych klas (nadklas *superclasses* lub klas bazowych *base classes*)

W UML 1.4.2 dziedziczenie było wyjaśniane za pomocą pojęć **pełnego deskryptora (*full descriptor*)** oraz **deskryptora segmentowego (*segment descriptor*)**

Deskryptor (*descriptor*) to formalny opis obiektu lub elementu modelu, który zawiera wszystkie jego istotne właściwości, takie jak atrybuty, operacje, relacje i ograniczenia. W kontekście UML oraz obiektowego modelowania danych deskryptor pozwala określić pełną charakterystykę elementu w sposób hierarchiczny, często wykorzystując mechanizmy dziedziczenia.

Pełny deskryptor (*full descriptor*) – zawiera całkowity opis obiektu, obejmujący wszystkie jego cechy, także te odziedziczone po przodkach. Jest tworzony przez łączenie wielu segmentów dziedziczonych po kolejnych klasach w hierarchii.

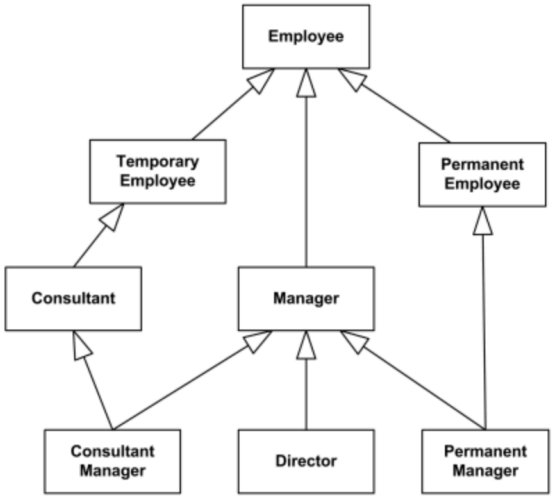
Deskryptor segmentowy (*segment descriptor*) – reprezentuje częściowy opis elementu, zawierający jedynie te cechy, które zostały bezpośrednio zadeklarowane w danej klasie lub innym klasyfikatorze.

Jeśli element podlegający generalizacji ma więcej niż jednego rodzica (**wielokrotne dziedziczenie *multiple inheritance***) to jego pełny deskryptor zawiera sumę cech z jego własnego deskryptora segmentowego oraz deskryptorów segmentowych wszystkich jego przodków.

Wielokrotne dziedziczenie

Jest domyślnie dozwolone przez standard UML, jednak **standard nie podaje jego formalnej definicji**.

W UML klasyfikator może mieć zero, jedną lub wiele relacji generalizacji od bardziej ogólnych klasyfikatorów. W OOAD wielokrotne dziedziczenie odnosi się do możliwości klasy do dziedziczenia zachowań i cech z więcej niż jednej klasy bazowej.



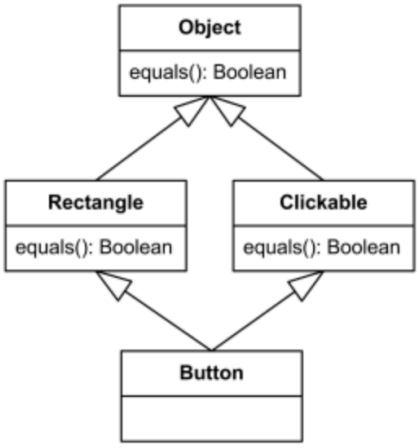
Pochodzenie wielokrotnego dziedziczenia

Wielokrotne dziedziczenie może wynikać z **ortogonalnych taksonomii**, które są ze sobą łączone. Na przykład, powyższy diagram przedstawia dwie różne klasyfikacje pracowników:

- Klasyfikacja według statusu zatrudnienia – pracownik może być stały lub tymczasowy.
- Klasyfikacja według stanowiska – pracownik może zajmować określoną pozycję, np. menedżera.

Problemy związane z wielokrotnym dziedziczeniem

Choć UML domyślnie pozwala na wielokrotne dziedziczenie, nie określa jednoznacznie, jak rozwiązywać znane problemy i niejednoznaczności, takie jak **problem diamentu (*diamond problem*)**.



W powyższym przykładnie klasa `Button` dziedziczy dwie różne implementacje metody `equals()`, ale nie definiuje własnej wersji tej metody.

Gdy wywołuje się `button.equals()`, nie jest jasne, która implementacja zostanie użyta – pochodząca z `Rectangle` czy z `Clickable`.

Rzeczą pozwalającą na doprecyzowanie semantyki generalizacji są **profile UML (*profiles*)**. Na przykład w profilu UML dla języka Java generalizacja klas powinna być ograniczona do **dziedziczenia pojedynczego**.

Zestaw generalizacji (*GeneralizationSet*)

Jest to mechanizm UML, który grupuje powiązane relacje generalizacji, aby wyrazić dodatkowe ograniczenia i semantykę.

Jest to **element pakowalny (*packageable element*)**, który pozwala na definiowanie hierarchii klasyfikacyjnych poprzez grupowanie niektórych relacji generalizacji dotyczących danego klasyfikatora ogólnego w podzbiory.

Każdy zbiór generalizacji może być również powiązany z klasyfikatorem zwanym **typem mocy (*powertype*)**.

Właściwości zbioru generalizacji:

- `isCovering` (pełny lub niepełny – *complete* lub *incomplete*)
- `isDisjoint` (rozłączny lub nakładający się – *disjoint* lub *overlapping*).

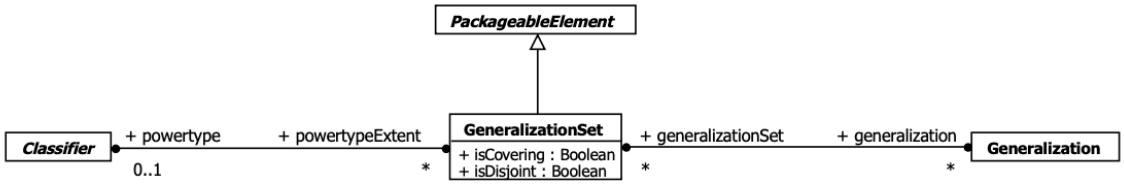
a) Właściwość `isCovering` (pełność):

- Określa, czy zestaw szczegółowych klasyfikatorów w zbiorze generalizacji jest kompletny.
- Jeśli zbiór generalizacji ma atrybut `{complete}`, oznacza to, że **każda instancja klasyfikatora ogólnego musi być także instancją przynajmniej jednego z klasyfikatorów szczegółowych**.
- Jeśli zbiór generalizacji ma atrybut `{incomplete}`, oznacza to, że **mogą istnieć instancje klasyfikatora ogólnego, które nie należą do żadnego klasyfikatora szczegółowego**.

b) Właściwość `isDisjoint` (rozłączność):

- Określa, czy klasyfikatory szczegółowe w zbiorze generalizacji mogą się **nakładać**.
- Jeśli zbiór generalizacji ma atrybut `{disjoint}`, to **żadna instancja jednego klasyfikatora szczegółowego nie może jednocześnie należeć do innego klasyfikatora szczegółowego** (klasyfikatory są rozłączne).
- Jeśli zbiór generalizacji ma atrybut `{overlapping}`, to **niektóre lub wszystkie klasyfikatory szczegółowe mogą współdzielić te same instancje**.

Składnia abstrakcyjna



`GeneralizationSet` – określa zbiór powiązań dziedziczenia. Posiada dwie istotne własności:

- `isCovering : Boolean` – określa, czy zestaw generalizacji pokrywa wszystkie możliwe przypadki (czyli czy każdy obiekt musi należeć do jednej z podklas)
- `isDisjoint : Boolean` – określa, czy klasy w zestawie generalizacji są rozłączne (obiekt może należeć do jednej, ale nie do kilku podklas jednocześnie).

`Classifier` – ogólna klasa UML reprezentująca klasyfikator, który może pełnić rolę typu nadrzędnego (superklasy):

- Ma on relację `powertype (0..1)`, co oznacza, że może, ale nie musi być typem nadrzędnym dla zestawu generalizacji.
- Posiada także `powertypeExtent`, który odnosi się do konkretnych instancji zestawu generalizacji.

`Generalization` – modeluje relację dziedziczenia pomiędzy klasyfikatorami:

- Każdy `GeneralizationSet` może zawierać wiele `Generalization`.
- Każdy `GeneralizationSet` jest powiązany z `GeneralizationSet`, co oznacza, że może należeć do więcej niż jednego zestawu.

Interpretacja i znaczenie elementów

a) `GeneralizationSet` pozwala grupować różne relacje dziedziczenia w logiczne zestawy, które mogą mieć określone własności (`isCovering`, `isDisjoint`).
b) `isCovering = true` oznacza, że każdy obiekt musi należeć do jednej z podklas
c) `isDisjoint = true` oznacza, że obiekt może należeć tylko do jednej podklasy
d) `powertype` wskazuje klasyfikator, który definiuje zestaw podtypów, co może być używane np. do dynamicznego określania typów w modelu.

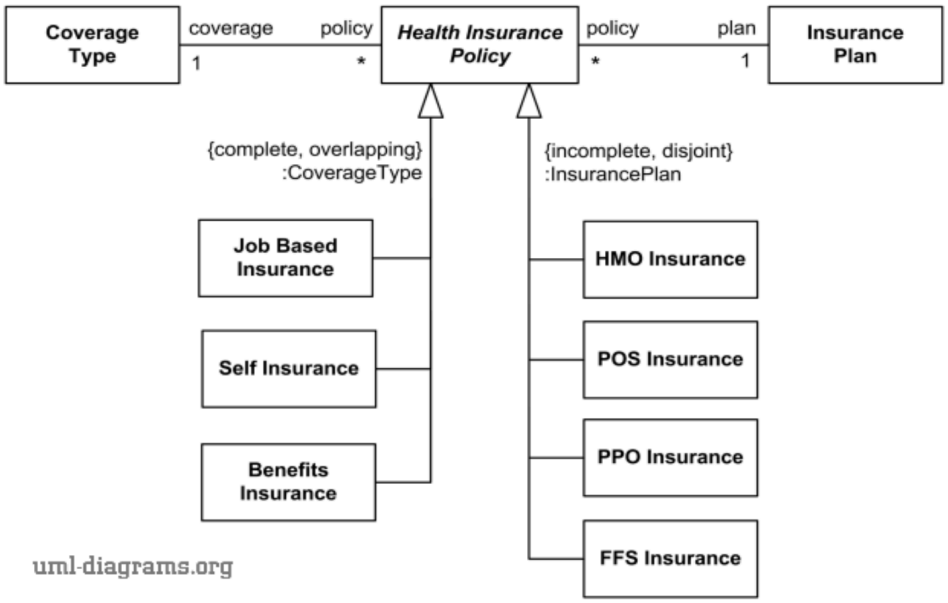
Typ mocy (*powertype*)

Zbiór generalizacji może opcjonalnie być powiązany z klasyfikatorem zwanym typem mocy:

a) Instancje typu mocy mogą być traktowane jako **semantycznie równoważne** każdemu z odpowiadających im klasyfikatorów szczegółowych.
b) UML nie określa, **jak należy implementować tę semantyczną równoważność** ani jak utrzymywać jej spójność.

Notacja na diagramie UML

- Ograniczenia zbioru generalizacji (`{complete}`, `{incomplete}`, `{disjoint}`, `{overlapping}`) umieszcza się obok zbioru, **blisko wspólnej strzałki generalizacji** lub **w pobliżu przerywanej linii** reprezentującej zbiór generalizacji.
- Specyfikacja typu mocy jest przedstawiona w postaci **dwukropka `(:)`, po którym następuje nazwa klasyfikatora typu mocy**, umieszczonego w pobliżu odpowiadającego mu zbioru generalizacji.



1.3 Asocjacja: podstawowe właściwości

Asocjacja jest relacją między klasyfikatorami, która służy do pokazania, że instancje klasyfikatorów mogą być ze sobą połączone logicznie lub fizycznie w pewną agregację (**oznacza istnienie trwałego powiązania między klasyfikatorami**).
Specyfikacja UML klasyfikuje asocjację jako relację semantyczną.

Asocjacja może być używana w różnych typach diagramów strukturalnych UML:

a) Diagram klas (asocjacje między klasami - *class diagram associations*)
b) Diagram przypadków użycia (asocjacje między aktorami a przypadkami użycia – *use case diagram associations*)
c) Diagram wdrożenia (asocjacje artefaktów – *deployment diagram artifact associations*)
d) Diagram wdrożenia (ścieżki komunikacji – *deployment diagram communication path*)

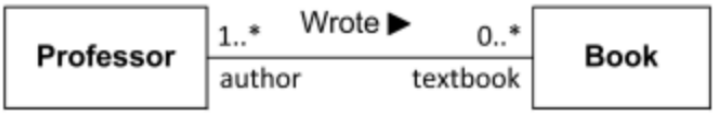
Pojęcia związane z asocjacją:

- Własność końców asocjacji (*association end ownership*)
- Nawigowalność (*navigability*)

- Arność asocjacji (*association arity*)
- Typ agregacji (*aggregation type*)

Notacja w UML
Asocjacja jest zazwyczaj rysowana jako **ciągła linia łącząca dwa klasyfikatory** lub **pojedynczy klasyfikator sam ze sobą** (asocjacja rekurencyjna).
Nazwa asocjacji może być umieszczona **blisko środka linii**, ale nie zbyt blisko żadnego z końców.
Każdy koniec linii może być **oznaczony nazwą roli** dla danego klasyfikatora.

Przykład:



Asocjacja `Wrote` („Napisał”) między klasami `Professor` i `Book`, gdzie końce asocjacji opisano jako `author` (autor) i `textbook` (podręcznik).

<u>Końce asocjacji (*Association ends)*</u>
Koniec asocjacji to połączenie między linią przedstawiającą asocjację a ikoną reprezentującą powiązany klasyfikator. Nazwa końca asocjacji może być umieszczona w pobliżu końca linii i jest często określana jako **nazwa roli**. Nazwa roli jest opcjonalna i może zostać pominięta.

Bazując na poprzednim przykładzie:
Profesor (`Professor`) „pełniący rolę” autora (`author`) jest powiązany z końcem podręcznik (`textbook`), który jest typu Książka (`Book`).

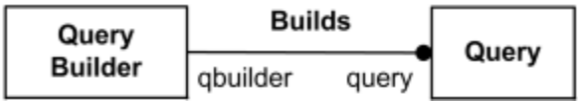
Idea roli polega na tym, że ten sam klasyfikator może odgrywać tę samą lub różne role w innych asocjacjach. Na przykład `Professor` może być autorem niektórych książek lub ich redaktorem.

Koniec asocjacji może być własnością:
- Klasyfikatora końcowego (*end classifier*)
- Samej asocjacji.

Końce asocjacji, które mają **więcej niż dwa końce** muszą **należeć do asocjacji.**

Własność końców asocjacji przez powiązany klasyfikator może być przedstawiona graficznie za pomocą **małego wypełnionego kółka (lub kropki**). Kropka jest rysowana w miejscu, gdzie linia spotyka się z klasyfikatorem. Można ją interpretować jako wskazanie,

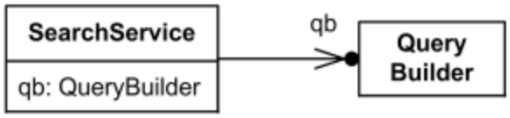
że model zawiera właściwość typu reprezentowanego przez klasyfikator, do którego kropka przylega. Właściwość ta jest własnością klasyfikatora **na drugim końcu**.



Koniec asocjacji `query` jest własnością klasyfikatora `QueryBuilder`, a koniec asocjacji `qbuilder` jest własnością samej asocjacji `Builds`.

Notacja „własności" za pomocą kropi może być używana w połączeniu z innymi graficznymi oznaczeniami właściwości asocjacji i jej końców, takimi jak typ agregacji i nawigowalność.

Standard UML nie wymaga stosowania jawnej notacji własności końców asocjacji, ale definiuje notację, która ma być stosowana w modelach, w których taka konwencja jest przyjęta. Notacja kropi musi być stosowana na poziomie pełnych asocjacji lub wyżej, tak aby brak kropki oznaczał własność asocjacji.
Dla końca asocjacji, który jest własnością klasy można użyć notacji atrybutu, ponieważ koniec asocjacji należący do klasy jest również atrybutem. Ta notacja może być używana w połączeniu z notacją strzałki liniowej, aby jednoznacznie wskazać, że atrybut jest również końcem asocjacji.



Koniec asocjacji `qb` jest atrybutem klasy `SearchService` i należy do tej klasy.

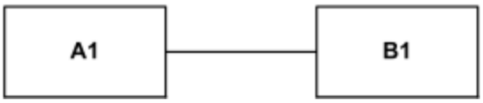
Nawigowalność (*Navigability*)
Właściwość końcowa asocjacji jest nawigowalna z przeciwnych końców asocjacji, jeśli instancje klasyfikatora na tym końcu **połączenia** (***link***) mogą być efektywnie dostępne w czasie działania z instancji na innych końcach połączenia.

Gdy właściwość końcowa asocjacji jest oznaczona jako nienawigowalna, oznacza to, że dostęp z innych końców może, ale nie musi być możliwy, a jeśli jest, to może być nieefektywny.

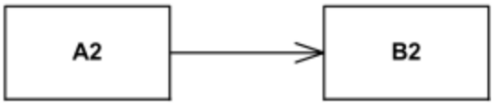
Notacja w diagramach UML
Nawigowalny koniec jest oznaczony **otwartym grotem strzałki** na końcu asocjacji.
Nienawigowalny koniec jest oznaczony **małym „x"** na końcu asocjacji.
Brak oznaczenia na końcu asocjacji oznacza nieokreśloną nawigowalność.

Przykłady:

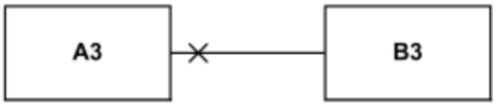
a) **Nieokreślona nawigowalność** – brak oznaczeń – oba końce mają nieokreśloną nawigowalność



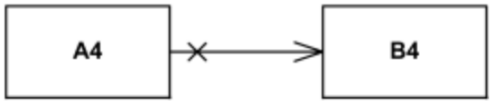
b) **`A2` ma nieokreśloną nawigowalność, a `B2` jest nawigowalne**



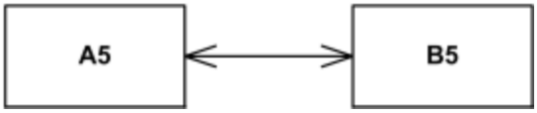
c) **A3 jest nienawigowalne, a `B3` ma nieokreśloną nawigowalność**



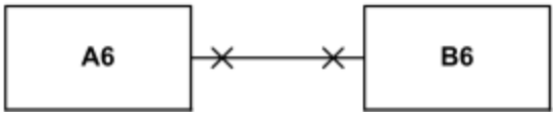
d) **A4 jest nienawigowalne z `B4`, a `B4` jest nawigowalne z `A4`**



e) **A5 jest nawigowalne z `B5` i `B5` jest nawigowalne z `A5`**



f) **A6 jest nienawigowalne z `B6` i `B6` jest nienawigowalne z `A6`**

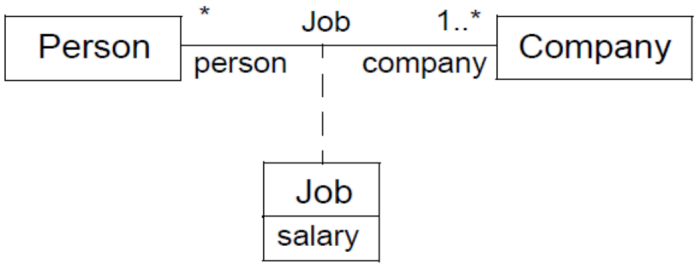


Klasa asocjacji (*Association class*)

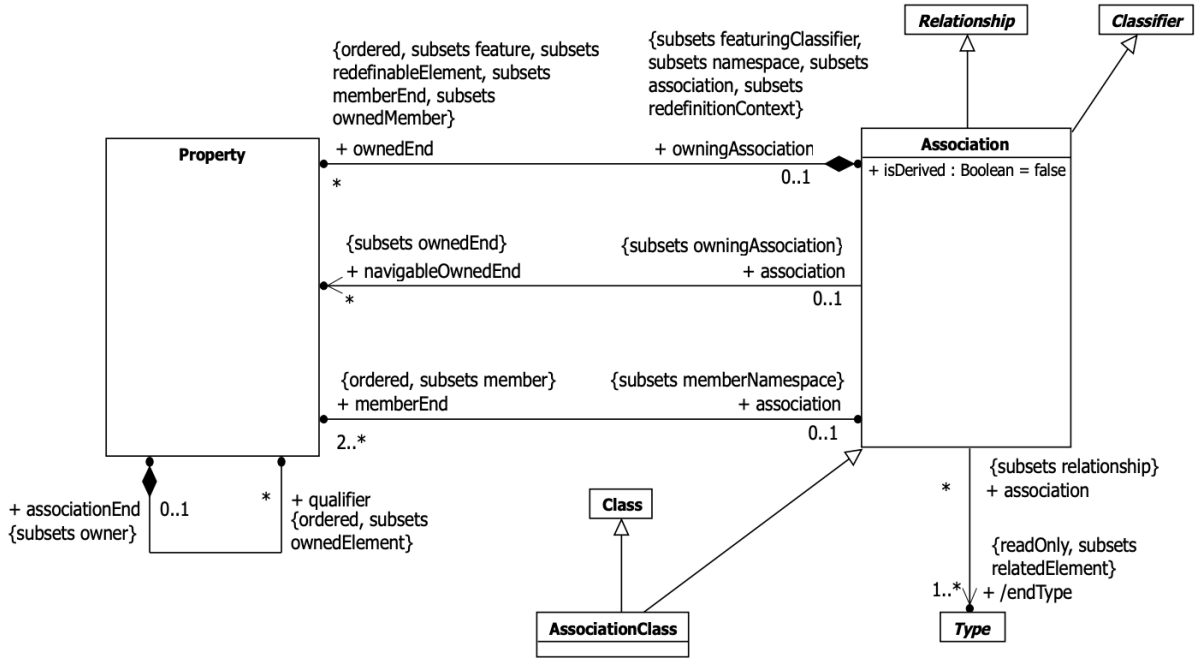
Asocjacja może być rozszerzona o własny zestaw cech, czyli cechy, które nie należą do żadnego z połączonych klasyfikatorów, lecz do samej asocjacji. Jest to **klasa asocjacji** – jest ona jednocześnie asocjacją, łączącą zestaw klasyfikatorów oraz klasą, co oznacza, że może posiadać swoje własne cechy i może być uwzględniona w innych asocjacjach.

Klasę asocjacji można traktować jako asocjację, która ma również właściwości klasy lub jako klasę, posiadającą właściwości asocjacji.

Klasa asocjacji jest przedstawiana jako symbol klasy połączony z ścieżką asocjacji **przerywaną linią**. Ścieżka asocjacji oraz symbol klasy asocjacji reprezentują ten sam element modelu i mają **jedną wspólną nazwę**. Nazwa może być umieszczona zarówno na ścieżce asocjacji, w symbolu klasy, jak i w obu miejscach jednocześnie, ale w każdym przypadku musi być taka sama.



Składnia abstrakcyjna



`Association` – reprezentuje związek między dwoma lub więcej klasyfikatorami:

- Ma atrybut `isDerived : Boolean = false`, co oznacza, że domyślnie asocjacja nie jest pochodna (nie wynika z innych zależności).
- Każda asocjacja może mieć wiele końców (`endType`), co oznacza, że może łączyć więcej niż dwie klasy (`n-ary association`).

`Property` (Własność) – reprezentuje końce asocjacji:

- `ownedEnd` – wskazuje, które końce są własnością asocjacji
- `navigableOwnedEnd` – określa, które końce są nawigowalne, czyli które można śledzić w systemie.
- `memberEnd` – lista wszystkich końców asocjacji.

`Type` – określa klasyfikatory powiązane przez asocjację:

- Każda asocjacja ma co najmniej dwa końce (`1.. /endType*`), które są instancjami klasyfikatorów

`AssociationClass` – specjalny przypadek asocjacji, który łączy cechy klasy i asocjacji:

- Może przechowywać własne atrybuty i operacje, ale nadal reprezentuje związek między klasyfikatorami.

Własności asocjacji:

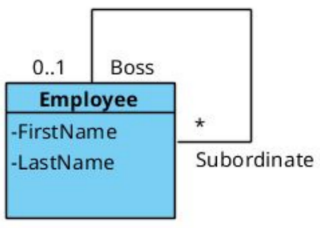
a) `Multiplicity` (Liczność / mnożność) – określa, ile instancji danej klasy może być powiązanych z inną klasą poprzez daną asocjację.
b) `Navigability` (Nawigowalność) – jeśli koniec asocjacji jest oznaczony jako `navigable`, oznacza to, że można do niego uzyskać dostęp z drugiego końca relacji.
c) `Directionality` (Kierunkowość) – może być jedno- lub dwukierunkowa, określając, w jakim kierunku można śledzić powiązanie.
d) Związek z `Property` – każdy koniec asocjacji jest reprezentowany jako `Property`, co oznacza, że może posiadać dodatkowe atrybuty.

1.4 Asocjacja: zaawansowane aspekty (asocjacja rekurencyjna, asocjacja n-arna)

Asocjacja rekurencyjna – jest to szczególny rodzaj asocjacji, w której klasyfikator jest powiązany sam ze sobą. Oznacza to, że instancje tej samej klasy mogą być powiązane w określony sposób. W UML asocjacja rekurencyjna jest reprezentowana jako połączenie między dwiema rolami tej samej klasy.

Przykłady rzeczywistych przypadków użycia asocjacji rekurencyjnej:

- Rodzina: jedna osoba klasy `Person` może być rodzicem innej osoby (również `Person`).
- Struktura folderów: folder (klasy `Folder`) może zawierać inne foldery (klasy `Folder`).
- Hierarchia pracowników w firmie: jeden pracownik (klasy `Employee`) może być przełożonym innego pracownika (również `Employee`).



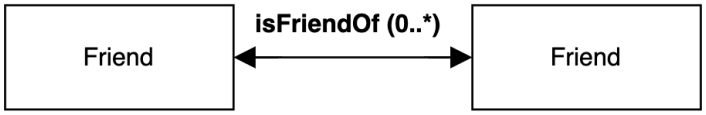
Notacja w UML
Asocjację rekurencyjną w UML rysuje się jako linia łącząca jeden klasyfikator ze sobą samym. Aby rozróżnić różne role instancji tej samej klasy w tej relacji, stosuje się nazwy ról na końcach asocjacji.

Mnożność (*multiplicity*)
Mnożność określa liczbę instancji biorących udział w relacji.

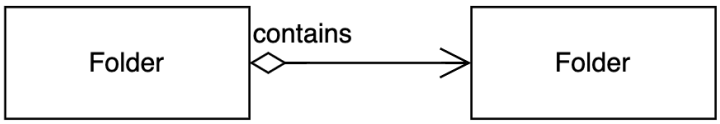
- W przypadku hierarchii pracowników: `1` (każdy pracownik ma jednego przełożonego) oraz `0..*` (przełożony może mieć dowolną liczbę podwładnych.)
- W przypadku struktury folderów: `0..1` (folder może mieć co najwyżej jeden folder nadrzędny) oraz `0..*` (folder może zawierać dowolną liczbę podfolderów).

Rodzaje asocjacji rekurencyjnej (*recursive association*):

a) **Asocjacja zwykła**
Najprostsza forma, w której instancje tej samej klasy są powiązane bez żadnych dodatkowych ograniczeń, np. osoby będące znajomymi w sieci społecznościowej.

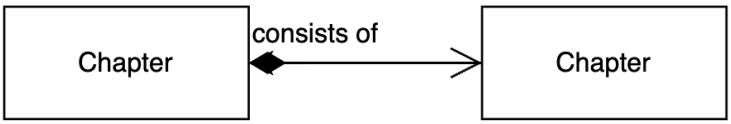


b) **Agregacja rekurencyjna**
Agregacja oznacza częściową **relację całość-część**, w której jeden obiekt jest „częścią" innego, np. folder zawierający inne foldery.



Tutaj symbol **pustego rombu** oznacza **agregację** – folder może zawierać inne foldery, ale te podfoldery mogą istnieć niezależnie.

c) **Kompozycja rekurencyjna**
Kompozycja jest silniejszą formą agregacji – oznacza, że podrzędne instancje **nie mogą istnieć bez nadrzędnej instancji**, np. rozdziały książki składają się z podrozdziałów



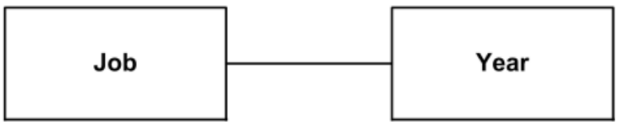
Tutaj **wypełniony romb** oznacza, że usunięcie nadrzędnego rozdziału spowoduje usunięcie jego podrzędnych rozdziałów.

Zastosowanie asocjacji rekurencyjnej
Asocjacje rekurencyjne są szeroko stosowane w programowaniu obiektowym oraz w modelowaniu danych. Można je znaleźć w:

- Hierarchicznych strukturach danych (np. drzewa, organizacje, kategorie produktów)
- Grach komputerowych (np. jednostki wojskowe dowodzące innymi jednostkami)
- Sieciach społecznościowych (relacje „znajomy", „obserwujący" itp.)
- Systemach zarządzania treścią (np. powiązane artykuły, drzewa komentarzy).

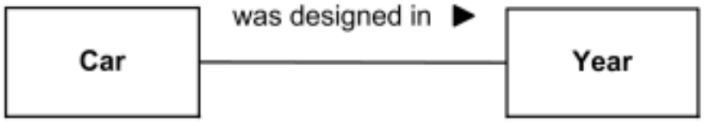
Asocjacja binarna (*binary association*)
Każda asocjacja ma określoną **krotność (*arity*),** ponieważ może łączyć dwa lub więcej elementów.

Asocjacja binarna łączy dwie typowane instancje. Zazwyczaj jest przedstawiana jako ciągła linia łącząca dwa klasyfikatory lub jako linia łącząca pojedynczy klasyfikator sam ze sobą (przy czym oba końce są odrębne). Linia może składać się z jednego lub więcej połączonych segmentów.



Klasyfikatory `Job` i `Year` są powiązane.

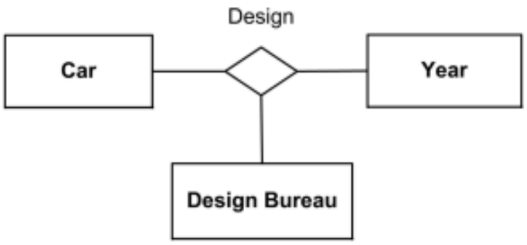
Obok nazwy asocjacji binarnej lub zamiast niej można umieścić **mały wypełniony trójkąt**, aby pokazać kolejność końców asocjacji. Strzałka wskazuje kierunek od pierwszego końca do ostatniego w określonej kolejności.



Czytanie asocjacji: `Car` → `was designed in` → `Year`.

Asocjacja n-arna (*n-ary association*)
Każda asocjacja może być przedstawiona jako **romb** (większy niż zakończenie linii) z ciągłą linią dla każdego końca tej asocjacji, łączącą romb z klasyfikatorem, który jest typem tego końca.

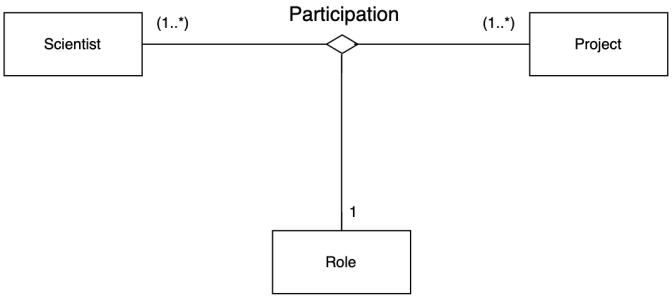
Asocjacja n-arna z więcej niż dwoma końcami **musi** być rysowana w ten sposób.



Przykład **asocjacji trójargumentowej (*ternary association*)** – asocjacja `Design` łączy trzy klasyfikatory: `Car`, `Year` i `Design Bureau`.

Każdy koniec asocjacji n-arnej może mieć określoną mnożność (*multiplicity*), podobnie jak w asocjacji binarnej:

- Jeden naukowiec może uczestniczyć w kilku projektach.
- Każdy projekt może mieć wielu naukowców.
- Każdy naukowiec w ramach projektu może pełnić dokładnie jedną rolę.



W przeciwieństwie do asocjacji binarnej, asocjacja n-arna:

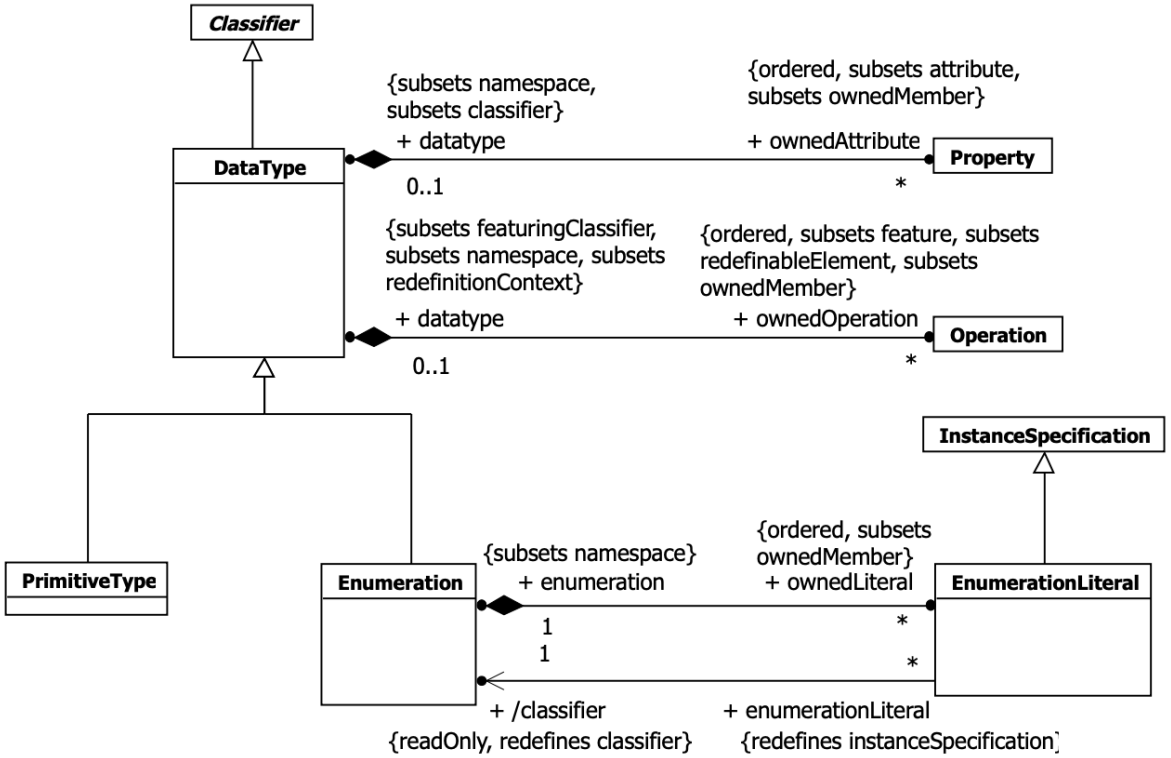
- Definiuje relację, która dotyczy **więcej niż dwóch** obiektów jednocześnie.
- Nie można jej rozłożyć na wiele asocjacji binarnych bez utraty znaczenia.
- Określa, że dana instancja asocjacji istnieje tylko wtedy, gdy są spełnione wszystkie połączenia na raz.

C03 1.5-1.7

1.5 Typ danych

Typ danych (**`DataType`**) jest klasyfikatorem. Różni się od `Class` tym, że jego instancje są identyfikowane tylko przez ich wartość (nie posiadają tożsamości). Wszystkie instancje `DataType` o tej samej wartości traktowane są jako równe instancje (nierozróżnialne).
Jeśli `DataType` posiada atrybuty (czyli instancje `Property`), używa się nazwy *structured* `DataType`. Instancje *structured* `DataType` są traktowane jako równe, wtedy i tylko wtedy, gdy ich struktura jest taka sama a wartości atrybutów są równe.
Ponadto, `DataType` nie posiada stanu, co oznacza, że po utworzeniu atrybutów nie można zmieniać ich wartości.

Składnia abstrakcyjna



`DataType` jako podtyp `Classifier`

`DataType` jest podtypem `Classifier`, co oznacza, że może mieć atrybuty (`Property`), ale ich wartości są kopiowane zamiast odwoływane.
`DataType` może mieć operacje (`Operation`), ale nie przechowuje stanu obiektu.
Nie posiada tożsamości obiektu – dwa obiekty o tych samych wartościach są nieodróżnialne.

Istotne cechy:

- `datatype` – `DataType` może być powiązany z innym `DataType`, tworząc hierarchię.
- `ownedAttribute` – lista atrybutów (`Property`), które definiują właściwości przechowywane w `DataType`.
- `ownedOperation` – operacje (`Operation`), które można wykonywać na wartościach `DataType`.

Podklasy `DataType`:

a) `PrimitiveType` – typy prymitywne:
   - Reprezentuje proste typy danych, jak `Integer`, `Double` czy `String` lub `Boolean`.
   - Zwykle są one predefiniowane w językach programowania.

b) `Enumeration` – typy wyliczeniowe:
   - Definiuje enumeracje, czyli zbiór skończonych wartości (instancji `EnumerationLiteral`).
   - Przykład: `Kolory = {Czerwony, Zielony, Niebieski}`.

Powiązane elementy UML:

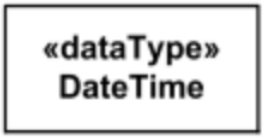
a) `Property`:
   - Atrybuty przechowywane w `DataType` (np. długość napisu w `String`).
   - `{ordered, subsets attribute, subsets ownedMember}` oznacza, że atrybuty mogą być uporządkowane i są częścią `ownedMember`.

b) `Operation` – `DataType` może mieć operacje, ale bez zmiennych instancji.

c) `InstanceSpecification` → `EnumerationLiteral`:
   - Instancje `Enumeration` są reprezentowane jako `EnumerationLiteral`, czyli są wartościami dostępnymi w enumeracji.
   - `InstanceSpecification` (ogólny mechanizm reprezentowania instancji dowolnego `Classifier`) może wskazywać na `EnumerationLiteral`, tworząc instancję `Enumeration`.
   - Zapis `{redefines instanceSpecification}` oznacza, że `EnumerationLiteral` dziedziczy zachowanie `InstanceSpecification`.

<u>Notacja w UML</u>

`DataType` w UML oznaczany jest jako prostokąt ze słowem kluczowym **«dataType»** (w cudzysłowie francuskim):

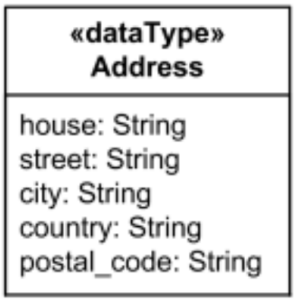


Typ danych `DateTime`.

Dla typów prymitywnych dodaje się słowo kluczowe **«primitive»:**

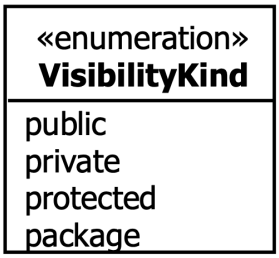


Jeśli typ danych zawiera atrybuty lub operacje, używana jest notacja podobna jak dla `Class`:



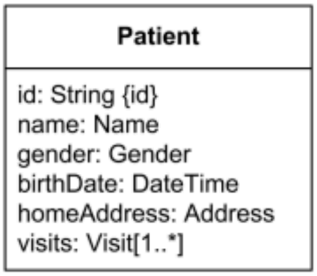
Ustrukturyzowany typ danych `Address`.

Jeżeli typ danych jest enumeracją, to potrzebne jest słowo kluczowe **«enumeration»**:



Enumeracja `VisibilityKind`.

Gdy stosowane jest odwołanie do typu danych, np. jako typu atrybuty danej klasy, może być on przedstawiony po prostu jako nazwa typu danych:



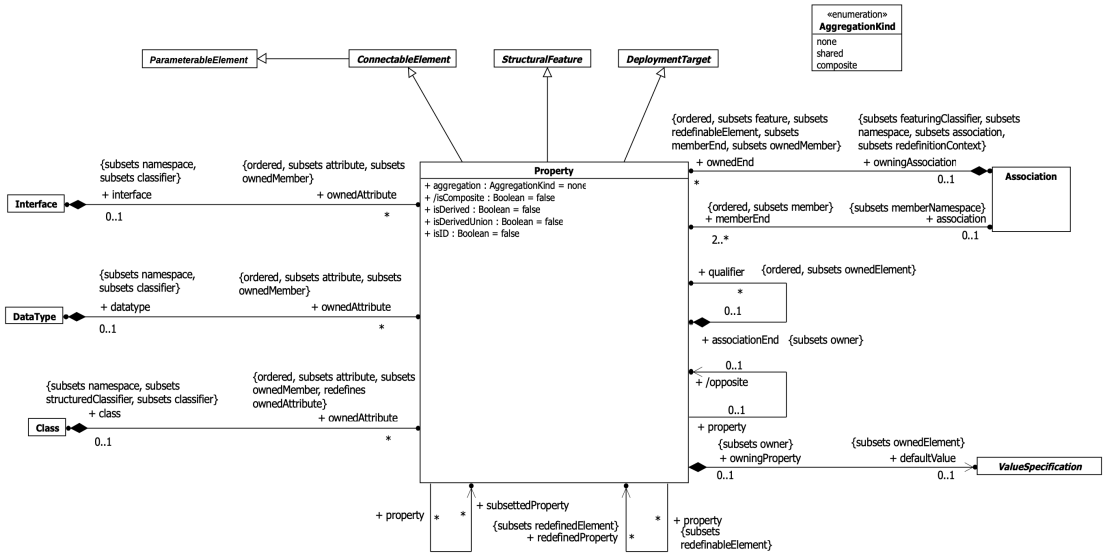
Atrybuty klasy `Patient` mają typy danych: `Name`, `Gender`, `DateTime`, `Address` i `Visit`.

1.6 Właściwość statyczna (w tym *read only*)

Definicja

`Property` (właściwość) to `StructuralFeature` (cecha strukturalna – określa, że dany element może przechowywać wartość lub odnosić się do innego elementu), która reprezentuje atrybuty `Class` (i `Classifier`), `memberEnds` (końce) dla `Association` i części `StructuredClassifier` (klasyfikator strukturalny – abstrakcyjna klasa, reprezentująca. klasyfikatory posiadające strukturę wewnętrzną).
Innymi słowy, `Property` jest jednym z podstawowych elementów używanych do definiowania atrybutów i relacji między obiektami.

Składnia abstrakcyjna



Definicja i hierarchia `Property`

`Property` jest podtypem kilku innych metaklas UML:

a) `StructuralFeature` – oznacza, że `Property` jest cechą strukturalną klasy lub innego elementu UML.
b) `ConnectableElement` – pozwala `Property` być używanym w interakcjach i modelach połączeń (może np. reprezentować porty lub składowe obiektu, które są połączone z innymi elementami).
c) `DeploymentTarget` – oznacza, że `Property` może być miejscem docelowym wdrożenia (może wskazywać np. fizyczną maszynę, na której jest wdrażana aplikacja).
d) `ParametrableElement` – pozwala `Property` występować jako parametr w szablonach UML.

Kluczowe właściwości `Property`

a) Agregacja i kompozycja: `+ aggregation: AggregationKind = none`
`AggregationKind` może przyjmować wartości:
   – `none` – zwykła asocjacja.

- `shared` – agregacji, elementy mogą istnieć niezależnie.
- `composite` – kompozycja, elementy są częścią większej całości.

b) Kompozycja i tożsamość:
- `+ /isComposite: Boolean = false` – oznacza, że domyślnie `Property` nie jest częścią kompozycji.
- `+ isID: Boolean = false` – oznacza, że `Property` nie jest unikalnym identyfikatorem obiektu.

c) Dziedziczenie i redefinicja:
- `+ isDerived: Boolean = false` – oznacza, że wartość atrybutu nie jest obliczana na podstawie innych wartości.
- `+ isDerivedUnion: Boolean = false` – oznacza, że `Property` nie jest automatycznie sumowane z innych podtypów.

d) Wartość domyślna:
`+ defaultValue: ValueSpecification` – oznacza, że `Property` może mieć wartość domyślna określoną przez `ValueSpecification`.

Powiązania `Property` z innymi elementami UML

a) Property może należeć do `Class`, `Interface` lub `DataType` jako atrybut:
- `+ class 0..1`
- `+ interface 0..1`
- `+ datatype 0..1`

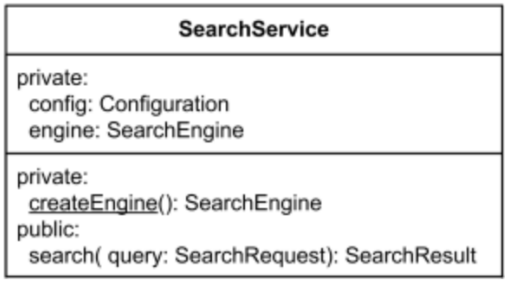
b) Końce asocjacji:
- `+ memberEnd 2..*` - określa, że `Property` może być jednym z końców asocjacji i musi mieć co najmniej dwóch uczestników.
- `+ assocationEnd 0..1 {subsets owner}` – jeśli `Property` jest częścią asocjacji, dziedziczy właściciela.

c) Odwrotność relacji:
- `+ opposite 0..1` – jeśli `Property` reprezentuje asocjację, może mieć wskazane pole `opposite`, które definiuje atrybut w drugiej klasie, będący odwrotnością tej relacji.

Statyczność i niemutowalność (*static* i *read-only*)

`Property` może mieć atrybuty określające jej statyczność oraz niemutowalność.
Właściwość statyczna oznacza, że atrybut nie jest związany z konkretną instancją klasy, lecz samą klasą jako całością, czyli zmiana jego wartości wpływa na całą klasę, a nie na pojedynczy obiekt. W UML statyczne właściwości są oznaczane poprzez podkreślenie nazwy atrybutu w diagramie klas.

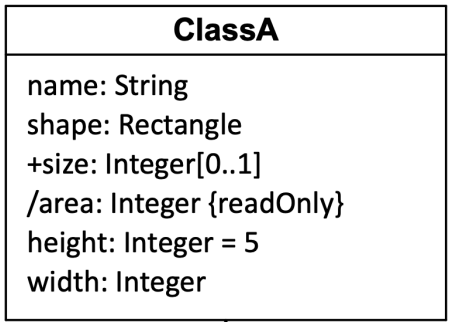
Przykład notacji:



Operacja `createEngine(): SearchEngine` jest statyczna.

Właściwość *read-only* oznacza, że atrybut nie może być zmieniany po inicjalizacji (w praktyce odpowiada stałym atrybutom – `const` w C++ lub `final` w Javie). W UML taka właściwość jest oznaczana przez modyfikator `{readOnly}`.

Przykład notacji:

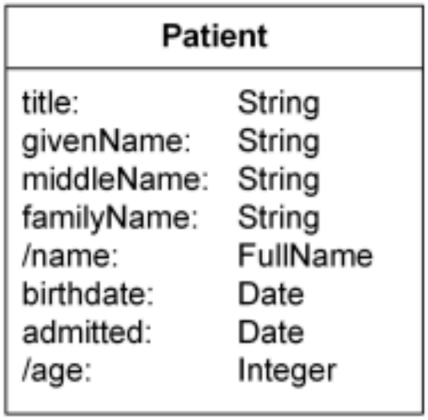


Atrybut `/area: Integer` klasy `ClassA` jest niemutowalny. Ponadto symbol `/` oznacza, że jest to atrybut pochodny / wnioskowany (*derived property*) – jego wartość wynika z innych atrybutów lub relacji. Jego wartość może być wyrażona jako funkcja zależna od innych właściwości klasy.

1.7 Właściwość wnioskowana (*derived property*)

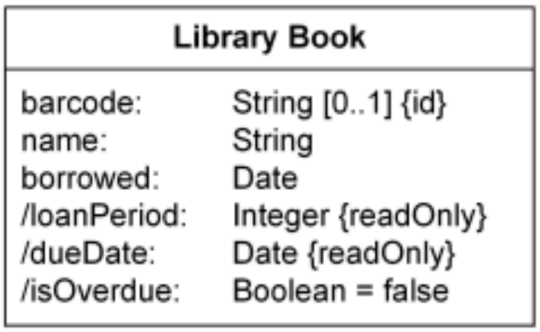
Właściwość wnioskowana to atrybut lub relacja, której wartość nie jest bezpośrednio przechowywana w obiekcie, lecz jest obliczana na podstawie innych atrybutów, relacji lub operacji (czyli jest obliczana **dynamicznie**, bez zapisywania w pamięci ani bazie danych). Często właściwości wnioskowane są stosowane w celu unikania duplikacji danych oraz zapewnienia, że wartości są zawsze zgodne z innymi atrybutami w systemie.

Atrybut wnioskowany na diagramach UML oznaczany jest symbolem `/`.



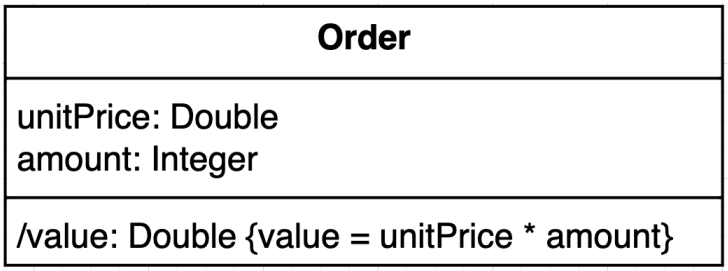
Atrybuty `/name: FullName` i `/age: Integer` klasy `Patient` są atrybutami wnioskowanymi.

Zdarza się, że właściwości wnioskowane oznaczane są jako `{readOnly}`, w celu uniemożliwienia zmiany wartości atrybutu bezpośrednio (ręcznie).



Atrybuty `/loanPeriod: Integer` oraz `/dueDate: Date` klasy `Library Book` są oznaczone jako `{readOnly}`.

Czasami spotyka się także regułę obliczeniową dla danego atrybutu, umieszczoną w nawiasach klamrowych `{}`.



C04 1.8-1.10

1.8 Krotności i liczności

W UML krotność (*multiplicity*) i liczność (*cardinality*) określają, ile instancji jednej klasy może być powiązanych z instancją innej klasy w relacji asocjacji, agregacji lub kompozycji.

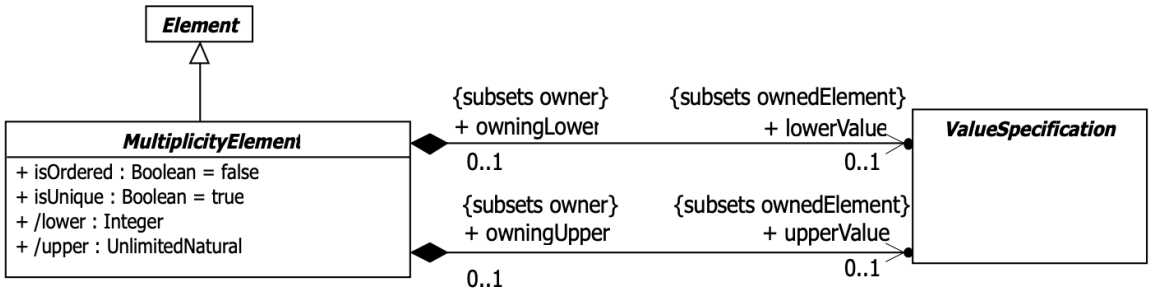
Krotność

Krotność określa minimalną i maksymalną liczbę instancji obiektów, które mogą uczestniczyć w danej relacji. Jest określona przez przedział zawierający nieujemne liczby całkowite.

Można wyróżnić dolną i (możliwie skończoną) górną granicę. Granice te mogą być stałymi wartościami lub wyrażeniami, które są obliczane jako liczby naturalne. Górna granica może być również oznaczona symbolem `*`, który oznacza nieograniczoną liczbę elementów.

Składnia abstrakcyjna

W UML krotność jest modelowana przez abstrakcyjną klasę `MultiplicityElement`, która jest używana przez różne elementy modelu UML, np. `Property`, `Parameter` czy `AssociationEnd`.



`MultiplicityElement` jest klasą abstrakcyjną, dziedziczącą po `Element`, definiującą mechanizm określania krotności.

Atrybuty:

- `isOrdered: Boolean = false` – określa, czy elementy w danym zbiorze są uporządkowane (`true`) czy nie (`false`).
- `isUnique: Boolean = true` – określa, czy wartości są unikalne (`true` – brak duplikatów).
- `/lower: Integer` – dolna granica krotności (np. dla `0..*` wartość `lower` wynosi `0`)
- `/upper: UnlimitedNatural` – górna granica krotności (np. dla `0..*` wartość upper wynosi `*`).

Jak można zauważyć, atrybuty `lower` i `upper` są wnioskowane (symbol `/`).

`ValueSpecification:`

Jest to klasa reprezentująca wyrażenie określające wartość, np. `1`, `*`, `3 + 2`, czy `someVariable`.

`MultiplicityElement` jest połączony z `ValueSpecification` za pomocą dwóch asocjacji (kompozycji):

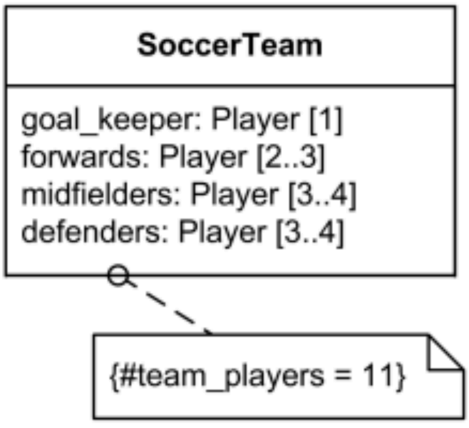
a) `lowerValue: ValueSpecification [0..1]:`
   - Określa wartość dolnej granicy krotności (`lower`).
   - Liczność `0..1` oznacza, że ta wartość może, ale nie musi być określona.
   - Oznaczone jako `{subsets ownedElement}`, co oznacza, że należy do podzbioru `ownedElement` (czyli jest częścią posiadanego elementu).

b) `upperValue: ValueSpecification [0..1]:`
   - Określa wartość górnej granicy krotności (`upper`).
   - Liczność `0..1` oznacza, że może być obecna, ale nie musi.
   - Tak jak `lowerValue`, jest podzbiorem `ownedElement`.

Przykłady:

- `0..1` – opcjonalna instancja (najwyżej jedna).
- `0..0` – brak instancji.
- `1..1` – dokładnie jedna instancja.
- `0..*` - dowolna liczba instancji (od zera do nieskończoności).
- `1..*` - co najmniej jedna instancja.
- `3..5` – od trzech do pięciu instancji.

Jeśli krotność jest powiązana z elementem, którego notacja jest ciągiem tekstowym (np. atrybutem klasy), zakres krotności jest umieszczany w nawiasach kwadratowych jako część tego ciągu tekstowego.

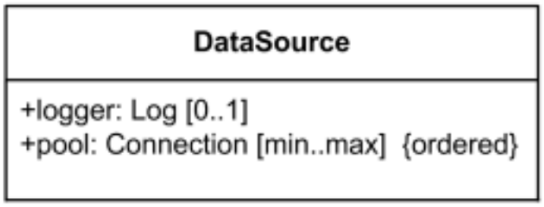


Krotności zawodników (`Players`) w drużynie piłkarskiej (`SoccerTeam`).

Jeśli element krotności jest wielowartościowy i określony jako uporządkowany `{ordered}`, to kolekcja wartości w instancji tego elementu jest sekwencyjnie uporządkowana. Domyślnie kolekcje nie są uporządkowane.

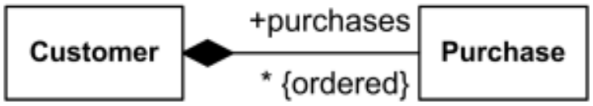
Jeśli element krotności jest wielowartościowy i określony jako unikalny `{unique}`, to każda wartość w kolekcji wartości w instancji tego elementu musi być unikalna. Domyślnie każda wartość w kolekcji jest unikalna.

Przykład uporządkowanej krotności dla atrybutów klasy:



`DataSource` może posiadać `logger: Log` oraz uporządkowaną pulę `pool: Connections` o liczbie od minimalnej do maksymalnej wartości. Każde `Connection` jest unikalne (domyślnie).

Przykład uporządkowanej krotności dla powiązań:



`Customer` może mieć od zera do wielu `purchases`. Instancje `Purchase` w kolekcji mają określoną kolejność i każdy z nich jest unikalny (domyślnie).

Liczność

Liczność określa liczbę instancji, które mogą być powiązane z danym elementem modelu, takim jak atrybut klasy, parametry operacji czy asocjacje między klasami. Jest to **szczególny przypadek krotności**, który odnosi się konkretnie do liczby dozwolonych wystąpień elementu.

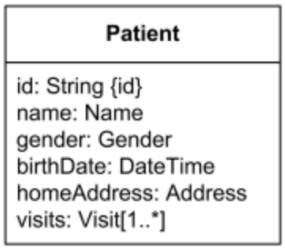
Krotność jest ograniczeniem dla liczności, która nie może być mniejsza niż dolna granica oraz większa niż górna granica krotności

1.9 Właściwość identyfikująca obiekty klasy (ograniczenie ID)

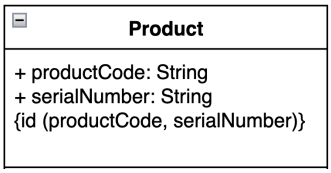
W UML istnieje możliwość oznaczenia jednej lub kilku właściwości klasy jako unikalnego identyfikatora poprzez zastosowanie ograniczenia `{id}`. Takie oznaczenie wskazuje, że dana właściwość (lub ich kombinacja) służy do jednoznacznej identyfikacji instancji tej klasy.

Ograniczenie {id} umieszcza się w nawiasach klamrowych obok atrybutu klasy, który pełni funkcję unikalnego identyfikatora. Jeśli identyfikator składa się z kilku atrybutów, zapisuje się je w nawiasach okrągłych po {id}.

Przykłady:

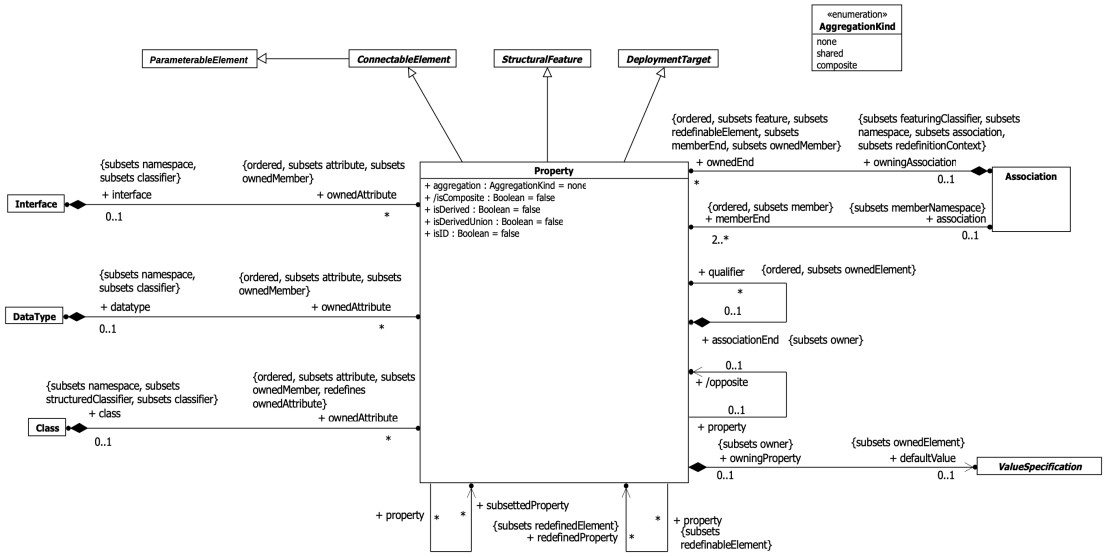


Atrybut `id: String` klasy `Patient` ma ograniczenie `{id}`.



Kombinacja atrybutów `productCode: String` i `serialNumber: String` daje identyfikator złożony dla instancji klasy `Product`.

Składnia abstrakcyjna



Składnia abstrakcyjna `Property` była już omawiana wcześniej. Skupmy się jedynie na atrybucie `isID: Boolean` i jego znaczeniu.

Atrybut ten domyślnie ma wartość `false` i informuje, czy dana właściwość jest częścią unikalnego identyfikatora danej klasy:

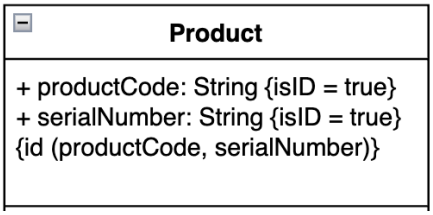
- Gdy `isID = true`, oznacza to, że dana właściwość jest częścią identyfikatora klasy.
- Gdy `isID = false`, oznacza to, że właściwość nie pełni funkcji identyfikatora.

Relacja `isID: Boolean` do ograniczenia `{id}`

Wartość `true` atrybutu `isID` oznacza, że właściwość **może** być częścią identyfikatora, a ograniczenie `{id}` wskazuje, które właściwości pełnią funkcję unikalnego identyfikatora.
Jeśli `{id}` odnosi się do więcej niż jednej właściwości, wszystkie te właściwości powinny mieć `isID = true`.

Innymi słowy: jeśli atrybut `isID` ma wartość `true`, to właściwość ta może być użyta do unikalnego identyfikowania instancji danej klasy.

Poprzedni przykład klasy `Product` może wyglądać następująco:

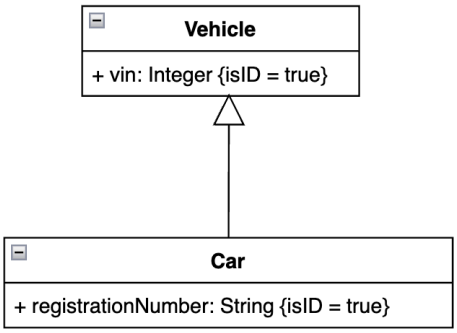


`isID` w kontekście dziedziczenia

Jeśli klasa `B` dziedziczy po klasie `A`, to może:

- **Odziedziczyć** `isID` – jeśli klasa bazowa `A` ma atrybut oznaczony `isID = true`, to podklasa `B` może go używać jako identyfikatora.
- **Rozszerzyć** `isID` – klasa dziedzicząca może dodać nowe właściwości jako identyfikatory.
- **Zastąpić** `isID` – jeśli podklasa redefiniuje identyfikator, może wskazywać inne właściwości jako `isID = true`.

Przykład



Atrybut `vin: Integer` identyfikuje `Vehicle` i jest dziedziczony przez `Car`. `Car` dodaje `registrationNumber: String` jako dodatkowy identyfikator.

1.10 Wartość domyślna właściwości

Wartość domyślna właściwości (*default value of a property*) określa początkową wartość, jaką przyjmuje dana właściwość (atrybut) obiektu, jeśli nie zostanie jawnie ustawiona. UML pozwala na określenie wartości domyślnych w modelu klasy, co pomaga w precyzyjnym definiowaniu zachowania systemu i poprawia czytelność modelu.

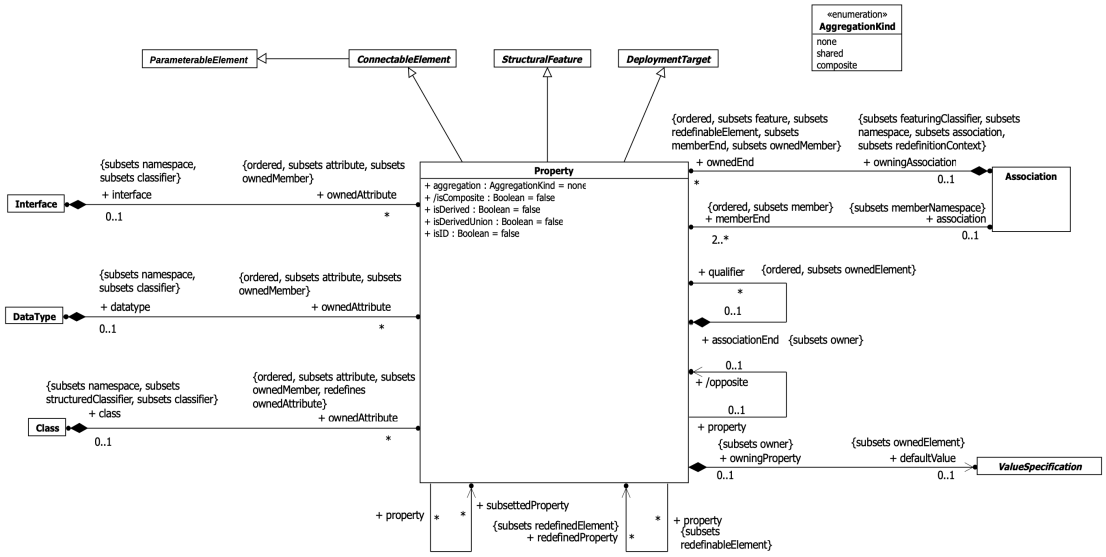
Reprezentacja wartości domyślnej w UML

W UML wartość domyślna właściwości jest zapisywana po znaku = w deklaracji atrybutu.



Wartość domyślna właściwości `pages: Integer` dla klasy `Magazine` to `64`.
Oznacza to, że jeśli nie zostanie podana wartość dla `pages`, to przypisana zostanie automatycznie wartość `64`.

Składnia abstrakcyjna



Opcjonalna własność klasy `Property` - `defaultValue` przechowuje domyślną wartość w postaci `ValueSpecification`.

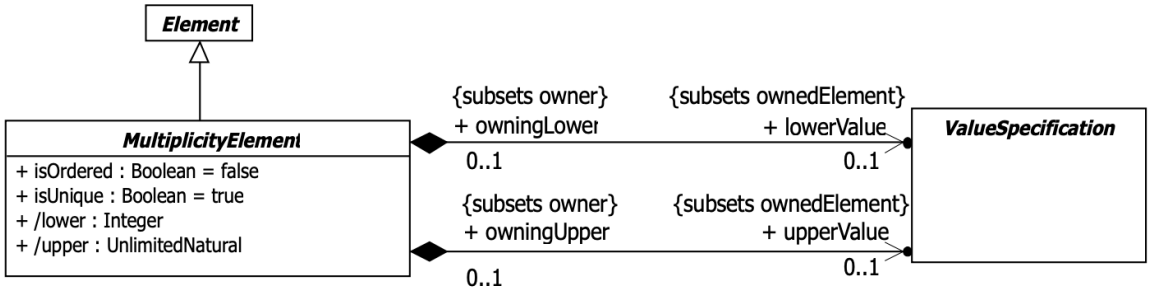
- Powiązanie `Property` z `ValueSpecification` o krotności `0..1` oznacza, że każda właściwość może mieć maksymalnie jedną wartość domyślną.
- `defaultValue` jest opcjonalne, co oznacza, że właściwość może, ale nie musi mieć określonej wartości domyślnej.
- Wartość domyślna, jeśli jest obecna, pochodzi z `ValueSpecification`, która może zawierać, np. stałe wartości liczbowe, łańcuchy znaków lub wyrażenia wyliczeniowe.

C05 1.11-1.13

1.11 Właściwość wielowartościowa (ograniczenie ORDERED, NOUNIQUE)

Właściwość wielowartościowa (*multivalued property*) to taka, której górna granica krotności jest większa niż 1 (może być nieograniczona), może zatem przyjmować więcej niż jedną wartość. Pojęcie właściwości wielowartościowej jest zdefiniowane w ramach klasy `MultiplicityElement`, której właściwości `lower`, `upper`, `isOrdered` i `isUnique` określają sposób przechowywania i interpretacji wartości.

Składnia abstrakcyjna



Krótkie przypomnienie:
`MultiplicityElement` to abstrakcyjna klasa, która:

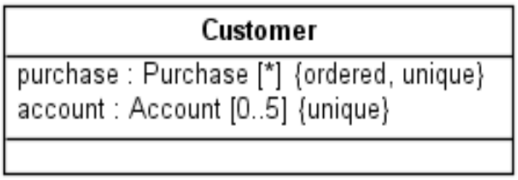
- Reprezentuje zakres liczności (*multiplicity*), czyli ile wartości może (lub musi) być przypisanych danemu elementowi.
- Jest dziedziczona przez takie elementy jak: `Property`, `Parameter`, `AssociationEnd`, `Variable`, itp.

Kluczowe atrybuty:
Oprócz `/lower: Integer` i `/upper: UnlimitedNatural`, `MultiplicityElement` posiada także:

- `isOrdered: Boolean = false` – określa on, czy element przypisane do danej właściwości są uporządkowane; dla wartości `true` kolejność jest zachowana, dla `false` natomiast nie ma ona znaczenia.
- `isUnique: Boolean = true` – określa, czy kolekcja wartości może zawierać duplikaty; dla wartości `true` wszystkie wartości muszą być unikalne (np. `Set`), dla `false` natomiast powtórzenia są dozwolone (np. `Sequence`).

Notacja w UML

Ograniczenia `ordered/unordered` i `unique/nounique` w UML są standardowo zapisywane w nawiasach klamrowych przy nazwie atrybutu lub końcu asocjacji:

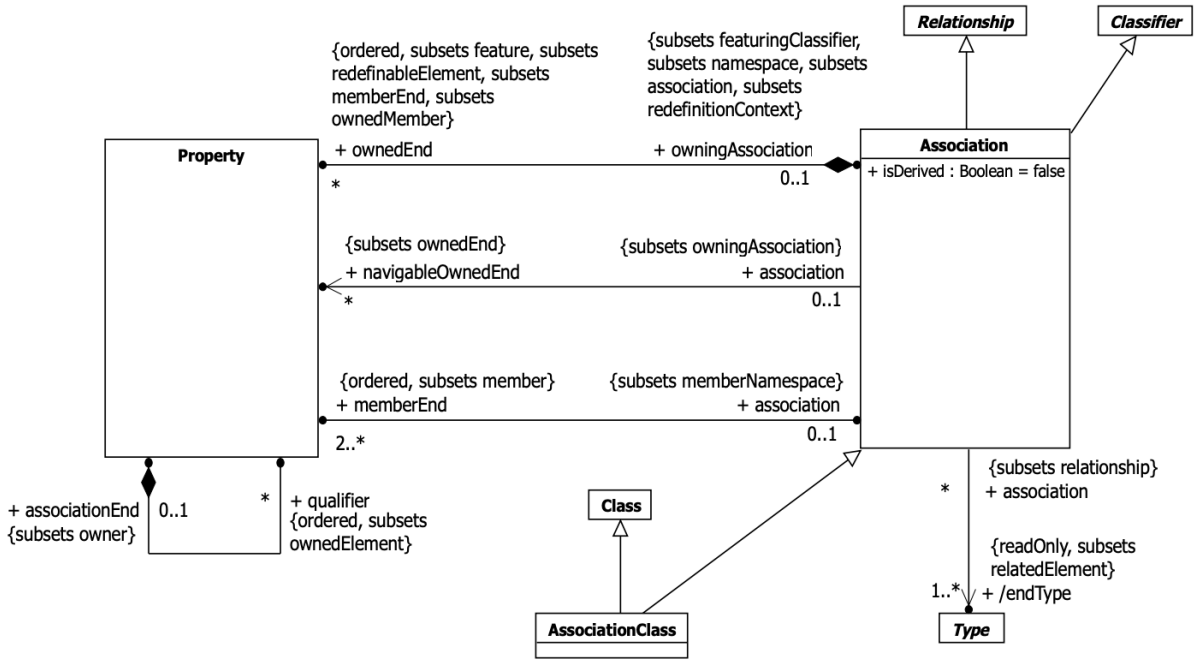


Dla klasy `Customer` atrybut `purchase: Purchase` ma krotność `[*]` i ograniczenia `{ordered, unique}` – oznacza to, że transakcji może być nieskończenie wiele, natomiast zachowana jest ich kolejność i nie powtarzają się.
Ponadto, instancja klasy `Customer` może posiadać (zgodnie z atrybutem `account: Account [0..5]`) od zera do pięciu kont, które przy ograniczeniu `{unique}` nie mogą się powtarzać (muszą być unikalne).

1.12 Klasa asocjacji

Asocjacja może zostać uszczegółowiona tak, aby posiadała własny zestaw cech – takich, które nie należą do żadnego z powiązanych klasyfikatorów, lecz są właściwe dla samej asocjacji.
Taka asocjacja nazywa się klasą asocjacji (`AssociationClass`) i jest ona zarówno asocjacją, łączącą zbiór klasyfikatorów, jak i klasą, a jako taka może posiadać własne cechy i może być powiązana innymi asocjacjami.

Składnia abstrakcyjna



`AssociationClass` dziedziczy jednocześnie po dwóch innych klasach:

a) `Class` – ma atrybuty (`ownedAttribute`), operacje, generalizacje, itd.
b) `Association` – definiuje związki (`memberEnd`) pomiędzy `Type` (czyli klasami).

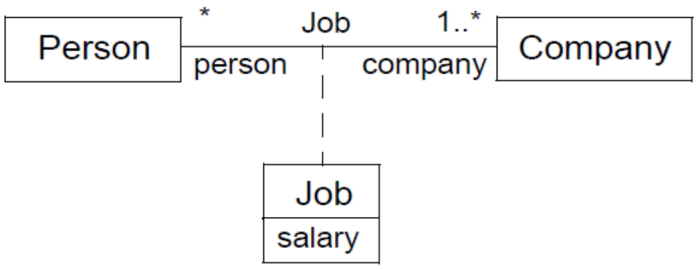
`Association`:

a) Właściwości:

  - `isDerived: Boolean = false` – mówi czy asocjacja jest wyprowadzona z innych (np. dziedziczona).
b) Składa się z `memberEnd` – końców asocjacji (minimum 2).
c) Najbardziej szczegółowe podzbiory końców asocjacji: `ownedEnd`, `navigableEnd`, `owningAssociation`.

`Property`:
a) Reprezentuje atrybut klasy lub **koniec asocjacji**.
b) Jeśli należy do klasy to jest atrybutem (`ownedAttribute`), jeśli do asocjacji to jest jej końcem (`ownedEnd`).
c) Każdy `Property` może mieć:
  - `qualifier` – do zdefiniowania klucza złożonego.
  - `associationEnd` – odniesienie do końca asocjacji.

Notacja w UML
Klasa asocjacji jest przedstawiana jako symbol klasy połączony ze ścieżką asocjacji **przerywaną linią**. Ścieżka asocjacji oraz symbol klasy asocjacji reprezentują ten sam element modelu i mają **jedną wspólną nazwę**. Nazwa może być umieszczona zarówno na ścieżce asocjacji, w symbolu klasy, jak i w obu miejscach jednocześnie, ale w każdym przypadku musi być taka sama.



1.13 Agregacja, Kompozycja

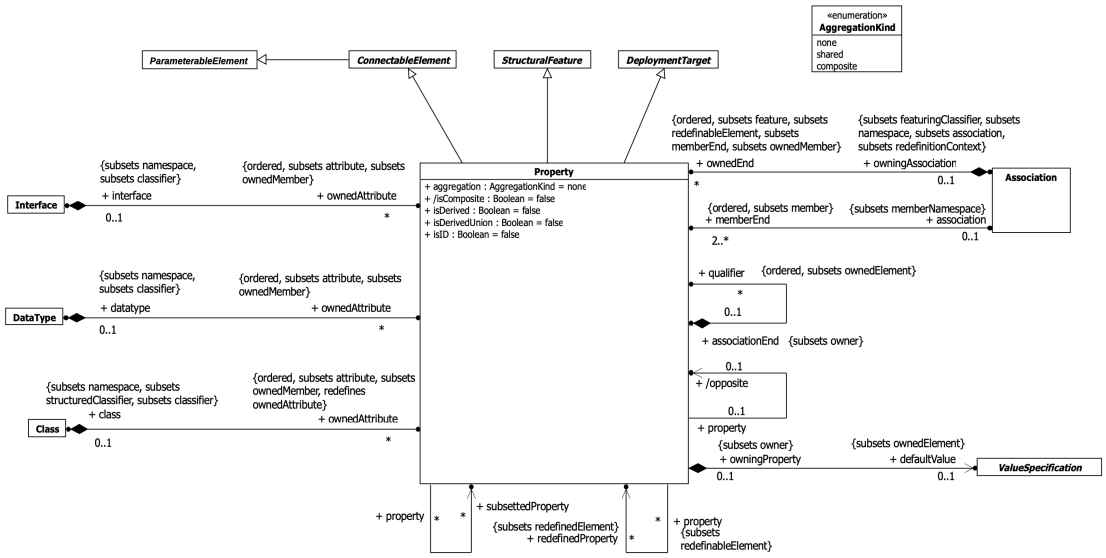
Agregacja to binarny związek (całość-część) pomiędzy właściwością (atrybutem) a jednym lub wieloma obiektami złożonymi (kompozytowymi), które grupują zbiór instancji.
W praktyce oznacza, że jeden obiekt jest częścią drugiego, ale może też istnieć niezależnie od niego.

Cechy agregacji:
- **Asymetria** – tylko jeden koniec związku może być oznaczony jako agregacja.
- **Tranzytywność** – powiązania agregacji powinny tworzyć **ukierunkowany, acykliczny graf**, tak aby żadna instancja kompozytu **nie mogła być częścią samej siebie** (nawet pośrednio).

– Część **może być współdzielona** pomiędzy wieloma kompozytami, i jeśli część lub wszystkie z tych kompozytów zostaną usunięte, **współdzielona część może nadal istnieć**.

Do modelowania sytuacji, w której jedna instancja jest używana do grupowania zbioru innych instancji stosuje się `Property` oraz jej atrybut `aggregation: AggregationKind`.

Składnia abstrakcyjna



`AggregationKind` jest typem wyliczeniowym (enumeracją), który zawiera następujące wartości:

– `none` – oznacza, że właściwość nie posiada semantyki agregacji.
– `shared` – oznacza, że właściwość posiada semantykę agregacji współdzielonej (*shared aggregation*) (jej dokładne znaczenie może różnić się w zależności od dziedziny zastosowania i interpretacji modelującego).
– `composite` – oznacza, że właściwość jest agregowana kompozycyjnie (*composite aggregation*), czyli:
    • Obiekt złożony (kompozyt) odpowiada za istnienie i przechowywanie obiektów składowych (części - *parts*).

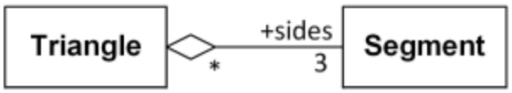
Kompozycja

Jest to **silna forma agregacji**, która wymaga, aby dana część była zawarta w **co najwyżej jednym** obiekcie kompozytowym w danym momencie. Jeśli obiekt kompozytowy zostanie usunięty, wszystkie jego części składowe, które są obiektami, również zostaną usunięte razem z nim.
Jeśli dany model na to pozwala, obiekt będący częścią może zostać usunięty z kompozytu przed jego usunięciem, i dzięki temu, nie musi być usunięty razem z całością.

Notacja w UML

Agregacja współdzielona jest przedstawiana jako asocjacja oznaczona pustym rombem po stronie agregatu (całości). Romb powinien być zauważalnie mniejszy niż symbol rombu używany w notacji asocjacji n-arnej.

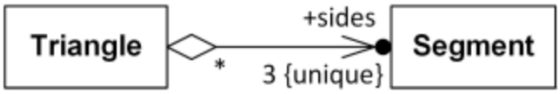
Przykłady:



Klasa `Triangle` agreguje (w kolekcji `sides`) dokładnie trzy instancje klasy `Segment` (trójkąt agreguje boki). Krotność `*` przy końcu agregacji po stronie `Triangle` oznacza, że każdy odcinek może być częścią wielu trójkątów, albo nie należeć do żadnego.
Usunięcie konkretnej instancji `Triangle` nie spowoduje usunięcia żadnego odcinka składowego.

Agregacja współdzielona może być przedstawiana w połączeniu z innymi elementami charakterystycznymi dla asocjacji, np.:

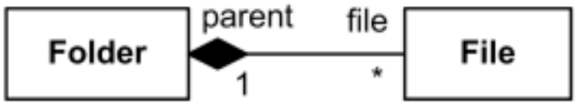
- Nawigowalność (czyli, w którą stronę można „przechodzić" między obiektami).
- Własność końca asocjacji (czy dany koniec należy do klasy czy do samej asocjacji).



Klasa `Triangle` tak jak w poprzednim przykładzie agreguje w kolekcji sides dokładnie trzy instancje klasy `Segment`. `Segment` jest nawigowalny od strony `Triangle` – można z trójkąta przejść do jego boków.
Ponadto, kolekcja sides zawiera trzy unikalne `{unique}` odcinki oraz jest własnością klasy `Triangle`, a nie relacji jako takiej.

Agregacja kompozycyjna (kompozycja) przedstawiana jest podobnie jak agregacja współdzielona, jednak z wypełnionym czarnym rombem po stronie agregatu.

Przykłady:



`Folder` może zawierać wiele instancji `File`, podczas gdy każda instancja `File` ma dokładnie jeden nadrzędny `Folder` (`Folder` *parent*). Jeśli `Folder` zostanie usunięty, wszystkie instancje `File` w nim zawarte również zostaną usunięte.

Warto zauważyć, że krotność po stronie kompozytu może być oznaczona jako `0..1` (co najwyżej jeden), co oznacza, że część może istnieć samodzielnie, bez bycia własnością konkretnej całości.



Każda instancja `Department` ma jakąś instancję `Staff`, a każda instancja `Staff` może być członkiem jednej instancji `Department` (lub żadnej). Jeśli Department zostanie usunięty, `Staff` również zostanie usunięty (za wyjątkiem samodzielnych instancji).

2. Diagram przypadków użycia

C06 2.1-2.3

2.1 Aktor

Aktor (`Actor`) jest klasyfikatorem, **reprezentującym rolę odgrywaną przez zewnętrzny podmiot** (np. człowieka, system, organizację itp.) w interakcji z systemem modelowym.
Aktor nie jest częścią systemu, ale wchodzi z nim w interakcję – stanowi element jego otoczenia (*system boundary*).

Aktorzy mogą:

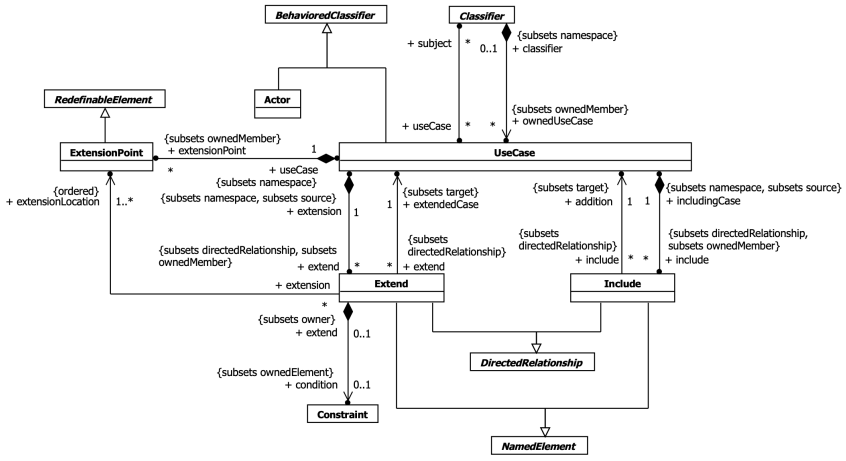
- Inicjować przypadki użycia (`Use Cases`).
- Uczestniczyć w przypadkach użycia.
- Być zewnętrznymi systemami, urządzeniami, organizacjami lub ludźmi pełniącymi określoną rolę.

Rodzaje aktorów:

a) Aktor główny (*primary*) – inicjuje przypadek użycia (np. `Client` rozpoczynający proces zakupu).
b) Aktor pomocniczy (*secondary*) – wspomaga działanie przypadku użycia, ale go nie inicjuje (np. `PaymentSystem` obsługujący płatność).

Zgodnie ze specyfikacją UML:
Aktor może mieć asocjacje tylko z przypadkami użycia (`UseCases`), komponentami (`Components`) i klasami. Ponadto, asocjacje te muszą być binarne.

<u>Składnia abstrakcyjna</u>



`Actor` dziedziczy po `BehavioredClassifier`, który z kolei dziedziczy po `Classifier` - w specyfikacji można znaleźć fragment:

10.5.1 BehavioredClassifier [Abstract Class]

10.5.1.3 Generalizations

Classifier

Jest on zatem pełnoprawnym klasyfikatorem, który może:
- Mieć nazwę (`NamedElement`).
- Uczestniczyć w relacjach (np. `Generalization`).
- Być specjalizowany (dziedziczenie ról).

Ponadto, dzięki dziedziczeniu po `BehavioredClassifier`, `Actor`:
- Może posiadać zachowania (`ownedBehavior`, `classifierBehavior`).
- Może implementować interfejsy (`InterfaceRealization`), choć jest to rzadko wykorzystywane w klasycznym modelowaniu przypadków użycia.

Co warte uwagi, w metamodelu UML `Actor` nie dodaje żadnych własnych atrybutów – korzysta w pełni z tego, co dziedziczy po `BehavioredClassifier`.

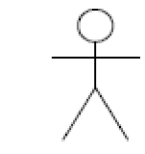
Każdy `Actor` może być powiązany z wieloma `UseCase`, a każdy `UseCase` może być powiązany z wieloma aktorami. Asocjacja z `UseCase` jest podzbiorem `ownedMember`, co oznacza, że `Actor` może być właścicielem tej relacji.
Atrybuty końców tej asocjacji:
- `+ useCase` – zestaw przypadków użycia, w których aktor uczestniczy.
- `+ subject` – wskazuje `Classifier`, który realizuje przypadki użycia (np. system).

Choć `Actor` nie ma bezpośredniej relacji z `Include`, `Extend`, `Constraint` czy `ExtensionPoint`, to może brać udział w przypadkach użycia, które wykorzystują te relacje.

Notacja w UML
W UML aktor może być przedstawiany na kilka sposobów – wszystkie są równoważne semantycznie, ale różnią się czytelnością i przeznaczeniem.

a) *Stickman* (człowieczek) – najbardziej znana notacja:
Przedstawiany za pomocą symbolicznego ludzika (*stick figure*) oraz nazwy roli obok lub pod spodem.
Najczęściej stosowany w diagramach przypadków użycia. Jest łatwo rozpoznawalny i dość intuicyjny.

Przykład:

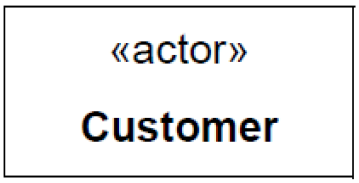


Customer

b) Prostokąt klasy z etykietą **«actor»**:
Używany najczęściej, gdy aktor:
   - Reprezentuje system lub urządzenie, a nie człowieka.
   - Ma być częścią klasyfikacji dziedziczenia (np. w diagramach klas).

Można go połączyć z interfejsami, komponentami itd.

Przykład:



c) Alternatywna ikona (np. komputer, chmurka, serwer):
Nie jest to formalna część notacji UML, ale jest dozwolona jako wizualne ułatwienie (np. w narzędziach typu Visual Paradigm czy Enterprise Architect).

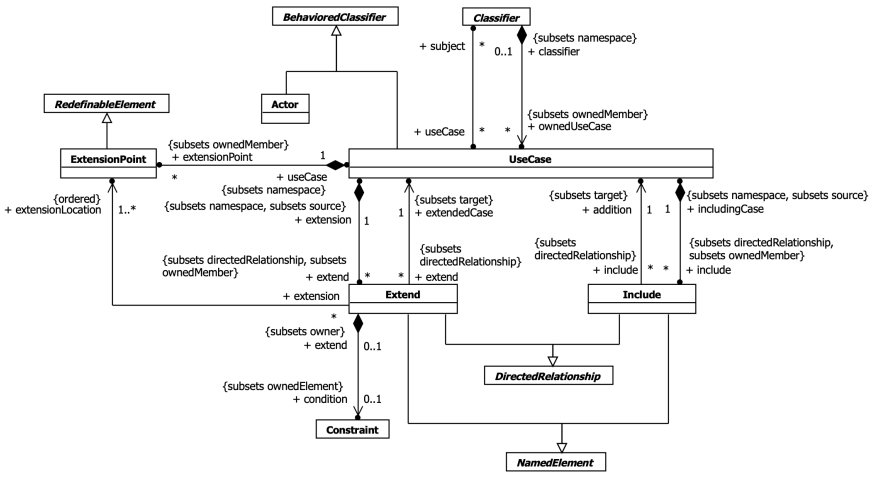
Przykład:



2.2 Przypadek użycia

Przypadek użycia (`UseCase`) to opis zachowania systemu widzianego z zewnątrz – z punktu widzenia aktora, który z niego korzysta. Opisuje wartościowy cel, który aktor chce osiągnąć przez interakcję z systemem.
Innymi słowy, opisuje **co** system robi na rzecz użytkownika, a **nie jak** to robi.

Składnia abstrakcyjna



`UseCase` dziedziczy po `BehavioredClassifier` oraz `Classifier`, co oznacza, że jest klasyfikatorem posiadającym zachowania, czyli:

- Może być typowany.
- Może mieć własne zachowanie (`ownedBehavior`).
- Może być specjalizowany (dziedziczenie).
- Może mieć relacje, takie jak asocjacje, dziedziczenie, użycia itp.

Dziedziczenie i własności odziedziczone

Dzięki dziedziczeniu po `BehavioredClassifier`, `UseCase`:

- `+ name: String` – przypadek użycia ma nazwę.
- `+ ownedBehavior: Behavior [*]` – może mieć zachowanie (np. *activity diagram*).
- `+ classifierBehavior: Behavior [0..1]` – może mieć zachowanie reprezentujące jego logikę.

Relacje i składniki `UseCase`:

a) `+ subject: Classifier [*]:`
   - `UseCase` jest realizowany przez jakiś klasyfikator (zwykle system lub komponent).
   - Relacja ta wskazuje, do jakiego systemu przypadek użycia należy.

b) `+ useCase: UseCase [*]:`
   - Pokazuje, że `Actor` uczestniczy w `UseCase`.
   - Asocjacja binarna, wiele do wielu.
   - Podzbiór `ownedElement`.

c) `+ extensionPoint: ExtensionPoint [*]:`
   - `UseCase` może posiadać punkty rozszerzenia – miejsca, gdzie inne przypadki mogą go rozszerzać.
   - Własność złożona `{subsets ownedMember} [1..*]` – musi być przypisany do jakiegoś `UseCase`.

d) `+ extend: Extend [*]:`
   - `UseCase` może być rozszerzany przez inne przypadki.
   - Relacja `Extend` → `extendedCase` (docelowy przypadek) i `extension` (który rozszerza).
   - Dopuszcza dodatkowy przypadek uruchamiany w warunkach (`condition: Constraint`).

e) `+ include: Include [*]:`
   - `UseCase` może zawierać inny przypadek użycia na zasadzie współdzielenia funkcjonalności.
   - Relacja `Include` → `includingCase` (główny) i `addition` (dodany).
   - Zawsze wykonywana (nie warunkowo).

Powiązane elementy:

- `ExtensionPoint` – definiuje miejsce, gdzie `Extend` może być „wpięty”.
- `Extend` – pozwala warunkowo rozszerzyć `UseCase`.
- `Include` – pozwala dodać inny `UseCase` w ramach działania.
- `Constraint` – określa warunki rozszerzenia (`condition` w `Extend`).
- `Actor` – zewnętrzna rola uczestnicząca w `UseCase`.
- `Classifier` – realizator `UseCase` – najczęściej system.

Inne właściwości:

- `UseCase` może być redefiniowalny (`RedefinableElement`) – czyli być modyfikowany przez specjalizację.
- Może występować w dziedziczeniu przypadków użycia (generalizacja).

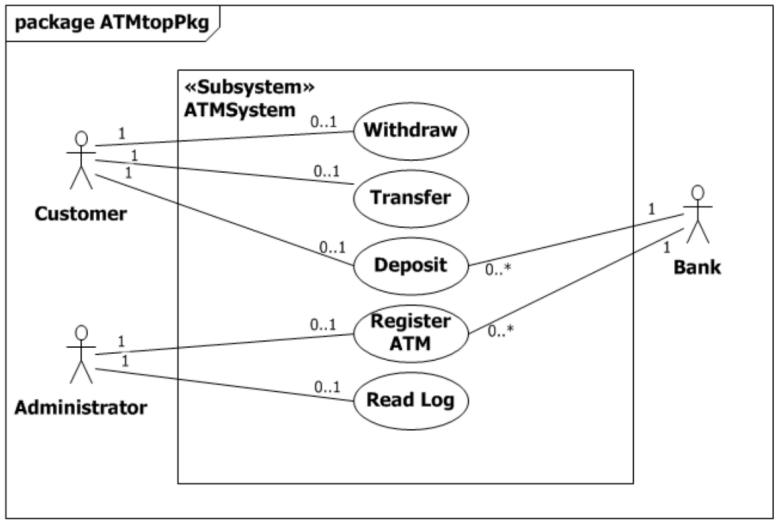
Notacja w UML

Przypadek użycia w UML reprezentowany jest jako elipsa (owal) z nazwą w środku lub pod spodem. Nazwa ta powinna być czytelna i zrozumiała – UML nie narzuca konkretnej formy, ale najczęściej spotykana jest forma czasownik + dopełnienie, np. „Zarejestruj użytkownika” czy „Wyszukaj książkę”.

Przykład:

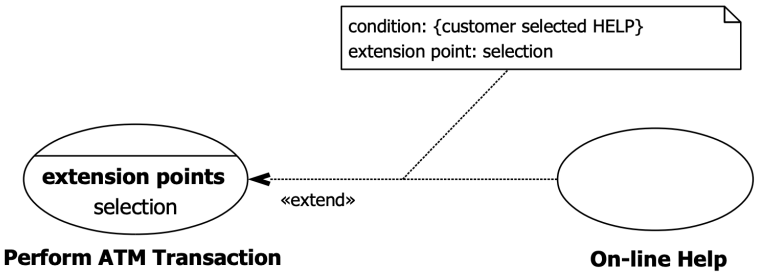


Bardziej rozbudowany przykład:



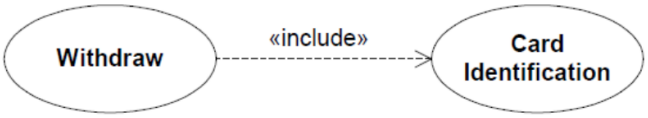
Pakiet `ATMtopPkg` zawiera zbiór przypadków użycia, aktorów i podsystem `ATMSystem`.

Przykład rozszerzenia:



Przypadek użycia `Perform ATM Transaction` ma `ExtensionPoint` nazwany `selection` i za jego pomocą jest rozszerzany przez przypadek użycia `On-line Help`. Warto zauważyć, że jest rozszerzany tylko, gdy `customer selected HELP`.

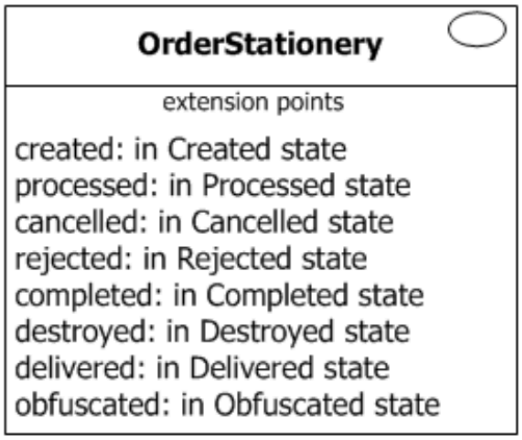
Przykład zawierania:



Przypadek użycia `Withdraw` zawiera niezależnie zdefiniowany przypadek użycia `Card Identification`.

Alternatywna notacja

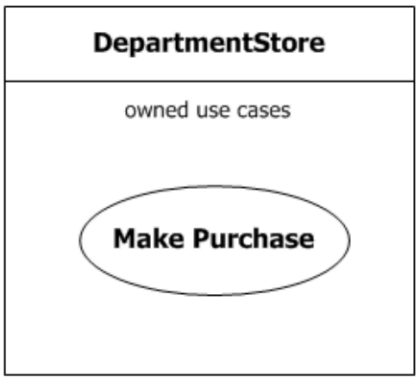
Czasami spotyka się także notację charakterystyczną dla klasyfikatorów, ale z małą elipsą w rogu i opcjonalnym segmentem `extension points`:



Przypadek użycia `OrderStationery` z punktami rozszerzeń:

- `created`
- `processed`
- `cancelled`
- `rejected`
- `completed`
- `destroyed`
- `delivered`
- `obfuscated`.

Przykład przypadku użycia posiadanego przez klasyfikator:



Klasyfikator `DepartmentStore` posiada przypadek użycia `Make Purchase` (znajduje się on w segmencie `owned use cases`).

2.3 Asocjacja

Sama asocjacja była już omawiana dokładnie tutaj, w tym temacie skupiono się na asocjacji w diagramie przypadków użycia.

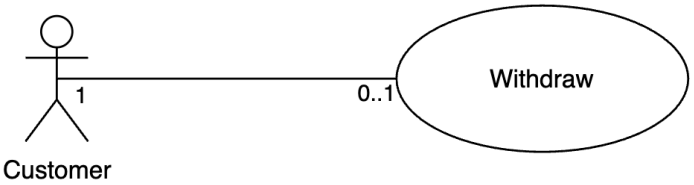
Asocjacja w diagramie przypadków użycia UML to relacja binarna łącząca aktora z przypadkiem użycia. Wskazuje ona, że dany aktor uczestniczy w realizacji danego przypadku użycia – inicjuje go lub w nim aktywnie uczestniczy.

Notacja w UML

Asocjacja ta przedstawiana jest standardowo jako prosta linia bez strzałki – zupełnie jak zwykła asocjacja. Łączy ona symbol aktora z elipsą przypadku użycia. UML nie wymaga wskazywania kierunku – relacja ma charakter dwukierunkowy semantycznie.

Jeden aktor może być powiązany z wieloma przypadkami użycia i odwrotnie.

Przykład:



Aktor `Customer` jest powiązany z przypadkiem użycia `Withdraw`. Ponadto, `Customer` może nie wykonywać wypłaty, ale jeśli już to robi, to bierze udział w dokładnie jednej instancji wypłaty. Z kolei `Withdraw` zawsze jest wykonywany przez jednego klienta – system nie obsłuży wypłaty „anonimowej” ani współdzielonej przez kilku klientów jednocześnie.

Istotne informacje:

a) `Actor` może być powiązany tylko z:
   - `UseCase`.
   - `Component`.

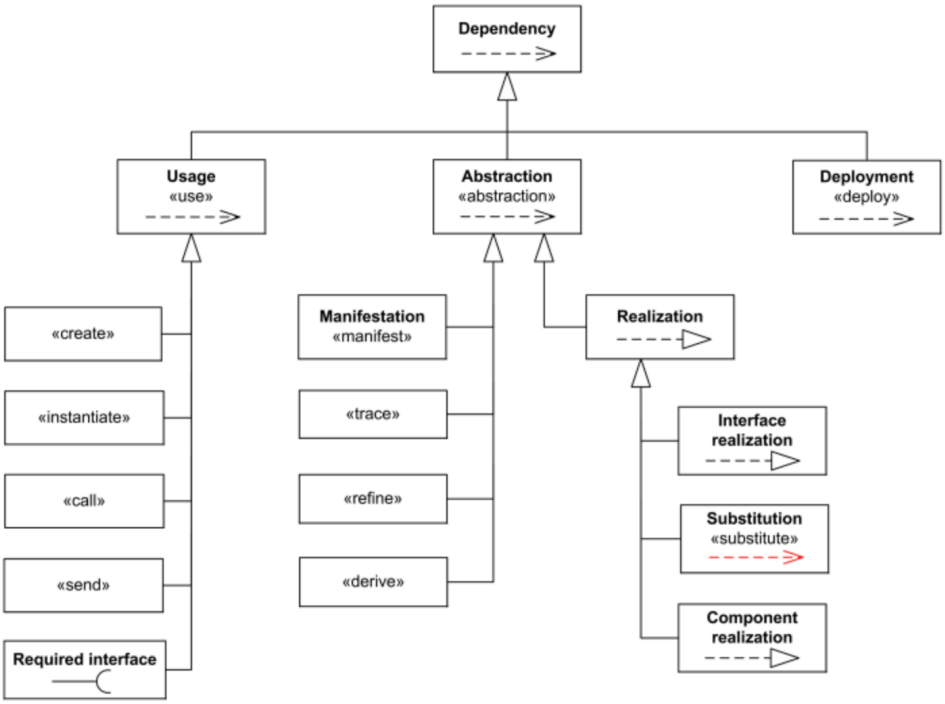
  - `Class`.

b) Asocjacje muszą być binarne – mogą mieć tylko dwa końce.
c) UML nie dopuszcza asocjacji między aktorami (np. `Customer` - `Salesman`).

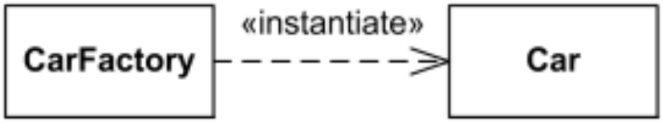
C07 2.4-2.5

2.4 Relacja zawierania (*include dependency*)

Zacząć należy od tego, czym jest samo `Dependency`. Jest to relacja skierowana między dwoma nazwanymi elementami modelu, wskazująca, że jeden element (klient) zależy od drugiego (dostawcy) w kontekście specyfikacji lub implementacji.
Istnieje wiele różnych rodzajów zależności:



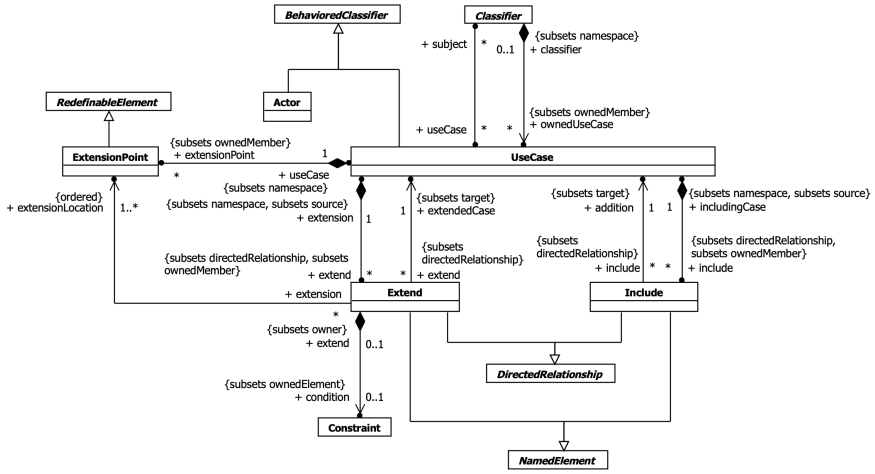
Na diagramach UML, relacja zależności jest przedstawiana jako przerywana strzałka skierowana od klienta do dostawcy. Strzałka może być opisana nazwą relacji lub stereotypem, np. `<<use>>`, `<<derive>>`, `<<trace>>`, `<<refine>>`, `<<realize>>` itp.



Relacja zawierania (`Include`) w UML to specjalny rodzaj zależności między przypadkami użycia, która służy do **wydzielenia wspólnej funkcjonalności** wykonywanej w wielu scenariuszach. Wskazuje, że zawarty przypadek użycia **jest zawsze wykonywany** w ramach przypadku, który go zawiera.

Innymi słowy, **`<<include>>`** opisuje fragment zachowania wspólnego dla więcej niż jednego przypadku użycia i pozwala na unikanie powielania tych samych kroków w różnych scenariuszach (rozszerzanie funkcjonalności bazowego przypadku użycia o zachowanie innego przypadku użycia).

Składnia abstrakcyjna



`Include` dziedziczy po `DirectedRelationship` oraz `NamedElement`, co oznacza, że jest relacją skierowaną i może posiadać nazwę.

Właściwości:

- `+ addition: UseCase [1]` – wskazuje przypadek użycia, który jest włączany
- `+ includingCase: UseCase [1]` – wskazuje przypadek użycia, który zawiera inny przypadek
- `+ name: String [0..1]` – nazwa relacji (opcjonalna, rzadko używana)
- `+ visibility: VisibilityKind` – widoczność relacji (zwykle `public`)
- `+ clientDependency: Dependency [*]` – dziedziczona właściwość z `NamedElement`

Relacja ta nie ma warunku wykonania – jej wykonanie jest bezwzględne i automatyczne w ramach głównego scenariusza.

Powiązane elementy:

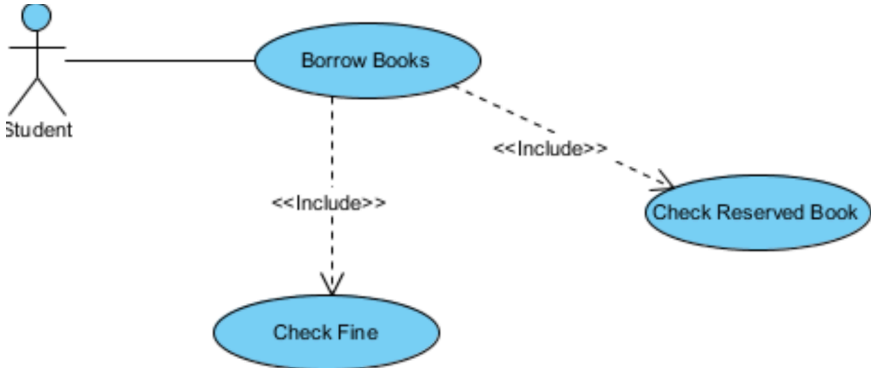
- `UseCase` – zarówno `includingCase`, jak i `addition` są przypadkami użycia.
- `Actor` – może uczestniczyć w nadrzędnym przypadku użycia, niekoniecznie w zawieranym.
- `Classifier` – przypadki użycia należą do klasyfikatora (systemu).
- `Include` – zależność między przypadkami.

Semantyka działania relacji:

- Zawsze wykonywana – nie zależy od żadnych warunków (w odróżnieniu od `Extend`).
- Rozszerza główny przypadek użycia o nową funkcję, ale w kontekście jego działania.
- Nie zmienia zachowania oryginalnego przypadku użycia, lecz go wydziela.
- Wspomaga ponowne użycie i refaktoryzację przypadków użycia.

Notacja w UML

Notacja relacji zawierania w UML przedstawiana jest na diagramie przypadków użycia jako przerywana strzałka z otwartym grotem, skierowana od nadrzędnego przypadku użycia do zawieranego. Nad strzałką umieszcza się stereotyp **<<include>>**, który jednoznacznie identyfikuje rodzaj relacji.
Elipsa reprezentująca zawierany przypadek użycia może znajdować się obok lub poniżej przypadku głównego – ważne, by zachować kierunek od „głównego" do „wspólnego".

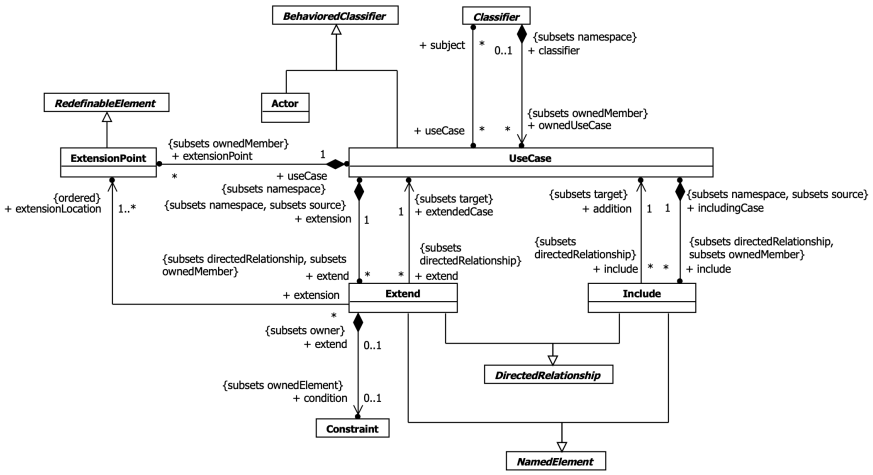


2.5 Relacja rozszerzania (*extend dependency*)

Relacja rozszerzania w UML to szczególny typ relacji między przypadkami użycia, który umożliwia warunkowe dołączenie dodatkowego zachowania do scenariusza bazowego. Służy do modelowania opcjonalnych funkcjonalności, które są realizowane tylko wtedy, gdy zostanie spełniony określony warunek.

Mówiąc prościej, **<<extend>>** pozwala dodać nowe kroki do głównego przypadku użycia tylko w określonych okolicznościach. Przypadek rozszerzający istniejący samodzielnie nie ma sensu, ponieważ jest wykonywany tylko jako uzupełnienie innego przypadku użycia.

Składnia abstrakcyjna



`Extend` dziedziczy po `DirectedRelationship` oraz `NamedElement`, co oznacza, że jest relacją skierowaną i może mieć nazwę (tak jak `Include`).

Właściwości:

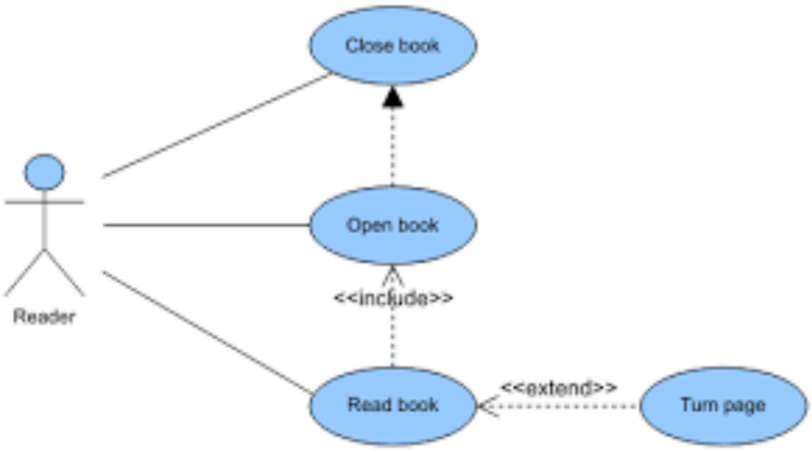
- `+ extendedCase: UseCase [1]` – przypadek użycia, który może zostać rozszerzony
- `+ extension: UseCase [1]` – przypadek użycia, który rozszerza inny przypadek – czyli dodatkowe zachowanie
- `+ condition: Constraint [0..1]` – określa warunek logiczny, który musi być spełniony, aby rozszerzenie zostało wykonane
- `+ extensionLocation: ExtensionPoint [*]` – określa konkretny punkt w głównym przypadku użycia, w którym następuje rozszerzenie.

Semantyka działania:

- Opcjonalne wykonanie – rozszerzenie następuje tylko, gdy spełniony jest warunek.
- Rozszerzający przypadek nie jest samodzielny – wykonuje się wyłącznie w kontekście `extendedCase`.
- Nie zmienia zachowania podstawowego przypadku, lecz je uzupełnia.

Notacja w UML

Relacja `Extend` jest przedstawiana jako przerywana strzałka z otwartym grotem, skierowana od przypadku rozszerzającego do rozszerzanego, z podpisanym stereotypem **`<<extend>>`** nad strzałką. Ponadto, do notacji dodany może zostać także warunek wykonania (jako adnotacja tekstowa (`note`) przy strzałce) oraz punkt rozszerzenia (`extension point` – widoczny w owalnym symbolu przypadku bazowego).



C08 2.6-2.7

2.6 Relacja generalizacji: aktorzy

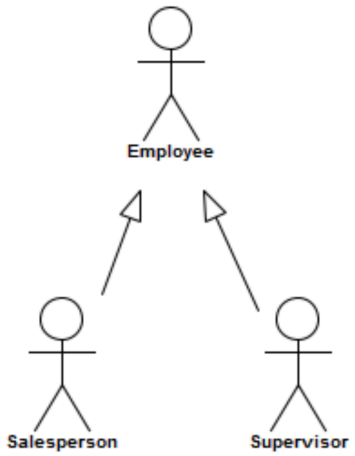
Relacja generalizacji w temacie aktorów służy do modelowania hierarchii ról, gdzie aktor specjalizujący się (potomek) dziedziczy zachowania i interakcje z przypadków użycia przypisanych do aktora głównego (rodzica). Pozwala to na uporządkowanie i uproszczenie diagramów przypadków użycia poprzez unikanie powielania tych samych relacji dla podobnych aktorów.

Semantyka działania:

- Dziedziczenie zachowań – aktor-potomek automatycznie uczestniczy we wszystkich przypadkach użycia, w których uczestniczy aktor-rodzic.
- Dodawanie specyfiki – aktor-potomek może mieć dodatkowe relacje z przypadkami użycia, które są specyficzne tylko dla niego.
- Zachowanie zgodności – aktor-potomek jest zgodny z aktorem-rodzicem, co oznacza, że może być używany wszędzie tam, gdzie wymagany jest aktor-rodzic.

Notacja w UML

Na diagramie przypadków użycia relacja generalizacji między aktorami jest przedstawiana jako ciągła linia z pustym trójkątnym grotem, skierowanym od aktora-potomka do aktora-rodzica (analogicznie do diagramu klas).



W tym przykładzie `Salesperson` i `Supervisor` dziedziczą zachowania aktora `Employee`.

2.7 Relacja generalizacji: przypadki użycia

Relacja generalizacji w kontekście przypadków użycia pozwala na organizowanie przypadków w hierarchie, w których jeden przypadek dziedziczy zachowanie innego. Jest to użyteczne, gdy wiele przypadków użycia ma podobny scenariusz, ale różnią się w szczegółach – pozwala to uniknąć powielania opisu i zachować przejrzystość modelu.

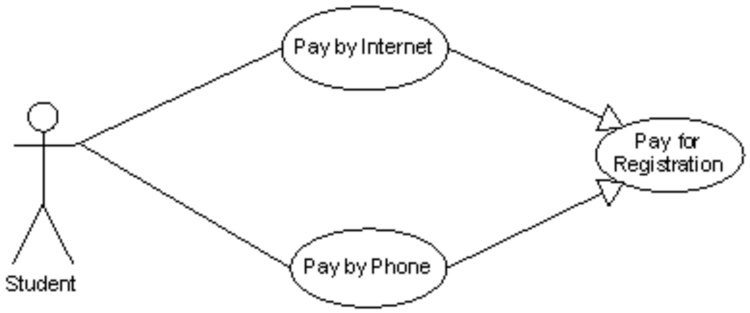
Relacja generalizacji pomiędzy przypadkami użycia oznacza, że przypadek potomny dziedziczy wszystkie cechy (czyli relacje, punkty rozszerzenia i zachowania) przypadku nadrzędnego. Może on także dodawać nowe lub modyfikować istniejące cechy (jeśli są one redefiniowalne).

Semantyka działania:

- Przypadek szczegółowy dziedziczy wszystkie relacje i zachowania przypisane do przypadku ogólnego.
- Przypadek szczegółowy może być użyty w jego miejscu.
- Możliwe jest zdefiniowanie różnych zachowań dla przypadków potomnych, np. przez przypisanie różnych `ownedBehavior`.

Notacja w UML

Na diagramie przypadków użycia relacja generalizacji przedstawiana jest jako ciągła linia zakończona pustym trójkątem, skierowanym od przypadku szczegółowego (*subcase*) do ogólnego (*supercase*). W praktyce, wygląda analogicznie do generalizacji między aktorami lub klasami.



W powyższym przykładzie przypadki `Pay by Internet` oraz `Pay by Phone` dziedziczą po przypadku `Pay for Registration`.

3. Diagram aktywności

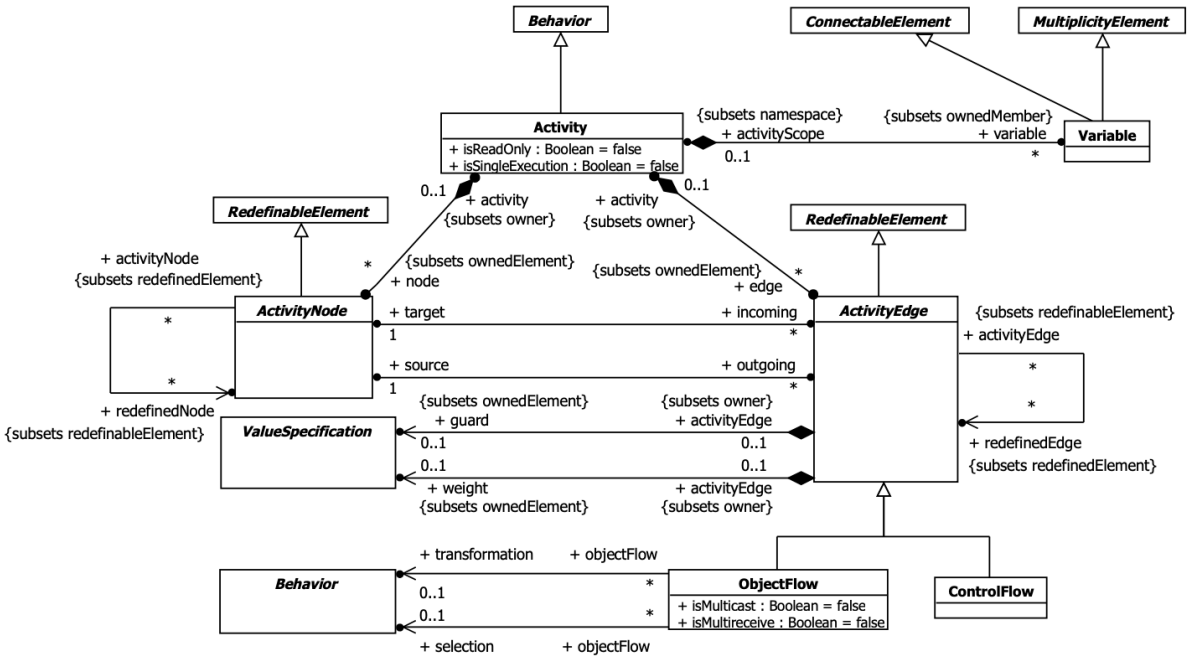
C09 3.1-3.5

3.1 Aktywność, a czynność (*Activity* vs *Action*)

Aktywność (`Activity`) to sparametryzowane zachowanie przedstawiane jako skoordynowany przepływ akcji.
Przepływ wykonania modelowany jest za pomocą **węzłów aktywności** połączonych **krawędziami aktywności**. Węzeł może reprezentować wykonanie podrzędnego zachowania, takiego jak obliczenie arytmetyczne, wywołanie operacji lub manipulację zawartością obiektów. Węzły aktywności obejmują również konstrukcje sterujące przepływem, takie jak synchronizacja, decyzje i kontrola współbieżności.

Aktywności mogą tworzyć hierarchie wywołań, przywołując inne aktywności, które ostatecznie prowadzą do pojedynczych akcji. W modelu obiektowym aktywności są zwykle wywoływane pośrednio jako metody powiązane z operacjami, które są bezpośrednio wywoływane.

Składnia abstrakcyjna



`Activity` dziedziczy po metaklasie `Behavior`, co oznacza, że jest rodzajem zachowania, które można wykonywać. Przedstawia koordynowany przepływ czynności (akcji, obiektów, sterowania).

Atrybuty `Activity`:

- `isReadOnly: Boolean = false` – określa, czy aktywność może modyfikować dane. Jeśli `true`, to żadne dane nie są zmieniane w jej trakcie.
- `isSingleExecution: Boolean = false` – określa, czy aktywność jest wykonywana jako pojedyncze (współdzielone) wykonanie dla wszystkich wywołań. Jeśli `true`, wszystkie instancje wykonują ten sam przebieg.

Własności strukturalne:

- `+ node: ActivityNode [*]` – zbiór wszystkich węzłów należących do aktywności (akcje, decyzje, merge, itp.)
- `+ edge: ActivityEdge [*]` – krawędzie łączące węzły i określające przepływ danych lub sterowania
- `+ variable: Variable [*]` – zmienne lokalne aktywności (np. do przechowywania stanu w trakcie jej działania)
- `+ activityScope: Activity (0..1)` – zakres zmiennych (zmienna może być lokalna dla danej aktywności).

Relacje z innymi metaklasami:

a) `ActivityNode` – to abstrakcyjna klasa reprezentująca każdy węzeł w aktywności. Dziedziczy po `RedefinableElement`.
Każdy węzeł aktywności:
   - należy do jednej aktywności (`+ activityNode {subsets redefinedElement}`)
   - ma źródło i cel (`+ source, + target`) dla krawędzi.

b) `ActivityEdge` – reprezentuje przepływ między węzłami (np. przepływ sterowania lub danych).
Może mieć:
   - warunek przejścia (`guard`) – `ValueSpecification`
   - wagę (`weight`) – określa, np. liczbę tokenów
   - typy specjalizacji: `ControlFlow`, `ObjectFlow`

c) `ObjectFlow` – specjalizacja `ActivityEdge`, służy do przenoszenia obiektów między akcjami.
Atrybuty:
   - `isMulticast`, `isMultireceive` – obsługa wielu odbiorców/nadawców

d) `ControlFlow` – specjalizacja `ActivityEdge` dla przepływu sterowania (bez przenoszenia obiektów).

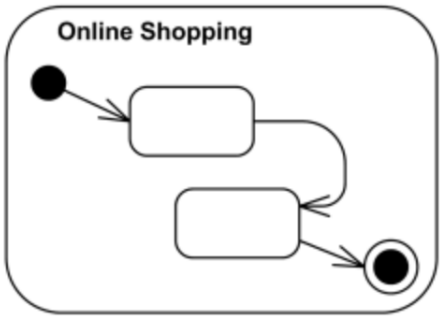
e) `Behavior` (związany z przekształceniem lub selekcją) – może być powiązany z `ObjectFlow` jako:

- `+ transformation` – zmienia dane w trakcie przepływu
- `+ selection` – wybiera, które dane mają być przesłane.

Notacja w UML

Activity może być przedstawiana jako prostokąt z zaokrąglonymi rogami, z nazwą aktywności w lewym górnym rogu oraz węzłami i krawędziami znajdującymi się wewnątrz obramowania. Nazwa aktywności zazwyczaj jest zapisywana pogrubioną czcionką.

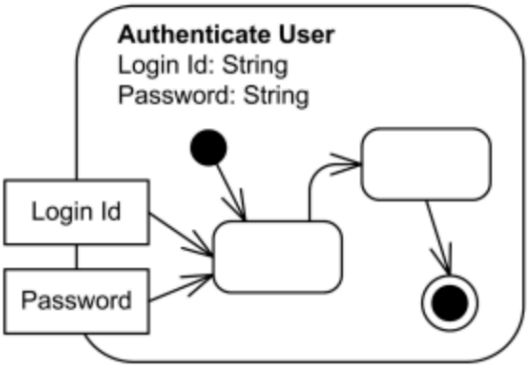
Przykład:



Aktywność `Online Shopping`.

Parametry aktywności są umieszczane na obramowaniu oraz wylosowywane pod nazwą aktywności w formacie `nazwa-parametru: typ-parametru`.

Przykład:

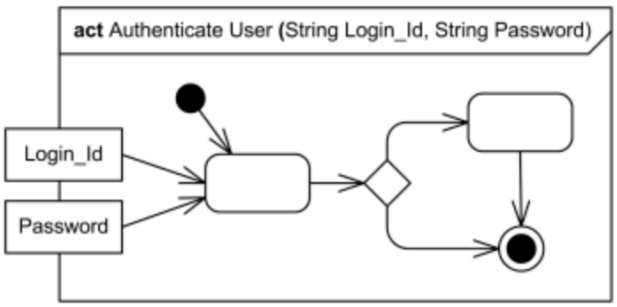


Aktywność `Authenticate User` z dwoma parametrami: `Login Id: String` i `Password: String`.

Jako `Behavior`, `Activity` może mieć ograniczenia wstępne i końcowe (`precondition/postcondition`). Jeśli występują, są oznaczone odpowiednio stereotypami **`<<precondition>>`** i **`<<postcondition>>`**.

Stereotyp **`<<singleExecution>>`** oznacza, że aktywność jest wykonana jako pojedyncze współdzielone wykonanie (singleton). W przeciwnym wypadku, każda instancja aktywności wykonuje się w swojej własnej przestrzeni.

Alternatywna notacja
Zamiast prostokąta z zaokrąglonymi rogami można użyć notacji ramki. Rodzaj ramki to `activity` (skrótowo `act`). Jeśli parametry aktywności istnieją, umieszczane są na ramce.



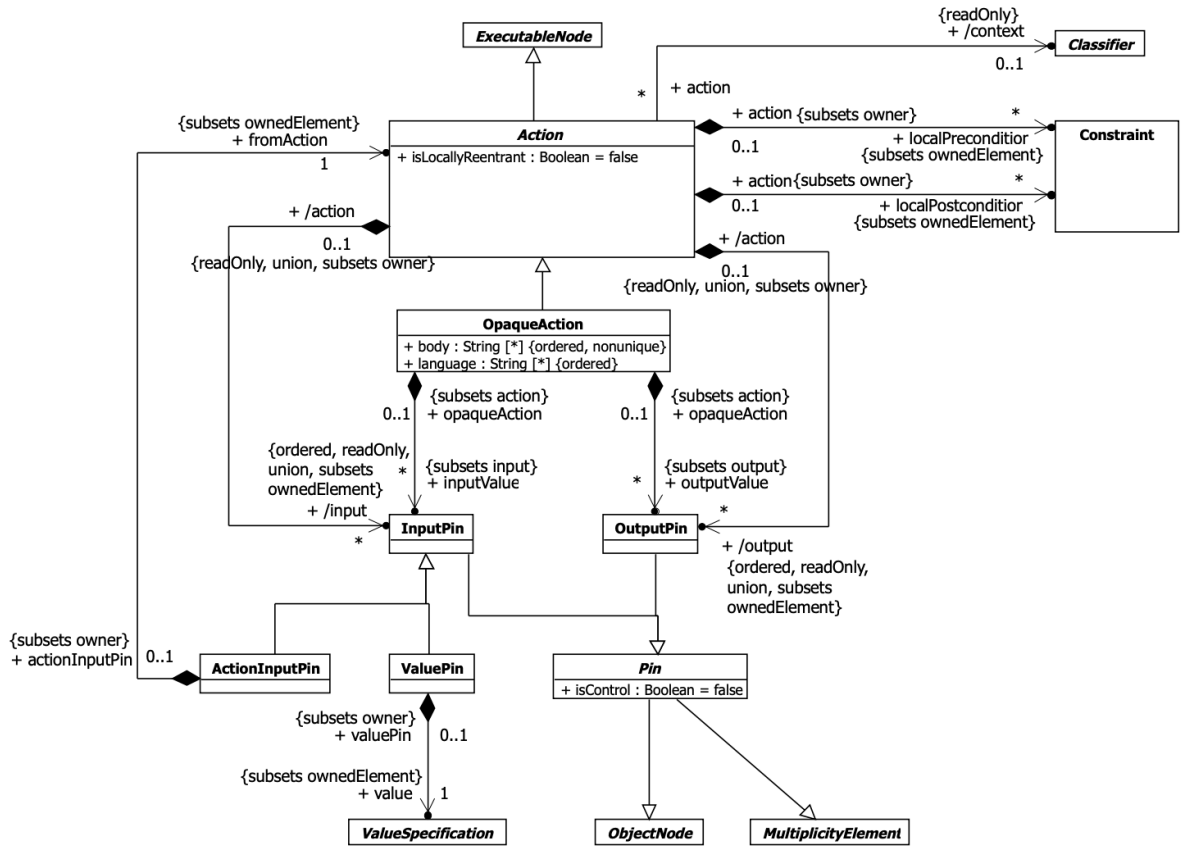
Ramka aktywności `Authenticate User` z dwoma parametrami: `Login Id: String` i `Password: String`.

Można także używać notacji klasy ze stereotypem **`<<activity>>`**, aby wskazać, że dana klasa jest klasą aktywności (*reflective activity*). W razie potrzeby można też stosować notacje dla maszyn stanów i asocjacji. UML dopuszcza również, aby zachowania produkowały tokeny, które same w sobie są aktywnościami i mogą zostać wykonane w czasie działania.

Czynność (`Action`) reprezentuje podstawową jednostkę zachowania, czyli coś, co **system wykonuje** (i coś, co nie jest dalej rozkładane na mniejsze części w danej aktywności). `Action` wykonuje operacje na obiektach, komunikuje się, przetwarza dane i wywołuje inne zachowania**. `Activity` reprezentuje zachowanie złożone z instancji `Action`**.

Należy zwrócić uwagę na fakt, że akcja wywołania zachowania (`CallBehaviorAction`) może odnosić się do aktywności – czyli ją wywoływać. Taka akcja jest prosta z punktu widzenia nadrzędnej aktywności, która ją zawiera, ale może mieć złożony efekt. Aktywność definiuje zachowanie, które można wielokrotnie wykorzystać.

Składnia abstrakcyjna



`Action` jest specjalizacją klasy `ExecutableNode`, co oznacza, że jest elementem wykonywalnym w ramach aktywności (`Activity`). Ponadto, korzysta z właściwości klas `ActivityNode` i `Element`.

Atrybuty klasy `Action`:

- `isLocallyReentrant: Boolean = false` – określa, czy akcja może zostać ponownie wykonana lokalnie, zanim poprzednie wykonanie się zakończy (czyli czy może się rekurencyjnie wywoływać w tym samym kontekście).

Wejścia i wyjścia akcji:

- `+ input: InputPin [*]` – zbiór wejściowych pinów akcji, czyli punktów przyjmujących tokeny danych.
- `+ output: OutputPin [*]` – zbiór wyjściowych pinów – miejsc, do których akcja przekazuje wyniki (tokeny danych).

Wszystkie piny (zarówno wejściowe jak i wyjściowe) dziedziczą po klasie `Pin`, która z kolei dziedziczy po `ObjectNode`. Pin może być:

- `ActionInputPin` – pin wejściowy o dodatkowej semantyce.
- `ValuePin` – specjalny pin z wartością przypisaną bezpośrednio (przez `ValueSpecification`).
- `OutputPin` – pin wyjściowy, przekazuje dane dalej w przepływie aktywności.

Każdy `Pin` ma atrybut `isControl: Boolean = false`, który określa, czy pin przenosi tokeny sterujące (a nie dane).

Ograniczenia:

- `+ localPrecondition: Constraint [*]` – ograniczenia, które muszą być spełnione przed rozpoczęciem działania akcji.
- `+ localPostcondition: Constraint [*]` – ograniczenia, które muszą być spełnione po zakończeniu działania.

Powiązania kontekstowe:

- `+ context: Classifier [0..1]` – określa kontekst wykonania – może to być, np. klasa, komponent lub inny klasyfikator, w ramach którego dana akcja funkcjonuje.

Specjalizacja – `OpaqueAction`:
`OpaqueAction` to jedna z domyślnych specjalizacji `Action`, która:

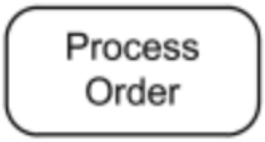
- Zawiera ciało (`body: String [*]`) – zdefiniowaną implementację w danym języku.
- Określa język (`language: String [*]`), np. Java, OCL, SQL.

Jest to typ akcji, którego treść jest zapisana jako tekst i nie jest dalej rozkładana semantycznie.

Notacja w UML

Akcje/czynności przedstawia się jako prostokąty z zaokrąglonymi rogami. Nazwa lub opis akcji znajduje się wewnątrz tego prostokąta.

Przykład:

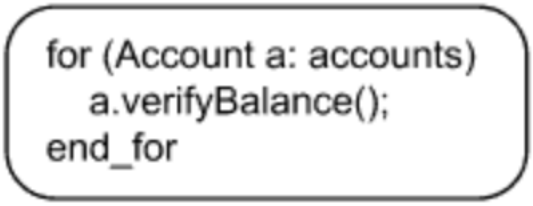


Akcja `Process Order`.

Nazwa akcji ma zwykle formę czasownika (np. `Register User`) lub rzeczownika z doprecyzowaniem. Nie powinno się używać nazw stanów jako nazw akcji.
Przykładowe nazwy akcji:

- `Fill Order` (Wypełnij zamówienie)
- `Review Document` (Przejrzyj dokument)
- `Enroll in Course` (Zapisz się na kurs)
- `Checkout` (Finalizuj zamówienie)
- `Show Error Page` (Pokaż stronę błędu)

Akcję można wyrazić również w zależnym od narzędzia języku akcji (np. pseudokodzie):



3.2 Aktywność: przepływ obiektów, węzły obiektów

Przepływ obiektów (`ObjectFlow`) oraz węzły obiektów (`ObjectNode`) są elementami diagramów aktywności, które umożliwiają modelowanie przepływu danych między różnymi czynnościami w systemie.

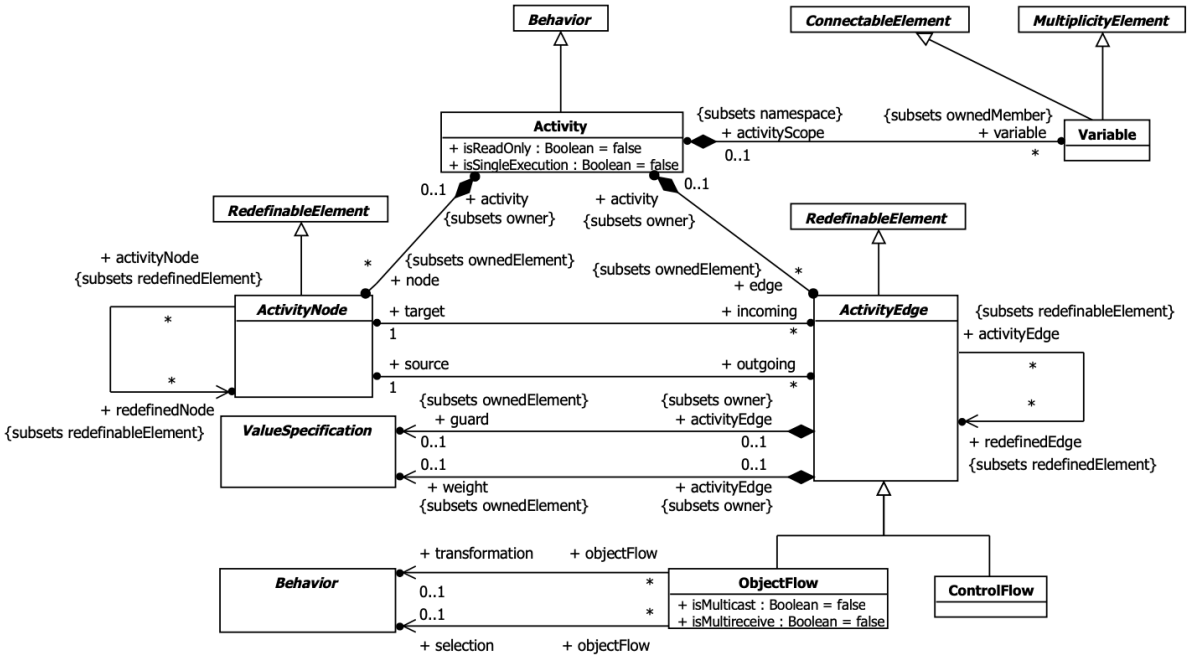
Przepływ obiektów (`ObjectFlow`)

`ObjectFlow` to specjalny rodzaj krawędzi aktywności (`ActivityEdge`), który reprezentuje przepływ obiektów (tokenów danych) między węzłami obiektów (`ObjectNode`). Umożliwia on modelowanie, jak dane są przekazywane od jednego działania do drugiego w trakcie wykonywania aktywności.

Kluczowe cechy `ObjectFlow`:

a) Przenoszenie danych – `ObjectFlow` umożliwia przekazywanie obiektów lub wartości między działaniami.
b) Źródło i cel – każdy `ObjectFlow` łączy dwa węzły obiektów, określając kierunek przepływu danych.
c) Warunki przepływu – można definiować warunki (`guards`), które określają, kiedy dane mogą przepływać przez daną krawędź.
d) Transformacje i selekcje – możliwe jest zastosowanie transformacji danych lub selekcji określonych wartości podczas przepływu.

Składnia abstrakcyjna `ObjectFlow`



`ObjectFlow` dziedziczy po `ActivityEdge`, co oznacza, że jest krawędzią przepływu między dwoma `ActivityNode`. Jest używany do modelowania przepływu obiektów/tokenów danych w diagramie aktywności.

Pozostałe cechy `ObjectFlow` zostały już krótko omówione w poprzednim punkcie.

Węzły obiektów (`ObjectNode`)

`ObjectNode` to węzeł w diagramie aktywności, który reprezentuje miejsce, gdzie obiekty są przechowywane, przetwarzane lub przekazywane dalej. Służy jako punkt wejścia lub wyjścia dla danych w działaniach.

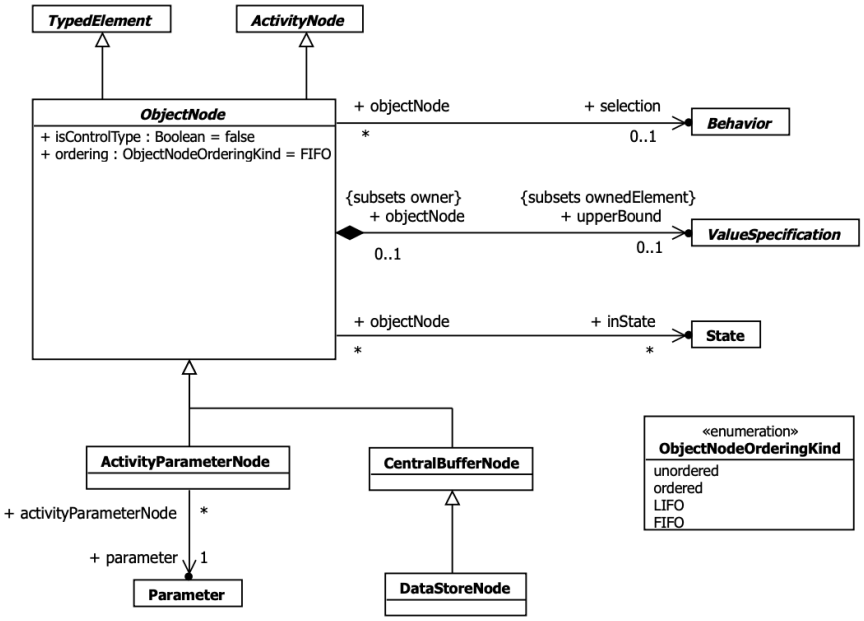
Typy węzłów obiektów:

a) `InputPin` – reprezentuje dane wejściowe dla działania.
b) `OutputPin` – reprezentuje dane wyjściowe z działania.
c) `CentralBufferNode` – służy jako bufor pośredni, przechowując dane między działaniami.
d) `DataStoreNode` – reprezentuje trwałe przechowywanie danych, które mogą być dostępne dla wielu działań.

Właściwości `ObjectNode`:

a) Typ danych – określa klasę lub typ danych, które mogą być przechowywane w węźle.
b) Stan – można określić stan obiektu, który musi być spełniony, aby dane mogły być przetwarzane.
c) Wielokrotność – określa liczbę obiektów, które mogą być przechowywane w węźle.

Składnia abstrakcyjna



`ObjectNode` dziedziczy po:

- `ActivityNode` – czyli może być elementem przepływu w aktywności
- `TypedElement` – czyli może mieć określony typ

Atrybuty `ObjectNode`:

a) `isControlType: Boolean = false`
b) `ordering: ObjectNodeOrderKind = FIFO` – określa sposób kolejkowania tokenów w węźle:
   - `unordered` – brak kolejności
   - `ordered` – dokładna kolejność zgodna z przyjściem
   - `LIFO` (*Last In First Out*) – ostatni przychodzi, pierwszy wychodzi (stos)
   - `FIFO` (*First In First Out*) – pierwszy przychodzi, pierwszy wychodzi (kolejka).

Powiązania i właściwości:

a) `+ selection: Behavior [0..1]` – zachowanie, które pozwala wybrać token z wielu dostępnych w węźle (np. filtr, selektor danych).
b) `+ upperBound: ValueSpecification [0..1]` – ograniczenie liczby tokenów, jakie mogą się znajdować jednocześnie w węźle.
c) `+ inState: State [*]` – tokeny w tym węźle muszą być w podanym stanie (np. `{approved}`), aby mogły być użyte.

Specjalizacje:

a) `ActivityParameterNode`:
   - Służy do reprezentowania danych wejściowych i wyjściowych aktywności.
   - Powiązany z obiektem `Parameter`.

b) `CentralBufferNode`:

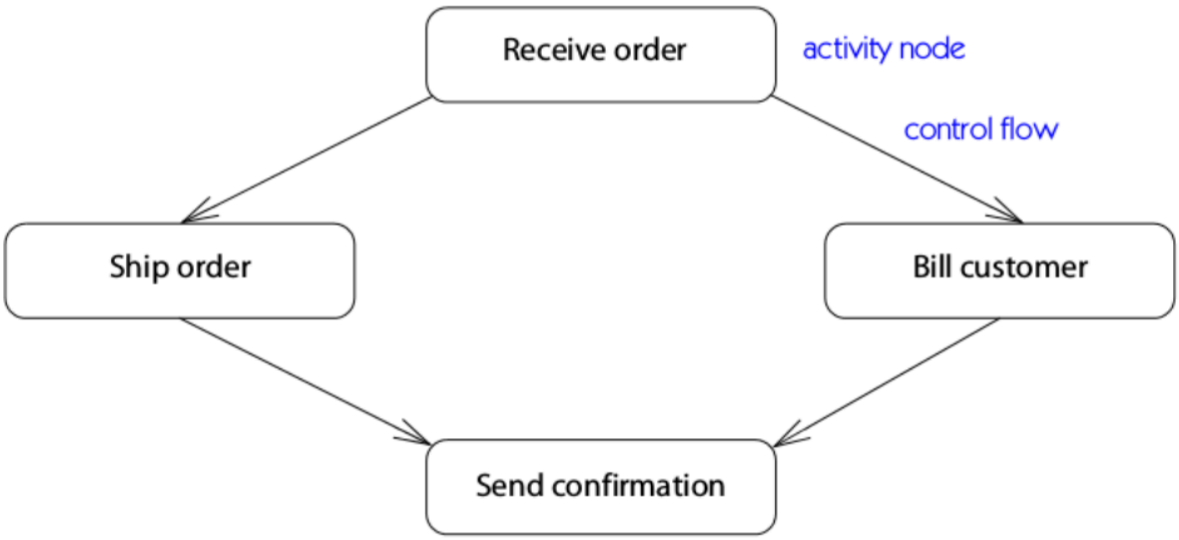
- Bufor pośredni między akcjami – przechowuje tokeny zanim zostaną przekazane dalej.
- Typowy przykład: kolejka.

c) `DataStoreNode` (dziedziczy po `CentralBufferNode`):
   - Przedstawia dane trwałe (np. magazyn, pamięć).
   - Tokeny nie znikają po użyciu – mogą być współdzielone między działaniami.

3.3 Aktywność: przepływ sterowania, węzły sterowania

Przepływ sterowania (`Control Flow`) jest krawędzią aktywności (*activity edge*), która łączy węzły aktywności, określając kolejność ich wykonywania. Reprezentuje on przepływ tokenów sterujących, które nie przenoszą danych, lecz sygnalizują rozpoczęcie wykonania kolejnego węzła.

Cechy przepływu sterowania:
- Tokeny sterujące są przekazywane między węzłami, inicjując ich wykonanie.
- Nie przenoszą danych, służą wyłącznie do kontroli kolejności wykonywania działań.
- Reprezentowane graficznie jako strzałki łączące węzły aktywności:



Węzły sterowania (`Control Nodes`) to specjalne węzły aktywności, które zarządzają przepływem sterowania w diagramie aktywności. Umożliwiają one modelowanie struktur przepływu, takich jak rozpoczęcie, zakończenie, rozgałęzienia czy synchronizacja działań.

Rodzaje węzłów sterowania:

a) Węzeł początkowy (`Initial Node`):
   Jest to węzeł sterujący, w którym rozpoczyna się przepływ, gdy aktywność zostaje wywołana.

Token sterujący zostaje umieszczony w węźle początkowym przy starcie aktywności, z wyjątkiem węzłów wewnętrznych struktur zagnieżdżonych. Tokeny w węźle początkowym są oferowane wszystkim wychodzącym krawędziom. Węzły początkowe są wyjątkiem od reguły, że węzły sterujące nie mogą przechowywać tokenów, jeśli nie mogą ich dalej przekazać (np. przez strażników - *guards*).

Aktywność może mieć więcej niż jeden węzeł początkowy – wtedy rozpoczynają się równoległe przepływy. Jednak przepływ może również rozpocząć się z innych węzłów, więc węzeł początkowy nie jest wymagany do uruchomienia aktywności.

Notacja:



b) Węzeł zakończenia przepływu (`Flow Final Node`):
Jest to węzeł końcowy, który kończy jeden przepływ. Niszczy wszystkie tokeny, które do niego dotrą, ale nie wpływa na inne przepływy aktywności.

Notacja:



c) Węzeł zakończenia aktywności (`Activity Final Node`):
Jest to węzeł kończący całą aktywność. Token, który do niego dotrze, kończy wszystkie działania i niszczy wszystkie tokeny (z wyjątkiem tych w węzłach parametrów wyjściowych). Dotyczy także synchronicznych wywołań – zostają one przerwane.

Jeśli nie chcemy przerywać wszystkich przepływów, należy użyć `Flow Final`.

Notacja:



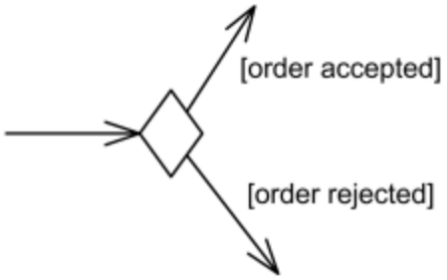
d) Węzeł decyzyjny (`Decision Node`):
Węzeł decyzyjny to węzeł, który przyjmuje tokeny na jednej lub dwóch krawędziach wejściowych i wybiera jedną krawędź wyjściową spośród jednej lub więcej dostępnych. Węzły decyzyjne zostały wprowadzone, aby wspierać warunkowe rozgałęzienia w aktywnościach.

Wszystkie krawędzie wchodzące i wychodzące z węzła decyzyjnego, z wyjątkiem przepływu wejściowego decyzji (*decision input flow*, jeśli występuje), muszą być albo wszystkie przepływami obiektów (`Object Flow`), albo wszystkie przepływami sterowania (`Control Flow`).

Każdy token, który dotrze do węzła decyzyjnego, może przejść tylko jedną z krawędzi wyjściowych. Tokeny nie są duplikowane. Token oferowany przez krawędź wejściową jest oferowany wszystkim krawędziom wyjściowym.

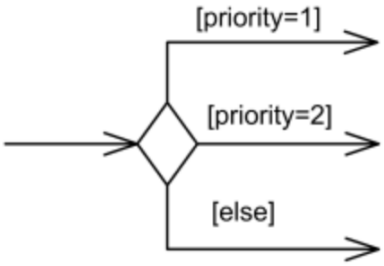
To, która z krawędzi zostanie wybrana rzeczywiście, zależy od oceny warunków (*guards*) przypisanych do krawędzi wyjściowych. Kolejność oceny warunków nie jest zdefiniowana, co oznacza, że nie należy polegać na kolejności ich rozmieszczenia graficznego lub tekstowego.

Notacja:



Węzeł decyzyjny z dwoma wychodzącymi krawędziami, ze strażnikami.

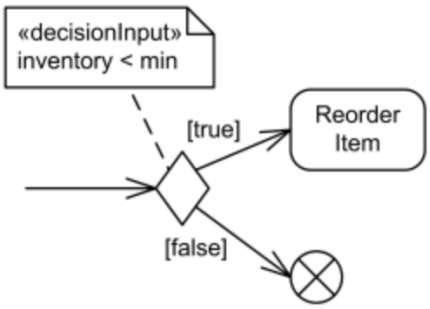
Modelujący powinien zadbać o to, by każdy token mógł przejść tylko jedną krawędź wyjściową. Dla punktów decyzyjnych można określić wstępnie zdefiniowany warunek `else`, ale tylko dla jednej krawędzi wyjściowej.



Węzeł decyzyjny z trzema wychodzącymi krawędziami i strażnikiem `[else]`.

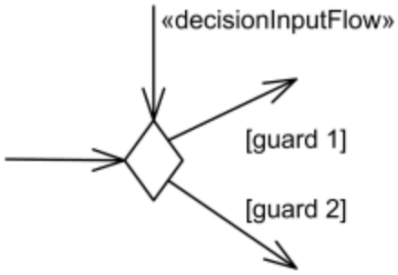
Węzeł decyzyjny może mieć przypisane zachowanie wejścia decyzji. Wprowadzono je, aby uniknąć powielania obliczeń w strażnikach.
W tym przypadku każdy token danych jest przekazywany do tego zachowania przed oceną strażników. Dla tokenów sterujących zachowanie jest wywoływane bez danych wejściowych. Wynik działania tego zachowania jest udostępniany każdemu strażnikowi.

Ponieważ zachowanie to jest używane w trakcie procesu oferowania tokenów do krawędzi wyjściowych, może być wielokrotnie wykonywane dla tego samego tokena – dlatego nie może mieć skutków ubocznych (nie może modyfikować stanu systemu).



Węzeł decyzyjny z zachowaniem wejścia decyzji (zwróć uwagę na stereotyp **<<decisionInput>>**).

Węzeł decyzyjny może mieć również przepływ wejściowy decyzji. Tokeny oferowane na tym przepływie są udostępniane strażnikom krawędzi wyjściowych i pomagają określić, czy token z głównej krawędzi wejściowej powinien przejść daną krawędzią wyjściową.



Węzeł decyzyjny z przepływem wejściowym decyzji.

Jeśli węzeł ma zarówno zachowanie wejścia decyzji, jak i przepływ wejściowy decyzji, token z przepływu wejściowego jest przekazywany do zachowania:

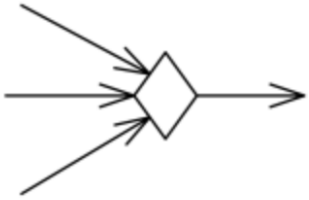
- jako jedyny argument, jeśli główny przepływ wejściowy to `Control Flow`
- jako drugi argument, jeśli główny przepływ wejściowy to `Object Flow`.

W takim przypadku, węzeł decyzyjny oferuje tokeny krawędziom wyjściowym tylko wtedy, gdy na każdej krawędzi wejściowej pojawi się token.

e) Węzeł scalający (`Merge Node`):
Węzeł scalający to węzeł sterujący, który scala wiele naprzemiennych przepływów wejściowych do jednego przepływu wyjściowego. Nie następuje tu łączenie tokenów – merge nie powinien być używany do synchronizacji przepływów równoległych.

Przykład:
Jeśli po rozwidleniu (*fork*) występuje decyzja, dwa przepływy wychodzące z tej decyzji należy złączyć przed przejściem do węzła synchronizacji (*join*). W przeciwnym razie,

węzeł *join* będzie czekał na oba przepływy, mimo że tylko jeden z nich faktycznie nadejdzie.
Wszystkie krawędzie wchodzące i wychodzące z węzła scalającego muszą być albo przepływami obiektów albo przepływami sterującymi – nie można ich mieszać.

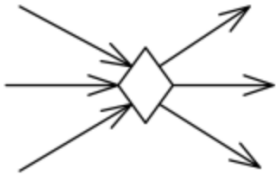


Węzeł scalający z trzema krawędziami wchodzącymi i jedną krawędzią wychodzącą.

Funkcjonalność węzła scalającego i węzła decyzyjnego może zostać połączona w jednym symbolu. Taki przypadek można odwzorować jako:

- węzeł scalający z wszystkimi krawędziami wejściowymi pokazanymi na diagramie
- węzeł decyzyjny z wszystkimi krawędziami wyjściowymi pokazanymi na diagramie.

Innymi słowy, można użyć tego samego rombu do reprezentacji punktu decyzji i scalania, ale semantycznie to dwa różne węzły w modelu UML.

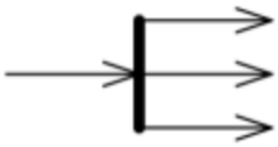


Węzeł scalający i węzeł decyzyjny połączone za pomocą jednego symbolu.

f) Węzeł rozwidlenia (`Fork Node`):
Jest to węzeł sterujący, który ma jedną krawędź wejściową i wiele krawędzi wyjściowych. Służy do rozbicia jednego przepływu na wiele przepływów równoległych. Węzły rozwidlenia zostały wprowadzone w celu wsparcia równoległości w aktywnościach.

Tokeny, które docierają do węzła rozwidlenia są duplikowane na wszystkie wychodzące krawędzie.
Jeśli przynajmniej jedna krawędź wyjściowa przyjmie token, tworzona jest jego kopia dla każdej krawędzi, która go zaakceptuje.
Krawędzie, które nie przyjmą tokena (np. z powodu braku gotowości ich celów), przechowują kopię tokena w niejawnej kolejce FIFO, aż będzie mogła zostać odebrana.
Pozostałe krawędzie wyjściowe nie otrzymują tokena.

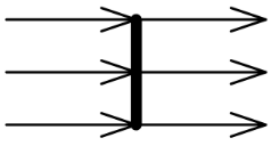
Węzeł rozwidlenia przedstawiany jest jako odcinek linii (pionowy lub poziomy) z:
- jedną krawędzią wejściową
- dwiema lub więcej krawędziami wyjściowymi.



Węzeł rozwidlenia z jedną krawędzią wejściową i trzema wyjściowymi.

Funkcjonalność węzła rozwidlenia i węzła synchronizacji można połączyć w jednym symbolu. W takim przypadku:
- w modelu istnieje węzeł synchronizacji z wszystkimi krawędziami wejściowymi pokazanymi na diagramie
- a następnie węzeł rozwidlenia z wszystkimi krawędziami wyjściowymi.

Czyli wizualnie widoczny jest jeden element, ale semantycznie to połączenie dwóch osobnych węzłów.

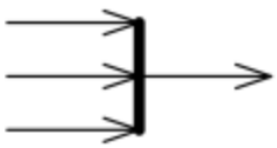


Węzeł synchronizacji i węzeł rozwidlenia połączone za pomocą jednego symbolu.

Jeśli na wychodzących krawędziach z węzła rozwidlenia stosuje się strażniki (*guards*), modelujący powinien upewnić się, że żaden węzeł synchronizacji nie zależy od nadejścia tokenów przez taką krawędź z warunkiem. Jeśli nie da się tego uniknąć, należy zastosować węzeł decyzyjny, który zawiera strażnik i przekierowuje token do odpowiedniego miejsca, jeśli warunek nie jest spełniony.

g) Węzeł synchronizacji (`Join Node`):
Jest to węzeł, który posiada wiele krawędzi wejściowych i jedną krawędź wyjściową. Używany jest do synchronizacji równoległych przepływów przychodzących. Węzły synchronizacji zostały wprowadzone, w celu obsługi równoległości w aktywnościach.

Węzeł synchronizacji przedstawia się jako odcinek linii, do którego wchodzi kilka krawędzi aktywności, a wychodzi tylko jedna.

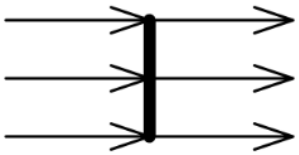


Węzeł synchronizacyjny z trzema krawędziami wejściowymi i jedną wyjściową.

Funkcjonalność węzła synchronizacji i rozwidlenia może zostać połączona za pomocą jednego symbolu. W takim przypadku:

- w modelu znajduje się węzeł synchronizacji z wszystkimi pokazanymi krawędziami wejściowymi
- a następnie węzeł rozwidlenia z wszystkimi pokazanymi krawędziami wyjściowymi.

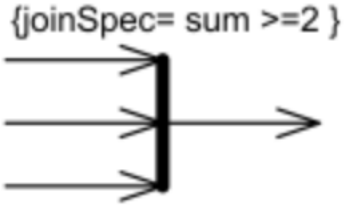
Czyli wizualnie widoczny jest jeden element, ale semantycznie to połączenie dwóch osobnych węzłów.



Węzeł synchronizacyjny i węzeł rozwidlenia połączone za pomocą jednego symbolu.

Węzły synchronizacji mogą mieć specyfikację `joinSpec`, która jest wyrażeniem logicznym, wykorzystującym nazwy krawędzi wejściowych, aby określić warunki, przy których węzeł przekaże token dalej.
Specyfikacja jest oceniana za każdym razem, gdy token zostaje zaoferowany na którejkolwiek krawędzi wejściowej.
Ocena nie jest przerywana, jeśli w międzyczasie zostaną zaoferowane nowe tokeny.
Nie uruchamia się równoległych ocen – każda nowa ocena czeka na zakończenie poprzedniej.

Domyślna specyfikacja: `and` oznacza, że token musi zostać zaoferowany na każdej krawędzi wejściowej, aby `join` przekazał token dalej.
Specyfikację joinSpec umieszcza się w nawiasach klamrowych obok węzła jako `{joinSpec = ...}`



Węzeł synchronizacji ze specyfikacją pokazaną w nawiasach klamrowych.

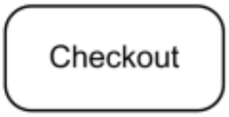
3.4 Aktywność: węzły wykonawcze (*executable nodes*)

Węzeł wykonawczy (`ExecutableNode`) jest rodzajem węzła aktywności (`ActivityNode`), który może być wykonany jako krok w ogólnym pożądanym zachowaniu zawartej w sobie aktywności (`Activity`).
Zazwyczaj węzły sterujące (`ControlNodes`) oraz węzły obiektowe (`ObjectNodes`) w ramach aktywności służą głównie do kontrolowania kolejności wykonania oraz zarządzania przepływem danych pomiędzy węzłami wykonawczymi tej aktywności.

Do węzłów wykonawczych w kontekście diagramów aktywności zalicza się wszelkie **typy akcji**. W UML istnieje wiele rodzajów akcji (czynności), lecz każda z nich jest odmianą węzła wykonawczego.
Do rodzajów akcji należą:

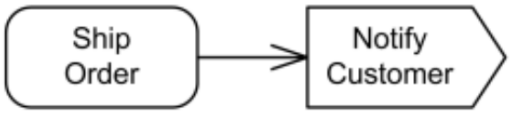
a) Akcje proste – np. operacje typu `Fill Order`, `Review Document` czy `Show Error Page`, będące pojedynczymi czynnościami wykonywanymi w procesie.

b) Akcje wywołania – są to akcje inicjujące wykonanie innego zachowania. Należą do nich m.in. wywołanie aktywności (*call behavior action*) lub wywołanie operacji (*call operation action*). Taka akcja powoduje uruchomienie innej zdefiniowanej aktywności jako kroku wewnątrz bieżącego procesu.



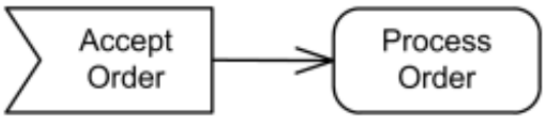
Akcja wywołania aktywności dla zachowania `Checkout`.

c) Akcje wysyłania sygnału – są to akcje inicjujące asynchroniczny sygnał kierowany do docelowego obiektu (adresata). Przykładem jest *send signal action*, która tworzy sygnał na podstawie danych wejściowych i wysyła go do wskazanego obiektu docelowego. Po wysłaniu sygnału wykonanie procesu natychmiast idzie dalej – nadawca nie czeka na odpowiedź.



Po tym jak zamówienie jest dostarczone, akcja wysyłania sygnału akcji `Notify Customer` tworzy i wysyła sygnał `Notify Customer`.

d) Akcje odbioru zdarzenia – są to akcje oczekujące na nadejście określonego zdarzenia (sygnału) zanim przekażą tok dalej. Tak zwany *accept event action* rozpoczyna nasłuch zdarzenia od momentu startu aktywności zawierającej tę akcję (jeśli nie ma ona wejściowych strzałek). Dopiero, gdy oczekiwany sygnał zostanie przechwycony, akcja kończy się i może przekazać token do kolejnych kroków procesu.

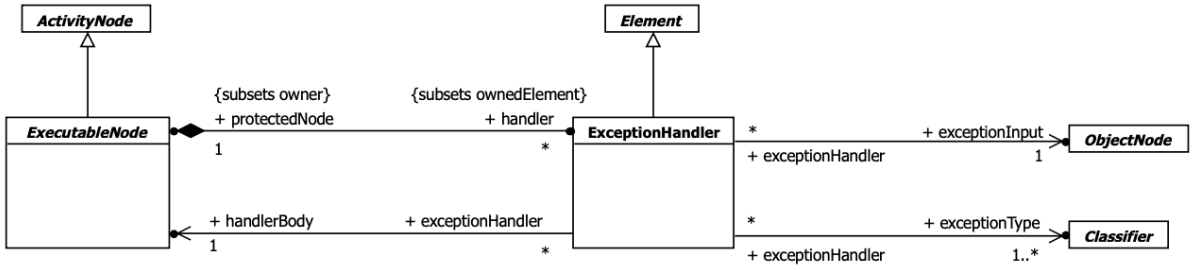


Akceptacja zdarzenia `Accept Order` przez akcję `Accept Order` powoduje wywołanie akcji `Process Order`. Akcja akceptacji zdarzenia jest aktywowana na wejściu zachowania, które je zawiera.

e) Inne wyspecjalizowane akcje – UML definiuje także szereg dodatkowych akcji, np. akcje czasu (czekające określony czas lub do ustalonej chwili), akcje zarządzania

zmiennymi, akcje operujące na powiązaniach (*link actions* związane z asocjacjami), akcje tworzenia i niszczenia obiektów, testowania identyczności obiektów itp. Wszystkie one również są traktowane jako węzły wykonawcze, ponieważ każda z tych akcji odpowiada za wykonanie konkretnej operacji w modelowanym procesie.

Składnia abstrakcyjna



`ExecutableNode` jest specjalizacją `ActivityNode`, czyli węzła występującego wewnątrz aktywności. Jest to węzeł, który może zostać uruchomiony, tzn. wykonać jakąś operację w ramach przepływu aktywności.

Właściwości i relacje:

a) `protectedNode: ExecutableNode [1]`:
   - każdy `ExecutableNode` może mieć jeden lub więcej handlerów wyjątków, które go chronią.
   - relacja kompozycji oznaczona jako `{subsets owner}` wskazuje, że handler należy do węzła, który chroni.

b) `handler: ExceptionHandler [*]`:
   - węzeł wykonawczy może mieć wiele handlerów wyjątków. Każdy handler związany jest z jednym `ExecutableNode`, który chroni.
   - jest to odniesienie odwrotne do `protectedNode`.

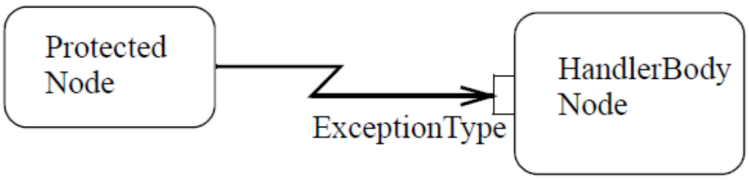
c) `handlerBody: ExecutableNode [1]`:
   - handler definiuje, jaki inny węzeł wykonawczy powinien zostać wykonany, gdy nastąpi wyjątek.
   - jest to operacja alternatywna względem standardowego przepływu, np. alternatywny scenariusz lub logika naprawcza.

Sam `ExceptionHandler` jest obiektem obsługującym wyjątki.

d) `exceptionInput: ObjectNode [1]` – definiuje wejściowych `ObjectNode`, który przechowuje przekazany wyjątek. Służy do przekazania danych wyjątków do handlera.

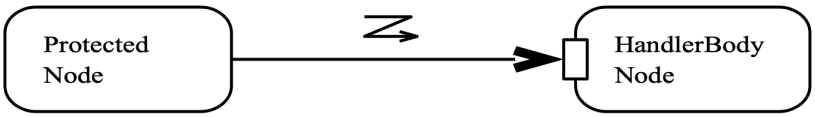
e) `exceptionType: Classifier [1..*]` – określa typ (klasę) wyjątku, który może być obsłużony przez dany `ExceptionHandler`. Można podać jeden lub więcej typów, które ten handler rozpoznaje.

Notacja `ExceptionHandler`:
„Obsługiwacz wyjątków" jest przedstawiany za pomocą symbolu błyskawicy (*zig-zag*), biegnącego od obramowania `ProtectedNode` do małego kwadratu znajdującego się na obramowaniu `ExceptionHandler`.
Nazwa typu wyjątku umieszczana jest obok symbolu błyskawicy. Mały kwadrat reprezentuje węzeł wejścia wyjątku (`exceptionInput node`).



Do jednego `ProtectedNode` może być przypisanych wiele instancji `ExceptionHandler`, każda oznaczona własną błyskawicą.

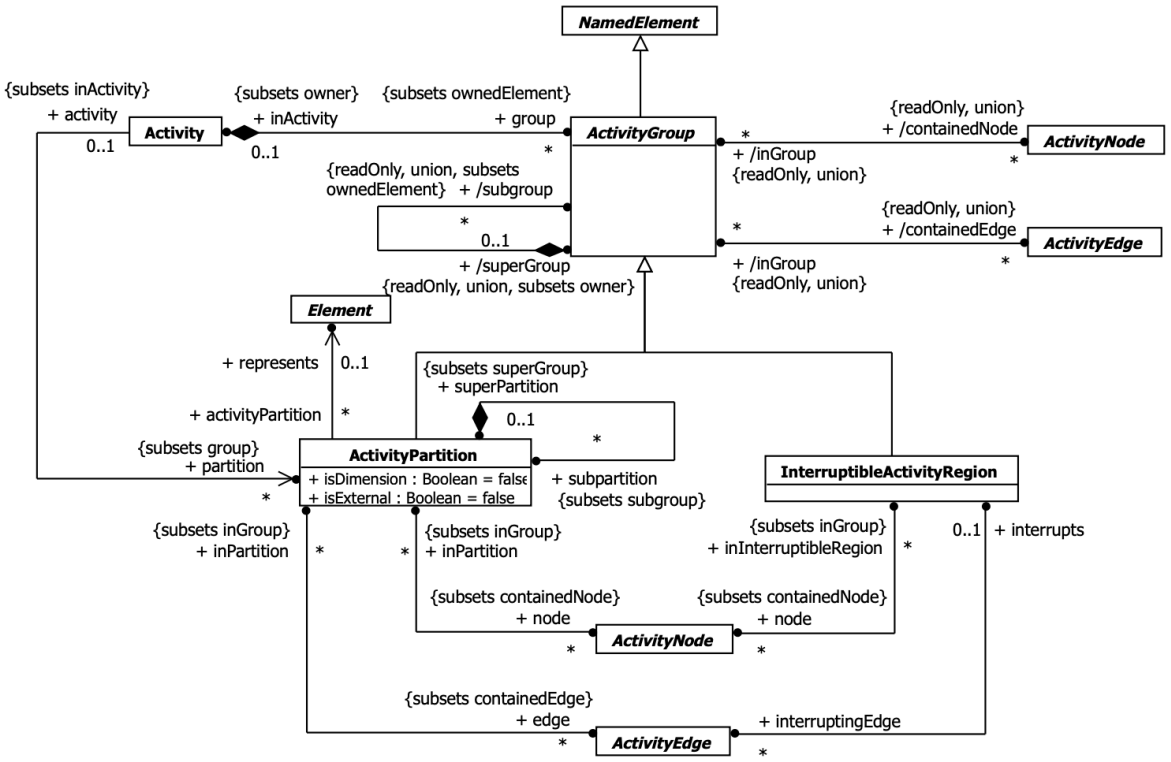
Notacja alternatywna dla `ExceptionHandler` w postaci ozdobnika ze strzałką w postaci błyskawicy:



3.5 Aktywność: grupy aktywności (*Activity Groups*), partycje aktywności (*Activity Partitions*)

Grupy aktywności (`ActivityGroups`) są strukturami grupującymi dla węzłów aktywności (`ActivityNodes`) i krawędzi aktywności (`ActivityEdges`). Węzły i krawędzie mogą należeć do więcej niż jednej grupy.

Składnia abstrakcyjna



Dzięki dziedziczeniu po `NamedElement`, `ActivityGroup` może posiadać nazwę oraz może być identyfikowane w modelu.

Relacje z innymi klasami:

a) `+group: ActivityGroup [*]` – odniesienie do wszystkich grup przypisanych do jednej aktywności – oznacza, że aktywność (`Activity`) może zawierać wiele grup poprzez atrybut, który jest podzbiorem `ownedElement`.

b) `+inActivity: Activity [0..1]` – określa, do której aktywności należy dana grupa. Jest to relacja kompozycji, więc grupa nie może istnieć bez przepisanej aktywności.

c) `+/subgroup` i `+/superGroup` – grupy mogą być zagnieżdżane hierarchicznie, tzn. jedna grupa może zawierać inne grupy a także sama może być częścią innej grupy.

d) `+containedEdge: ActivityEdge [*]` – odnosi się do krawędzi przepływu logicznie należących do grupy.

Specjalizacje `ActivityGroup`:

a) `ActivityPartition` – służy do przedstawienia ról lub elementów systemu, które uczestniczą w wykonywaniu aktywności.

b) `InterruptibleActivityRegion` – reprezentuje region przerywalny, w którym węzły mogą zostać przerwane przez inne zdarzenia.

Zawiera:
- `ActivityNode` i `ActivityEdge`
- `interruptingEdge` – krawędź, która powoduje przerwanie przepływu
- `interrupts` – wskazuje aktywność, która może być przerwana.

Partycja aktywności (`ActivityPartiton`) to rodzaj `ActivityGroup`, służący do identyfikowania węzłów aktywności, które mają pewne wspólne cechy. `ActivityPartition` mogą dzielić zawartość. Często odpowiadają jednostkom organizacyjnym w modelu biznesowym. Mogą być używane do przypisywania cech lub zasobów do węzłów w aktywności.

`ActivityPartition` nie wpływają na przepływ tokenów w modelu. Ograniczają i dostarczają widoku na zachowania (`Behaviors`) wywołane w wyniku wykonania węzłów (`containedNodes`) i krawędzi (`containedEdges`) zawartych w partycji, w tym wywołań operacji (`Operation calls`) i wysyłania sygnałów (`Signal sends`). Może to wynikać nie tylko z jawnych akcji wywołań (`InvocationActions`), ale także z niejawnych wywołań, np. zachować transformacji (`transformation`) i specyfikacji (`specification Behaviors`).

Ograniczenia różnią się w zależności od rodzaju elementu, który partycja reprezentuje, zgodnie z poniższymi zasadami:

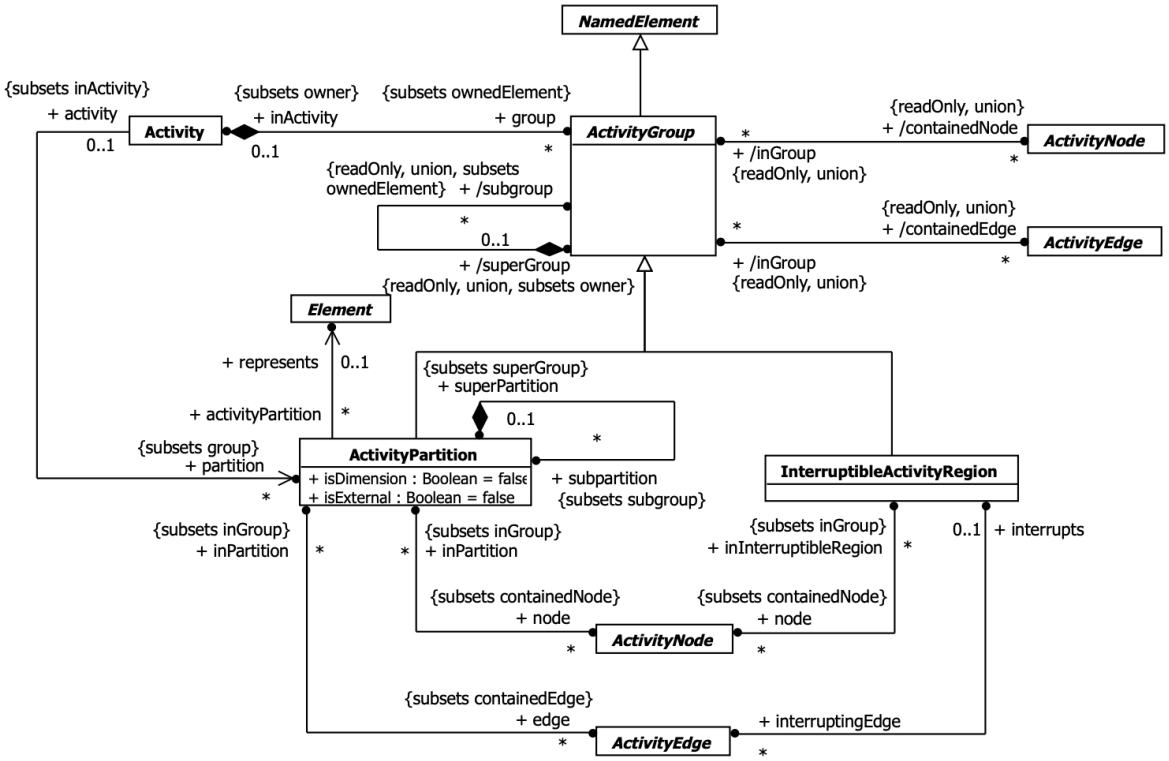
a) `Classifier` (klasyfikator) – zachowania wywołane w obrębie partycji są odpowiedzialnością instancji klasyfikatora reprezentowanego przez partycję. Kontekst wszystkich wywoływanych zachowań musi być tym klasyfikatorem. Wywołania operacji i wysyłanie sygnałów w obrębie partycji muszą w czasie wykonywania być skierowane do obiektów, które są instancjami tego klasyfikatora.

b) `InstanceSpecification` (specyfikacja instancji) – zachowania wywoływane w obrębie partycji są odpowiedzialnością instancji modelowanej przez specyfikację instancji, którą partycja reprezentuje. Kontekst wszystkich wywoływanych zachowań musi być klasyfikatorem tej specyfikacji instancji. Wywołania operacji i wysyłanie sygnałów w obrębie partycji muszą być skierowane do instancji reprezentowanych przez tę specyfikację.

c) `Property` (właściwość) – zachowania wywoływane w obrębie partycji są odpowiedzialnością instancji (jednej lub więcej) przechowywanych przez właściwość reprezentowaną przez partycję. Kontekst wszystkich wywoływanych zachowań musi być typem tej właściwości. Wywołania operacji i wysyłanie sygnałów w obrębie partycji muszą być skierowane do instancji przechowywanej przez tę właściwość w czasie wykonania. Jeśli właściwość przechowuje więcej niż jedną wartość, wywołania traktuje się tak, jakby były wykonywane równolegle na każdej wartości, a dane

wywołanie nie kończy się, dopóki wszystkie równoległe instancje nie zakończą działania.

Składnia abstrakcyjna



Dzięki dziedziczeniu po `ActivityGroup`, `ActivityPartition` może posiadać nazwę (przez `NamedElement`), zawierać węzły i krawędzie (`containedNode` i `containedEdge`) oraz należeć do aktywności (poprzez `inActivity`).

Atrybuty:

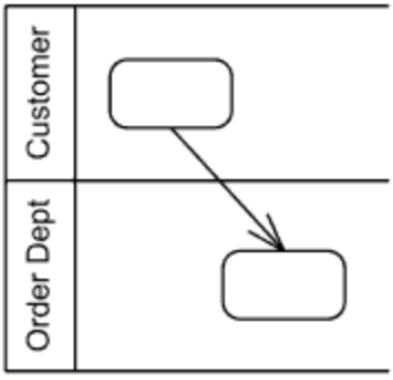
a) `isDimension: Boolean = false` – określa, czy partycja jest wymiarowa, czyli czy reprezentuje sposób podziału innych partycji.
b) `isExternal: Boolean = false` – oznacza, czy partycja reprezentuje coś spoza modelowanej aktywności (np. zewnętrzny system).

ActivityPartition może tworzyć hierarchie partycji:

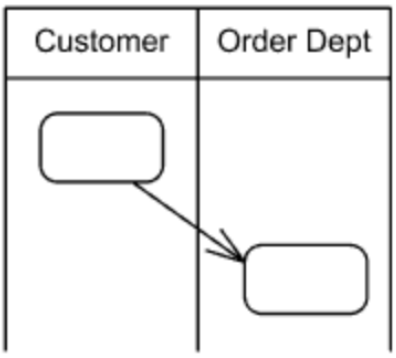
- `superPartition: ActivityPartition [0..1]`
- `subpartition: ActivityPartition [*]`
- umożliwia podział logiczny, np. według działów i ról w dziale.

Notacja w UML

`ActivityPartition` może być przedstawiona za pomocą **notacji torów pływackich** (*swimlane*), czyli z użyciem dwóch, zazwyczaj równoległych linii, poziomych lub pionowych oraz nazwy partycji umieszczonej w ramce na jednym z końców. Każdy węzeł aktywności oraz krawędzie umieszczone pomiędzy tymi liniami są uznawane za należące do tej partycji.

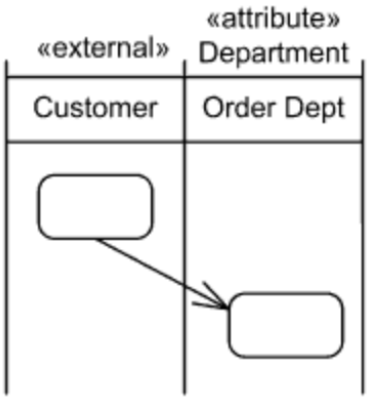


Partycje aktywności `Customer` i `Order Dept` jako poziome tory pływackie.



Partycje aktywności `Customer` i `Order Dept` jako pionowe tory pływackie.

Hierarchiczne partycjonowanie jest przedstawiane poprzez użycie torów pływackich dla podpartycji:



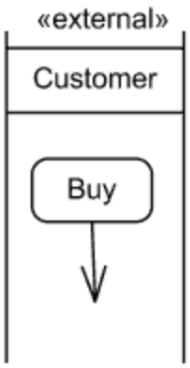
Partycja nadrzędna zawierająca zagnieżdżone partycje podrzędne.

Partycja może być oznaczona jako wymiar (*dimension*) dla swoich podpartycji, aby grupować je wzdłuż danego wymiaru. Na przykład, dana aktywność może mieć jeden wymiar partycji odpowiadający lokalizacji, w której działania są wykonywane, oraz drugi wymiar odpowiadający kosztowi ich realizacji. Partycje będące wymiarami nie mogą być zawarte w żadnej innej partycji.

Diagramy mogą być również partycyjnie wielowymiarowe, gdzie każda komórka toru (*swim cell*) jest przecięciem wielu partycji. Partycje w obrębie danego wymiaru mogą być zgrupowane w nadrzędną partycję aktywności z ustawieniem `isDimension = true`, której nazwą jest nazwa tego wymiaru. Zamiast przedstawiać sam wymiar jako partycję, jego nazwa jest umieszczana obok zestawu partycji należących do tego wymiaru.

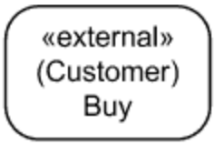
Partycja może reprezentować jednostkę zewnętrzną, do której struktura partycjonowania nie ma zastosowania. Zewnętrzne partycje są celowymi wyjątkami od zasad struktury partycji. Na przykład, dany wymiar może zawierać partycje reprezentujące części strukturalnego klasyfikatora, ale może też zawierać zewnętrzną partycję, która nie reprezentuje żadnej z jego części, lecz zupełnie oddzielny klasyfikator. W modelowaniu biznesowym zewnętrzne partycje mogą służyć do odwzorowania jednostek spoza firmy.

Jeśli aktywność jest uważana za występującą poza zakresem danego modelu, partycję można oznaczyć stereotypem **`<<external>>`**, nadpisuje to oznaczenia *swimlane* i wymiaru.



Akcja `Buy` odbywa się w zewnętrznej partycji `Customer`.

W sytuacjach, gdy nie można użyć *swimlane* do pokazania partycji, można zastosować alternatywną notację tekstową z kwalifikowaną nazwą akcji. W tym przypadku nazwa partycji jest umieszczona w nawiasie nad nazwą akcji. Lista nazw partycji rozdzielona przecinkami oznacza, że dany węzeł należy do więcej niż jednej partycji. Podwójny dwukropek w nazwie partycji oznacza partycję zagnieżdżoną, gdzie wcześniejsze człony odnoszą się do partycji nadrzędnych.



Akcja `Buy` odbywa się w zewnętrznej partycji `Customer`.

C10 3.6-3.8

3.6 Akcja: podział na rodzaje, funkcja przekaźników (*Pins*)

Akcje (`Actions`) to podstawowe, niepodzielne jednostki zachowania w ramach diagramów aktywności. Reprezentują pojedynczy, atomowy krok w wykonywaniu procesu. Akcje są zawsze częścią większej aktywności i mogą mieć różny charakter w zależności od tego, co robią.

Akcje można podzielić na kilka rodzajów:

a) Akcje obiektów (`Object Actions`)
   Służą do manipulowania obiektami i ich stanem (np. tworzenia, usuwania i odczytywania informacji o obiektach).
   Przykłady:
   - `CreateObjectAction` – tworzy nową instancję obiektu.
   - `DestroyObjectAction` – usuwa obiekt.
   - `ReadSelfAction` – odwołuje się do obiektu kontekstowego (np. `this`).

b) Akcje zmiennych (`Variable Actions`)
   Pozwalają odczytywać i zapisywać wartości zmiennych w ramach aktywności.
   Przykłady:
   - `ReadVariableAction` – pobiera wartość zmiennej.
   - `WriteVariableAction` – przypisuje wartość zmiennej.

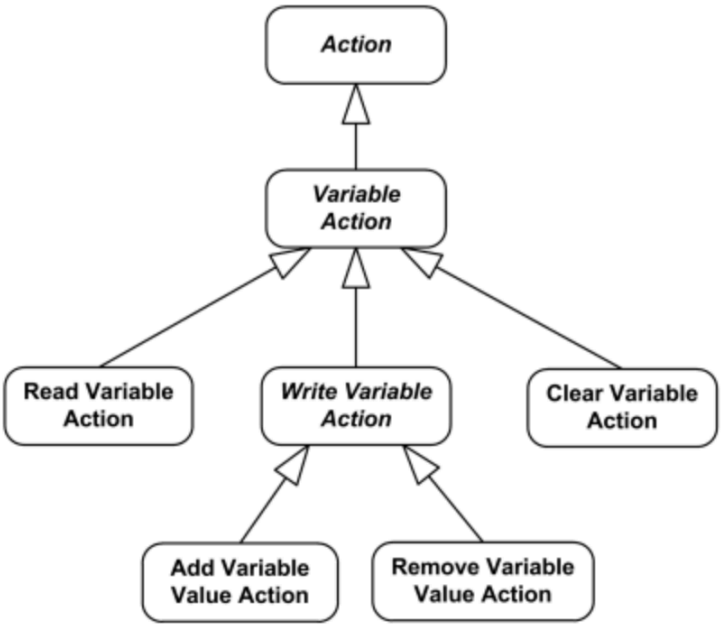


Diagram poglądowy akcji zmiennych.

c) Akcje wywołania (`Invocation Actions`)

Służą do wywoływania innych zachowań lub operacji.
Przykłady:

- `CallBehaviorAction` – bezpośrednie wywołanie zachowania (np. aktywności).
- `CallOperationAction` – wywołanie operacji na obiekcie (metody).
- `SendSignalAction` – wysyłanie sygnału (asynchronicznie).

d) Akcje cech strukturalnych (`Structural Feature Actions`)
Pozwalają odczytywać lub modyfikować Atrybuty obiektów.
Przykłady:

- `ReadStructuralFeatureAction` – odczytuje wartość atrybutu.
- `WriteStructuralFeatureAction` – zmienia wartość atrybutu.

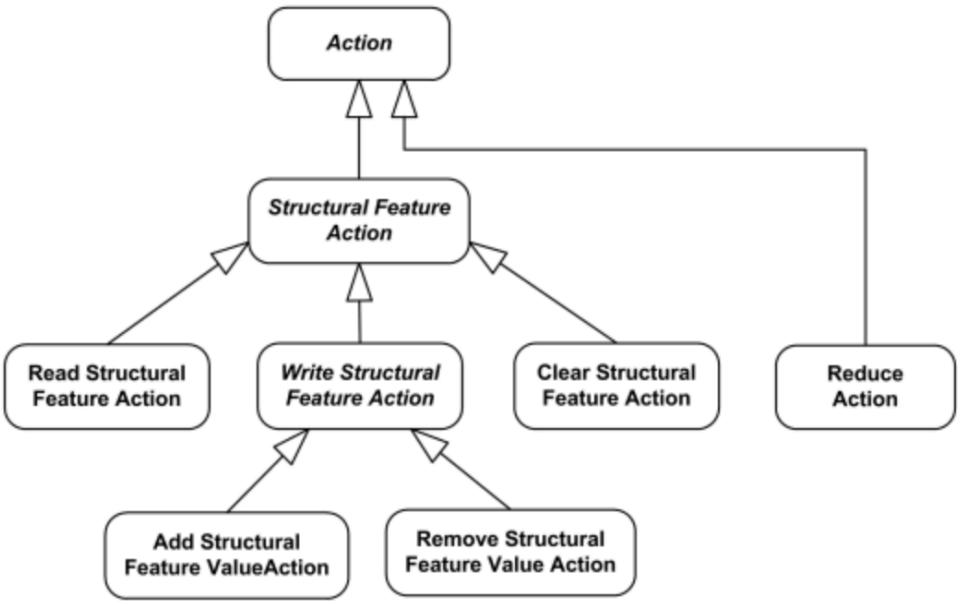
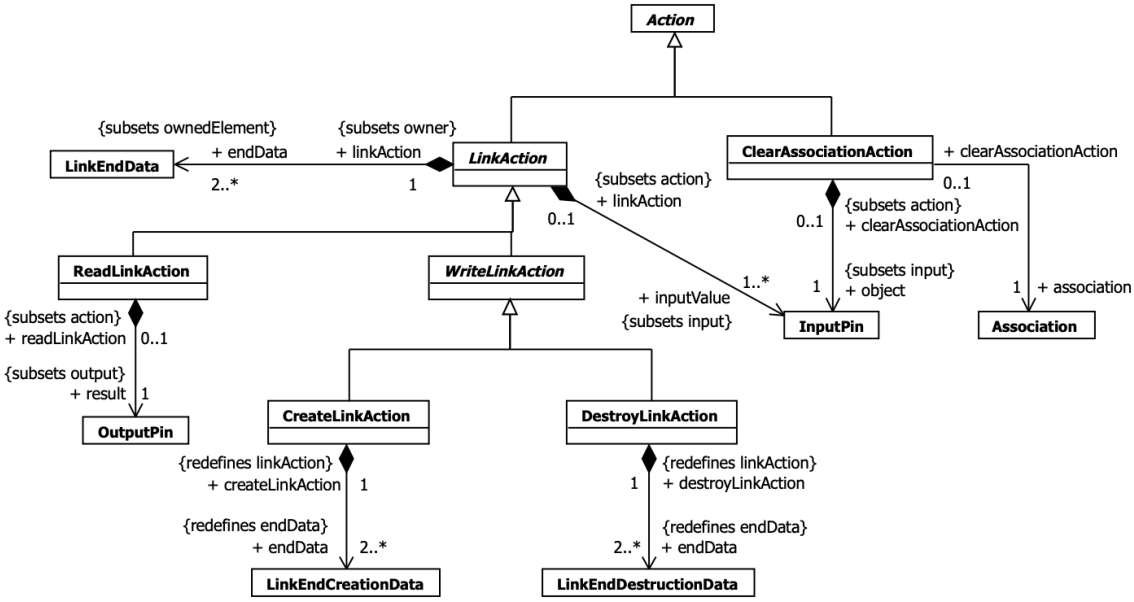


Diagram poglądowy akcji cech strukturalnych.

e) Akcje połączeń/linków (`Link Actions`)
Akcje połączeń obejmują kilka akcji działających na asocjacjach: odczyt, zapis, tworzenie, usuwanie oraz czyszczenie powiązań.

Składnia abstrakcyjna



Na podstawie składni abstrakcyjnej można wyróżnić kilka rodzajów akcji połączeń:

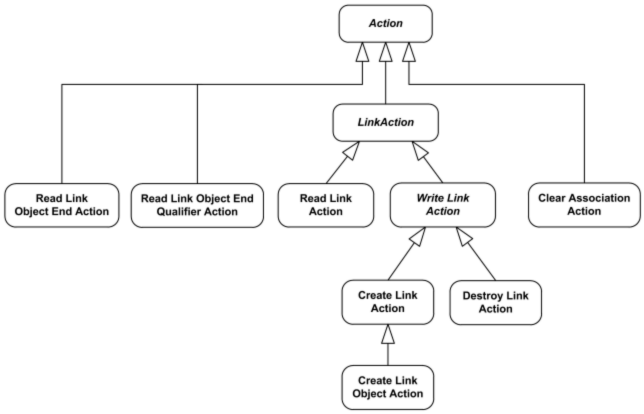
- `ReadLinkAction` – odczytuje informacje z istniejącego powiązania (asocjacji). Wynik jest przekazywany na OutputPin.

- `WriteLinkAction` – nadklasa dla działań modyfikujących powiązania.

- `CreateLinkAction` – tworzy nowe powiązanie między obiektami. Wymaga zdefiniowania co najmniej dwóch końców powiązań (`LinkEndCreationData`).

- `DestroyLinkAction` – usuwa istniejące powiązania między obiektami. Wymaga określenia końców powiązań.

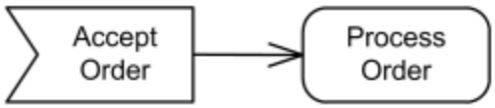
- `ClearAssociationAction` – usuwa wszystkie powiązania wskazanej asocjacji dla danego obiektu.



Ogólny diagram poglądowy hierarchii akcji połączeń.

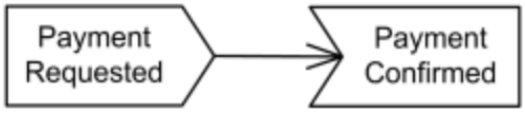
f) Akcje zdarzeń (`Accept Event Actions`)
Służą do przyjmowania i obsługi zdarzeń (*event occurances*). Ich głównym zadaniem jest oczekiwanie na wystąpienie określonego zdarzenia i rozpoczęcie dalszego przetwarzania, gdy to zdarzenie nastąpi.
Przykłady:

- `AcceptEventAction` – czeka na dowolne zdarzenie (np. sygnał, wywołanie, zdarzenie czasowe). Akcja akceptacji zdarzenia jest oznaczana jako wklęsły pięciokąt. Domyślnie nazwa tej akcji odpowiada nazwie zdarzenia, które akcja akceptuje, tak więc np. akcja akceptacji zdarzenia o nazwie `Accept Order` oczekuje na zdarzenie `Accept Order`.



Akceptacja zdarzenia `Accept Order` powoduje wywołanie akcji `Process Order`.

- `AcceptSignalAction` – oczekuje na konkretne zdarzenie typu sygnał. Akcja akceptacji sygnału jest oznaczana w taki sam sposób jak akcja akceptacji zdarzenia, czyli wklęsły pięciokąt. Domyślnie nazwa akcji akceptacji sygnału odpowiada nazwie sygnału, który ta akcja akceptuje.



Sygnał `Payment Requested` jest wysyłany. Następnie, aktywność czeka na otrzymanie sygnału `Payment Confirmed`.

- `AcceptCallAction` – oczekuje na wywołanie operacji (np. zdalne żądanie wykonania jakiejś funkcji).
- `AcceptTimeEventAction` – oczekuje na upływ określonego czasu lub wystąpienie zdarzenia czasowego.

Wszystkie te akcje blokują przepływ, dopóki zdarzenie nie nastąpi.

g) Akcje strukturalne (`Structured Actions`)
Grupują inne Akcje w ramach jednej, większej jednostki. Pozwalają tworzyć bardziej złożone fragmenty logiki w jednym „opakowaniu”, zamiast rozpisywać wszystko na pojedyncze akcje. Można je porównać do bloków kodu lub funkcji w programowaniu.
Przykłady:

- `StructuredActivityNode` – jest to ogólna grupa akcji wykonywanych razem. Może mieć własny lokalny kontekst (zmienne i parametry).

- `LoopNode` – modeluje pętle (np. `while`, `for-each`). Zawiera warunek wejścia, warunek kontynuacji i ciało pętli.
- `ContitionalNode` – odpowiada za modelowanie instrukcji warunkowych (`if-else`). Określa alternatywne ścieżki wykonania na podstawie warunków.
- `Clause` – używane wewnątrz `ContitionalNode`, definiuje warunek i odpowiadającą mu akcję.

h) Inne akcje (`Other Actions`)
Są to akcje, które nie mieszczą się bezpośrednio w głównych grupach. Zwykle mają bardziej specyficzne zastosowanie.
Przykłady:
- `TestIdentityAction` – służy do porównywania dwóch obiektów (czy są tą samą instancją, a nie tylko równoważne).
- `ValueSpecificationAction` – tworzy lub dostarcza wartość określaną bezpośrednio (np. liczby stałe, wartości logiczne).
- `ClearAssociationAction` – usuwa wszystkie powiązania obiektu w ramach wskazanej asocjacji.
- `ReclassifyObjectAction` – służy do zmiany klasyfikacji obiektu w czasie wykonywania modelu (dynamiczne zmiany klas, np. zmiana statusu obiektu).
- `StartClassifierBehaviorAction` – uruchamia zachowanie domyślne powiązane z klasą (np. maszyny stanów przypisanej do obiektu).

Przekaźniki (`Pins`) w UML są specjalnymi elementami używanymi w diagramach aktywności do modelowania przepływu danych między akcjami. Służą one jako punkty połączeń, przez które akcje odbierają dane wejściowe i przekazują wyniki wyjściowe.

Rodzaje przekaźników:

a) `InputPin` – reprezentuje dane wejściowe dostarczane do akcji. Akcja nie może rozpocząć wykonania, jeśli którykolwiek z jej `InputPins` nie otrzyma wymaganej liczby wartości zgodnie z dolną granicą krotności (np. akcja „`Oblicz VAT`” może mieć `InputPin` przyjmujący wartość `kwotaNetto`).

b) `OutputPin` – służy do przekazywania wyników działania akcji. Po zakończeniu akcji, dane są umieszczane na `OutputPin` i mogą być przekazane do kolejnych akcji (np. akcja „`Oblicz VAT`” może mieć `OutputPin` zawierający wartość `kwotaBrutto`).

c) `ActionInputPin` – specjalny typ `InputPin`, obliczający swoją wartość poprzez wykonanie przypisanej akcji. Umożliwia dynamiczne generowanie danych wejściowych dla akcji (np. akcja „`Zapisz wynik`” może mieć `ActionInputPin`,

który najpierw wykonuje akcję „`Oblicz wynik`”, a następnie przekazuje wynik do „`Zapisz wynik`”).

d) `ValuePin` – jest to `InputPin`, który dostarcza określoną wartość zdefiniowaną przez wyrażenie wartości (`ValueSpecification`). Wartość jest oceniana i dostarczana do akcji w momencie jej rozpoczęcia (np. akcja „`Ustaw status`” może mieć `ValuePin` ustawiający wartość „`Aktywny`”).

Funkcje przekaźników:

a) Przekazywanie danych wejściowych i wyjściowych:
   - InputPin dostarcza dane do akcji.
   - OutputPin odbiera wyniki działania akcji.

b) Synchronizacja przepływu sterowania i danych:
   - Akcje rozpoczynają się dopiero, gdy wszystkie `InputPins` mają wymagane dane.
   - Pozwala to kontrolować kolejność wykonywania działań w procesie.

c) Rozdzielanie danych (*unmarshalling*):
   - Gdy akcja odbiera złożone obiekty (np. komunikaty lub sygnały), może rozdzielić atrybuty na osobne `OutputPins`.
   - Funkcja ta jest realizowana przez ustawienie `isUnmarshall = true`.

d) Umożliwienie dynamicznych obliczeń i wartości stałych:
   - `ActionInputPin` generuje wartość wejściową poprzez wykonanie innej akcji.
   - `ValuePin` dostarcza stałą wartość bez konieczności wcześniejszego działania.

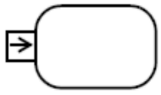
e) Kontrola przepływu obiektów i tokenów:
   - `Pins` uczestniczą w zarządzaniu tokenami obiektów i sterowania, które przemieszczają się w modelu aktywności, zgodnie z `Object Flow` i `Control Flow`.

Notacja w UML

Jako `ObjectNodes`, przekaźniki są przedstawiane jako prostokąty, lecz takie, które są dołączone do symbolu akcji, która je posiada. Nazwa przekaźnika może być umieszczona w pobliżu tego przekaźnika (sama nazwa nie jest ograniczona żadnymi regułami, lecz często pokazuje po prostu typ obiektu lub danych, które przepływają przez przekaźnik).



Jeśli w diagramie nie ma krawędzi, które odróżniają InputPins od OutputPins, można opcjonalnie umieścić strzałkę wewnątrz prostokąta przekaźnika:

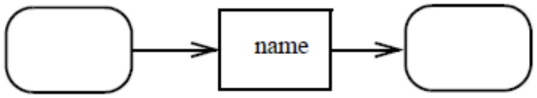


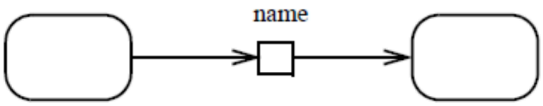
Input pin,
pin-style, with arrow



Output pin,
pin-style, with arrow

Sytuacja, w której `OutputPin` jednej akcji jest połączony z `InputPin` o tej samej nazwie w innej akcji za pomocą `ObjectFlow`, może być przedstawiony za pomocą opcjonalnej notacji strzałki:



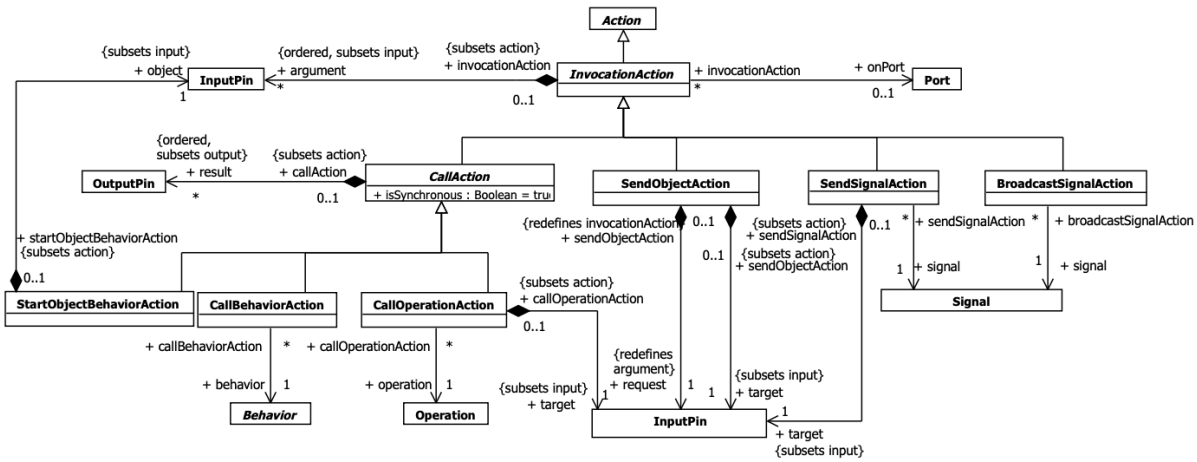


Samodzielny przekaźnik w tej notacji występuje zarówno w roli `OutputPin` jak i `InputPin`, odwzorowując także krawędź `ObjectFlow` między akcjami. **Ta notacja powinna być unikana, jeśli przekaźniki nie są tego samego typu.**

3.7 Akcja: akcje wywołania (*invocation actions*)

`InvocationAction` jest akcją, która prowadzi bezpośrednio lub pośrednio do wywołania zachowania (`Behavior`). Akcje wywołania obejmują `CallActions`, służące do wywoływania operacji (`Operations`) lub zachowań, które zostały wcześniej zainicjowane. Dodatkowe rodzaje akcji wywołania umożliwiają wysyłanie sygnałów i innych obiektów do określonych odbiorców (*targeted sending*) a także rozgłaszanie sygnałów (*broadcast*) do wszystkich dostępnych odbiorców.

Składnia abstrakcyjna



`InvocationAction` jest specjalizacją klasy `Action`, służącą do wywoływania zachowań lub operacji, a także do wysyłania sygnałów i obiektów. To abstrakcyjna koncepcja, na której bazują różne konkretne akcje.

Własności:

- `onPort [0..1]` – wskazuje `Port`, przez który akcja jest wywoływana.
- `invocationAction [*]` – powiązanie z konkretnymi wywołaniami akcji.

Specjalizacje:

- `CallAction` (Akcja wywołania) – podklasa służąca do wywoływania zachowań lub operacji.
  Atrybuty:
  - `isSynchronous: Boolean = true` – określa, czy wywołanie jest synchroniczne.

  Własności:
  - `+argument [*]` (`InputPin`) – dane wejściowe dla wywołania.
  - `+result [*]` (`OutputPin`) – dane wyjściowe, wynik działania.

  Podtypy:
  - `CallBehaviorAction` (akcja wywołania zachowania) – specjalny typ akcji wywołania, bezpośrednio wywołuje zachowanie, zamiast wywoływać operację, która z kolei wywołuje to zachowanie.
    Parametry mogą być przekazywane przez akcję do wywoływanego zachowania. Liczba pinów argumentów (`InputPins`) musi być równa liczbie parametrów zachowania o typach `in` i `in-out`. Ponadto, liczba pinów wyników (`OutputPins`) musi być równa liczbie parametrów zachowania o typach `return`, `out` i `in-out`.

    Dla wywołań synchronicznych:
    - Wykonanie akcji wywołania zachowania czeka, aż zakończy się wykonanie wywoływanego zachowania.
    - Wykonanie akcji wykonującej jest zablokowane do momentu otrzymania odpowiedzi.
    - Odpowiedź zawiera wartości dla parametrów `return`, `out` oraz `in-out`.
    - Wyniki są umieszczane na `pinach` wyjściowych akcji, a wykonanie tej akcji kończy się po ich zwróceniu.
    - Jeśli podczas wykonywania zachowania zostanie rzucony wyjątek, jest on przekazywany z powrotem do akcji wywołującej, aby rozpocząć poszukiwanie odpowiedniego handlera wyjątków.

Dla wywołań asynchronicznych:

- Akcja kończy się natychmiast po rozpoczęciu zachowania.
- Wartości `return` lub `out` nie są przekazywane z powrotem.

Notacja w UML

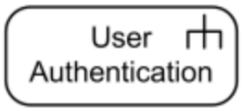
Akcja wywołania zachowania jest przedstawiana jako akcja z nazwą wywoływanego zachowania lub opisem zachowania umieszczonym wewnątrz prostokąta z zaokrąglonymi rogami. Jeśli nazwa węzła jest inna niż nazwa zachowania, to właśnie ona jest wyświetlana w symbolu.



Akcja wywołania zachowania dla zachowania `Checkout`.

Uwaga: Ponieważ wygląda to dokładnie tak samo jak zwykła akcja, nie można, patrząc jedynie na diagram, rozpoznać czy jest to zwykła nazwa akcji, nazwa akcji wywołania zachowania czy nazwa zachowania.

`CallActivityAction` można oznaczać także przez umieszczenie symbolu w formie grabi wewnątrz symbolu akcji. Symbol grabi przypomina miniaturową hierarchię, wskazując, że to wywołanie uruchamia inną aktywność (choć UML dostarcza tej notacji, nie istnieje oficjalna akcja `CallActivityAction` w specyfikacji UML).



Akcja wywołania aktywności dla aktywności `User Authentication`.

Alternatywną notacją dla wywoływanej aktywności jest pokazanie zawartości wywoływanej aktywności wewnątrz dużego prostokąta z zaokrąglonymi rogami. Krawędzie wchodzące do wywołania są połączone z węzłami obiektów parametrów w wywoływanej aktywności. Węzły obiektów parametrów są pokazane na granicy wywoływanej aktywności.

- `CallOperationAction` – wywołuje operację.

- `StartObjectBehaviorAction` – rozpoczyna zachowanie na obiekcie.

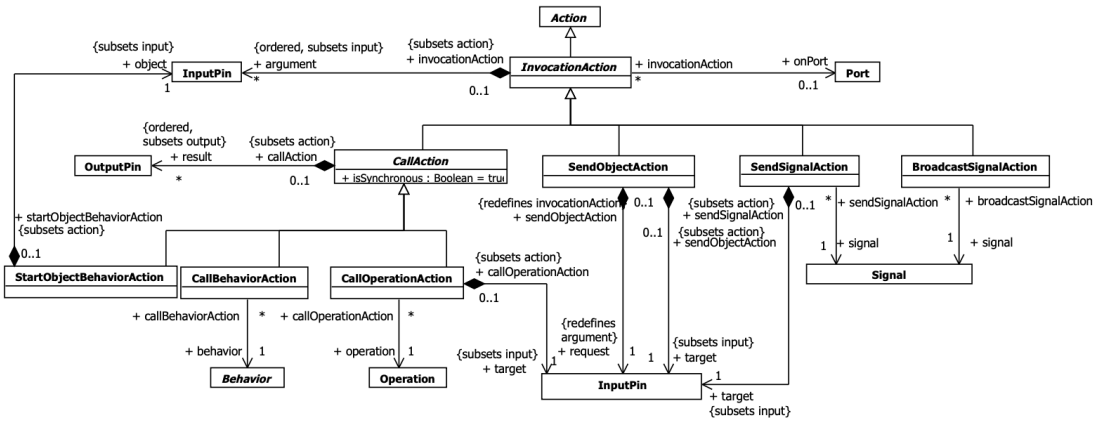
– `SendObjectAction` – wysyła obiekt do określonego odbiorcy.
Własności:
  - `+request` (`InputPin`) – obiekt do wysłania.
  - `+target` (`OutputPin`) – adresat wysyłki.

– `SendSignalAction` (akcja wywołania sygnału) – akcja wywołania, która tworzy sygnał na podstawie swoich danych wejściowych i przekazuje go do określonego obiektu docelowego, gdzie może spowodować wyzwolenie przejścia maszyny stanów lub wykonanie aktywności.
Gdy wszystkie warunki wstępne wykonania akcji są spełnione, sygnał jest generowany z argumentów i przekazywany do zidentyfikowanego obiektu docelowego. Obiekt docelowy może być lokalny lub zdalny. Jeśli dane wejściowe akcji są już sygnałem, należy zamiast tego użyć akcji wysyłania obiektu (`SendObjectAction`).
Nadawca sygnału (nazywany również żądającym - *requestor*) natychmiast kontynuuje wykonywanie, bez oczekiwania na jakąkolwiek odpowiedź. Akcja wysyłania sygnału nie otrzymuje żadnej odpowiedzi od wywołanego zachowania. Każda próba odpowiedzi jest po prostu ignorowana i nie następuje żadne przesłanie z powrotem do nadawcy.

Sposób przesyłania sygnału, czas wymagany na jego przesłanie, kolejność, w jakiej transmisje docierają do różnych obiektów docelowych, oraz ścieżka dotarcia do tych obiektów nie są zdefiniowane w UML. Instancja sygnału może zostać skopiowana podczas transmisji, więc tożsamość obiektu może nie zostać zachowana.

Gdy transmisja dotrze do obiektu docelowego, może wywołać zachowanie w tym obiekcie. Efekt otrzymania sygnały przez obiekt jest określony w specyfikacji Common Behaviors. Takie efekty obejmują wykonanie aktywności oraz wyzwolenie przejść maszyny stanów.

Składnia abstrakcyjna



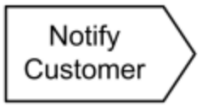
Właściwości:

- `+sendSignalAction [*] {subsets action}` – relacja wskazująca, że `SendSignalAction` jest podtypem akcji i może występować wielokrotnie.
- `+sendObjectAction [0..1] {subsets action}` – opcjonalnie może redefiniować akcję `SendObjectAction`, co oznacza możliwość wykorzystania obiektów jako sygnałów.
- `+signal [1]` – wskazuje sygnał, który ma zostać wysłany. Jest to obowiązkowy element typu `Signal`.
- `+target [1] {subsets input}` – określa obiekt docelowy, do którego sygnał jest wysyłany. Definiowany jest przez `InputPin`.

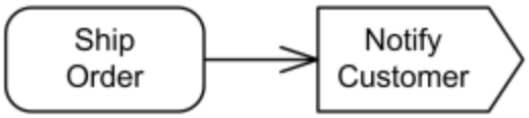
Notacja w UML

Akcja wysyłania sygnału jest oznaczana jako wypukły pięciokąt. Należy zauważyć, że nazwa akcji odpowiada nazwie klasy sygnału, który jest wysyłany. Obiekt docelowy nie jest określany za pomocą tej notacji.

Przykłady:



Akcja wysyłania sygnału `Notify Customer`, tworząca i wysyłająca sygnał `Notify Customer`.



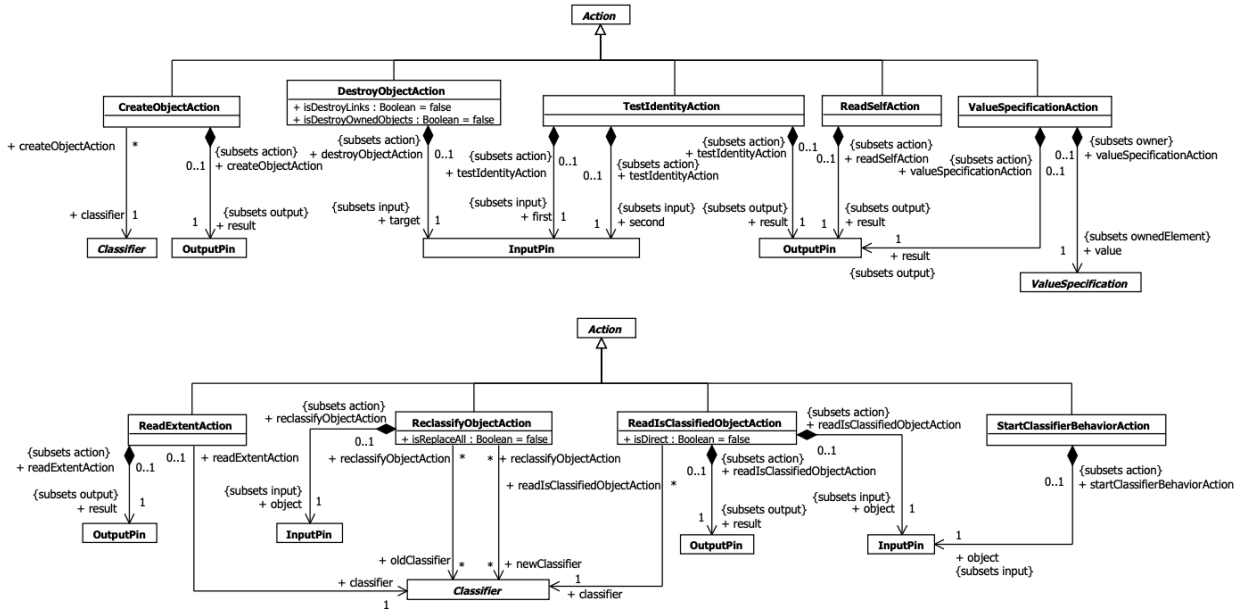
Po tym jak zamówienie jest dostarczone, akcja wysyłania sygnału `Notify Customer` tworzy i wysyła sygnał `Notify Customer`.

– `BroadcastSignalActionn` – wysyła sygnał do wszystkich dostępnych odbiorców.
Własności:
- `+signal [1]` – wysyłany sygnał.
- Brak określonego odbiorcy.

3.8 Akcja: akcje obiektów (*object actions*)

Akcje obiektów (`Object Actions`) dotyczą tworzenia, niszczenia i porównywania instancji klasyfikatorów. Obejmują również akcje umożliwiające odczytywanie instancji danego klasyfikatora, sprawdzanie klasyfikacji instancji oraz zmienianie klasyfikacji instancji.

Składnia abstrakcyjna



`Object Actions` można podzielić na wiele poszczególnych podtypów:

a) `CreateObjectAction` – tworzy nowy obiekt określonego klasyfikatora.
Właściwości:
- `classifier: Classifier` – klasyfikator określający typ tworzonego obiektu.
- `result: OutputPin` – przekaźnik, na którym umieszczany jest nowo utworzony obiekt.

b) `DestroyObjectAction` – niszczy określony obiekt.
Właściwości:
- `isDestroyLinks: Boolean` – jeśli `true`, niszczy również wszystkie powiązania obiektu.
- `isDestroyOwnedObjects: Boolean` – jeśli `true`, niszczy również wszystkie obiekty będące własnością obiektu docelowego.
- `target: InputPin` – wskazuje obiekt do zniszczenia.

c) `TestIdentityAction` – porównuje dwa obiekty, aby sprawdzić, czy są identyczne.
Właściwości:
- `first: InputPin` oraz `second: InputPin` – dwa porównywane obiekty.
- `result: OutputPin` – zwraca wynik logiczny `true` lub `false`.

d) `ReadSelfAction` – zwraca referencję do bieżącego obiektu (`this`/`self`).
Właściwości:
- `result: OutputPin` – zwraca obiekt, który wykonuje akcję.

e) `ValueSpecificationAction` – określa wartość i przekazuje ją dalej.
Właściwości:
   - `value: ValueSpecification` – określa wartość.
   - `result: OutputPin` – zwraca wartość.

f) `ReadExtentAction` – odczytuje wszystkie istniejące instancje określonego klasyfikatora.
Właściwości:
   - `classifier: Classifier` – zadany klasyfikator.
   - `result: OutputPin` – lista obiektów danego typu.

g) `ReclassifyObjectAction` – zmienia klasyfikację obiektu (jego typ).
Właściwości:
   - `object: InputPin` – obiekt do przeklasyfikowania.
   - `oldClassifier: Classifier` – typy, które mają być usunięte.
   - `newClassifier: Classifier` – typy, które mają być przypisane.
   - `isReplaceAll: Boolean` – jeśli `true`, usuwa wszystkie stare klasyfikacje.

h) `ReadIsClassifiedObjectAction` – sprawdza, czy obiekt jest instancją danego klasyfikatora.
Właściwości:
   - `object: InputPin` – obiekt sprawdzany.
   - `classifier: Classifier` – sprawdzany klasyfikator.
   - `isDirect: Boolean` – określa, czy klasyfikacja musi być bezpośrednia.
   - `result: OutputPin` – wynik logiczny `true` lub `false`.

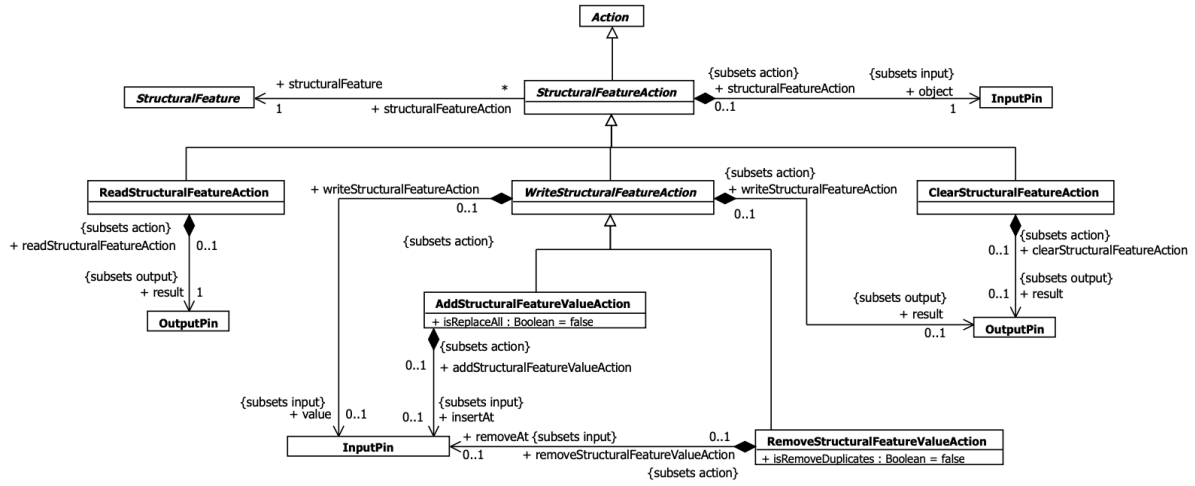
i) `StartClassifierBehaviorAction` – uruchamia domyślne zachowanie klasyfikatora dla danego obiektu.
Właściwości:
   - `object: InputPin` – obiekt, dla którego ma być uruchomione zachowanie.

C11 3.9-3.11

3.9 Akcja: akcje cech strukturalnych (*Structural Feature Action*), akcje zmiennych (*Variable Actions*), akcje odpowiedzi na zdarzenia (*Accept Actions*), pozostałe akcje (*Other Actions*)

Akcje cech strukturalnych (`StructrualFeatureActions`) wspierają odczytywanie i zapisywanie cech strukturalnych (`StructuralFeatures`). W UML nie została dla nich zdefiniowana żadna szczególna notacja.

Składnia abstrakcyjna



`StructuralFeatureAction` reprezentuje akcję działającą na określonym `StructuralFeature`. Wymaga wejściowego przekaźnika wskazującego obiekt (`+object`), na którym operacja ma być wykonana. Wszystkie akcje poniżej dziedziczą po tej klasie.

Klasy dziedziczące:

a) `ReadStructuralFeatureAction`:
   - Odczytuje wartość `StructuralFeature` z podanego obiektu.
   - Zwraca wynik poprzez `OutputPin` (`+result`).
   - Używana do pozyskiwania wartości atrybutów lub właściwości obiektów.

b) `ClearStructuralFeatureAction`:
   - Usuwa wszystkie wartości danej cechy strukturalnej obiektu.
   - Może opcjonalnie zwracać usunięte dane przez `OutputPin`.

c) `WriteStructuralFeatureAction`:
   - Jest to abstrakcyjna podklasa bazowa dla wszystkich akcji modyfikujących `StructuralFeature`.

d) `AddStructuralFeatureValueAction`:
   - Dodaje nową wartość do cechy strukturalnej.
   - Atrybut `+isReplaceAll: Boolean = false` określa, czy zastąpić wszystkie istniejące wartości (`true`), czy tylko dodać (`false`).
   - Wymaga `+value` jako wartości do dodania oraz opcjonalnie `+insertAt` jako pozycji dodania w kolekcji.

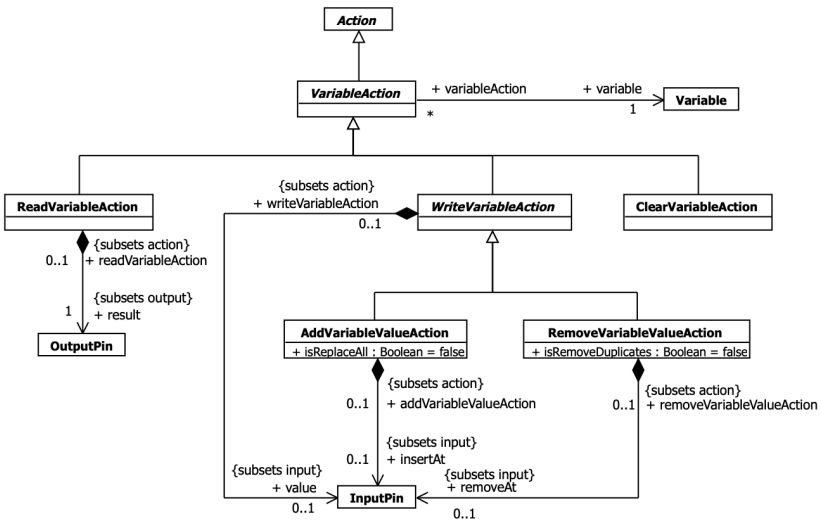
e) `RemoveStructuralFeatureValueAction`:
   - Usuwa wskazaną wartość z cechy strukturalnej.
   - `+isRemoveDuplicates: Boolean = false` – jeżeli `true`, Usuwa wszystkie wystąpienia wartości.
   - `+removeAt` (opcjonalnie) – pozycja, z której należy usunąć wartości.

f) `ValueSpecificationAction`:
   - Pozwala ustawić konkretną wartość (np. liczbową, tekstową) jako `ValueSpecification`.
   - Jest przydatna przy ustalaniu danych wejściowych do innych akcji.

g) `ReadSelfAction`:
   - Zwraca referencję do obiektu (`this`) w kontekście działania.

Akcje zmiennych (`VariableActions`) wspierają odczytywanie i zapisywanie zmiennych (`Variables`). Akcje zmiennych operują na statycznie zdefiniowanych zmiennych, co oznacza, że zmienna musi być zdefiniowana przez aktywność lub `StructuredActivityNode`, który zawiera `VariableAction`.

Składnia abstrakcyjna



`VariableAction` posiada odniesienie do konkretnej `Variable`, na której operuje (`+variable`).

Klasy dziedziczące:

a) `ReadVariableAction`:
   - Odczytuje aktualną wartość zmiennej.
   - Wynik trafia do `OutputPin` (`+result`).
   - Stosowana do przekazywania wartości zmiennej do dalszego przetwarzania.

b) `ClearVariableAction`:
   - Usuwa wszystkie wartości przypisane do zmiennej (np. jeśli to kolekcja, czyści całość).
   - Nie przyjmuje żadnych danych wejściowych ani nie zwraca danych wyjściowych.

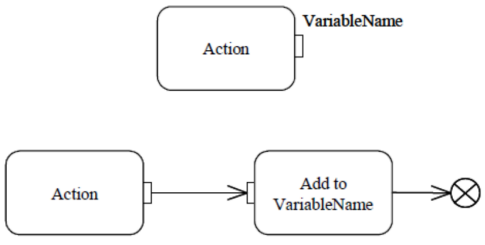
c) `WriteVariableAction`:
   - Jest to abstrakcyjna klasa, która jest nadklasą dla akcji modyfikujących zawartość zmiennej.

d) `AddVariableValueAction`:
   - Dodaje wartość do zmiennej.
   - Atrybut `+isReplaceAll: Boolean = false` określa, czy zastąpić wszystkie istniejące wartości (`true`), czy tylko dodać (`false`).
   - Wymaga `InputPin` z wartością do dodania (`+value`) i opcjonalnie pozycję w kolekcji (`+insertAt`).

e) `RemoveVariableValueAction`:
   - Usuwa daną wartość ze zmiennej.
   - `+isRemoveDuplicates: Boolean = false` – jeżeli `true`, Usuwa wszystkie wystąpienia wartości.
   - `+removeAt` (opcjonalnie) – pozycja, z której należy usunąć wartości.

Notacja w UML

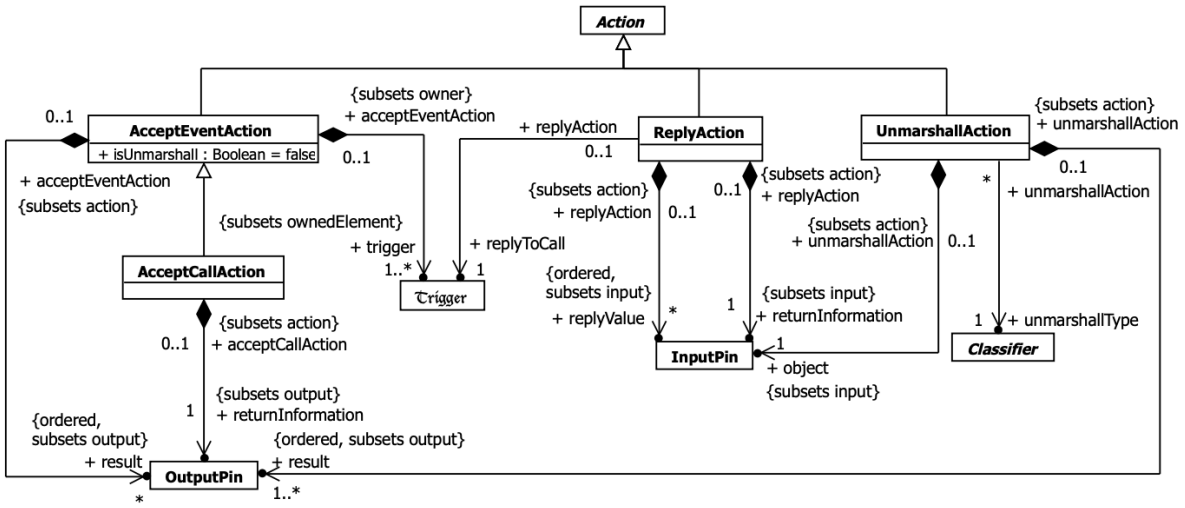
Akcje zmiennych przedstawia się analogicznie do innych akcji, lecz jeśli `AddVariableValueAction` ma ustawiony atrybut `isReplaceAll` na `true`, można to oznaczyć za pomocą adnotacji tekstowej `{replaceAll}` obok nazwy zmiennej.



Akcje odpowiedzi na zdarzenia (`AcceptEventActions`) oczekują na wystąpienie jednego lub więcej zdarzeń (`Events`). Jeśli odebrane zdarzenie dotyczy `CallEvent`, wówczas można użyć `ReplyAction` do odpowiedzi na nie. Jeśli odebrane zdarzenie dotyczy `SignalEvent`, to odebrany egzemplarz sygnału (`Signal instance`) może zostać:

- rozpakowany (`unmarshalled`) natychmiast do swoich wartości atrybutów.
- rozpakowany później przy użyciu `UnmarshalledAction`.

Składnia abstrakcyjna



`AcceptEventAction` dziedziczy po `Action`. Obsługuje jedno lub więcej zdarzeń, których wystąpienie aktywuje dalszy przebieg aktywności.
Posiada atrybut `+isUnmarshall: Boolean = false` – jeśli `true`, to atrybuty otrzymanego obiektu są automatycznie rozbijane na osobne dane wyjściowe (`OutputPin`).

Główne elementy:

- `+trigger: Trigger [1..*]` – określa, jakie zdarzenia aktywują akcję.
- `+result: OutputPin [*]` – przechowuje dane wyjściowe.
- `+acceptEventAction: AcceptEventAction [0..1]` – związane z ogólnym działaniem.

Specjalizacja – `AcceptCallAction` – przyjmuje `CallEvent`, czyli wywołania metod lub operacji.
Zawiera:

- `+returnInformation: OutputPin [1]` – uchwyt potrzebny do późniejszej odpowiedzi (`ReplyAction`).
- `+result: OutputPin [1..*]` – wartości zwracane z wywołania.

Powiązana akcja – `ReplyAction` – używana do odpowiedzi na `AcceptCallAction`.
Wymaga:

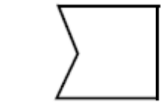
- `+replyToCall: InputPin` – odpowiadający `returnInformation` z `AcceptCallAction`.
- `+replyValue: InputPin [*]` – wartości zwracane do wywołującego.

Rozpakowywanie sygnału – `UnmarshallAction` – używane, jeśli `AcceptEventAction.isUnmarshall = false` i chcemy ręcznie rozdzielić sygnał na atrybuty.
Parametry:

- `+object: InputPin` – obiekt do rozpakowania.
- `+result: OutputPin [*]` – atrybuty jako osobne dane.
- `+unmarshallType: Classifier [1]` – typ klasy sygnału do rozpakowania.

Notacja w UML

Ogólna akcja `AcceptEventAction` jest oznaczana symbolem wklęsłego pięciokąta. Nazwa akcji może być umieszczona wewnątrz symbolu.



Accept event action

Akcja akceptacji zdarzenia.

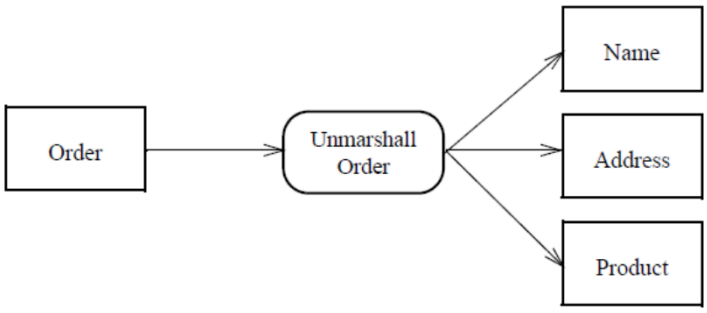
Akcja oczekiwania na upływ czasu (czyli `AcceptEventAction` z jednym wyzwalaczem typu `TimeEvent`) jest oznaczana symbolem klepsydry. Nazwa akcji może być umieszczona pod symbolem.



Accept time event action

Akcja akceptacji zdarzenia czasowego.

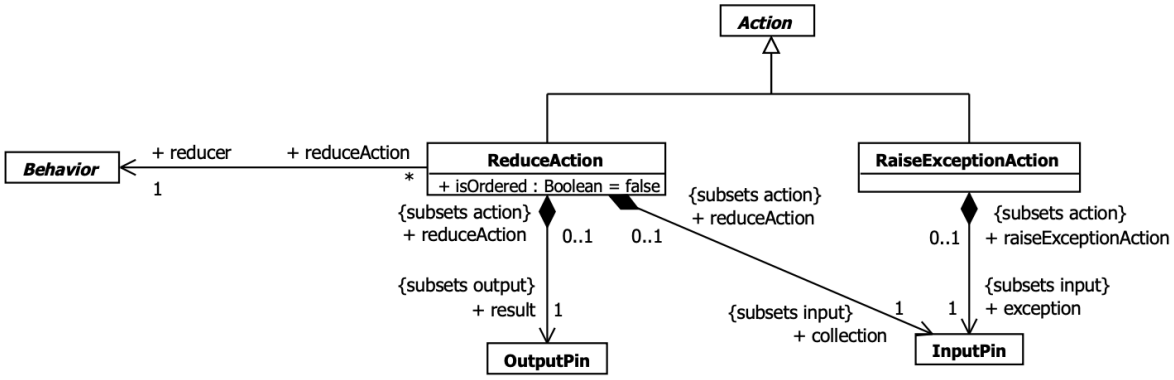
Przykład `UnmarshallAction`:



`Order` jest rozpakowane na wartości atrybutów `Name`, `Address` i `Product`.

Inne akcje (`OtherActions`) – omówione zostaną dwa dodatkowe rodzaje akcji: `ReduceAction`, służące do wielokrotnego wywoływania zachowania w celu zredukowania zbioru wartości do jednej wartości oraz `RaiseExceptionAction`, służące do zgłaszania wyjątków. UML nie definiuje dla nich żadnej szczególnej notacji.

Składnia abstrakcyjna



`ReduceAction` służy do zredukowania kolekcji wielu wartości do jednej wartości poprzez wielokrotne zastosowanie przypisanego zachowania (*reducer*).

Własności:

- `+reducer: Behavior [1]` – każde `ReduceAction` musi być powiązane z jednym zachowaniem, które określa sposób redukcji wartości (np. suma, minimum).
- `+collection: InputPin [1]` – kolekcja danych wejściowych, które mają zostać zredukowane.
- `+result: OutputPin [1]` – wynik końcowy działania redukcji.
- `+isOrdered: Boolean = false` – flaga mówiąca, czy redukcja powinna zachować kolejność danych (istotne, np. przy łączeniu tekstu lub sumowaniu we właściwej kolejności).

`RaiseExceptionAction` służy do zgłaszania wyjątku, co może zakończyć bieżące przetwarzanie i uruchomić mechanizm obsługi wyjątków.

Własności:

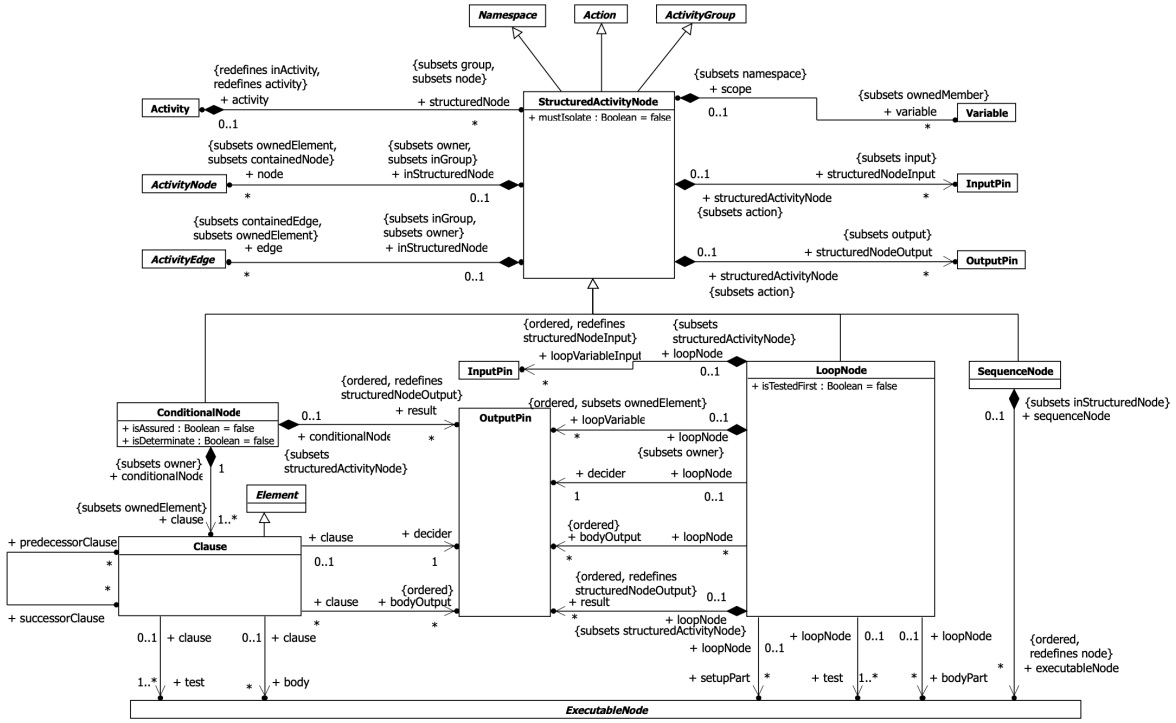
- `+exception: InputPin [1]` – wejście, przez które podawany jest wyjątek do zgłoszenia (może to być instancja klasy wyjątku).
- `+raiseExceptionAction` – odniesienie do akcji, która zgłasza wyjątek.

3.10 Akcja: akcje strukturalne (*Structured Actions*)

`StructuredAcvitivityNode` (uporządkowany węzeł aktywności) to akcja, która jest jednocześnie grupą aktywności. Oznacza to, że zawiera węzły aktywności

(`ActivityNodes`) oraz krawędzie aktywności (`ActivityEdges`), które definiują jej zachowanie podczas wykonywania.
Ponadto, wyspecjalizowane rodzaje `StructuredActivityNodes`, takie jak `ConditionalNodes` (węzły warunkowe), `LoopNodes` (węzły pętli) oraz `SequenceNodes` (węzły sekwencji), definiują szczególne reguły sterowania określające, w jaki sposób wykonywane są zawarte w nich `ExecutableNodes` (węzły wykonawcze).

Składnia abstrakcyjna



a) `StructuredActivityNode`:
   - Dziedziczy po:
     - `Action` – ponieważ sam jest rodzajem działania.
     - `ActivityGroup` – może grupować węzły i krawędzie aktywności.
     - `Namespace` – może zawierać elementy nazwane (np. zmienne).
   - Atrybuty:
     - `mustIsolate: Boolean = false` – jeśli `true`, węzeł jest wykonywany w odizolowaniu od reszty aktywności (czyli bez współbieżnego dostępu).
   - Zawiera:
     - `ActivityNode` i `ActivityEdge` – elementy wewnętrznej logiki.
     - `InputPin` i `OutputPin` – do odbierania i przekazywania danych.
     - `Variable` – zmienne lokalne.

b) `ConditionalNode`:
   Pozwala na wykonywanie jednego z wielu możliwych bloków (klauzul) w zależności od spełnienia warunku.

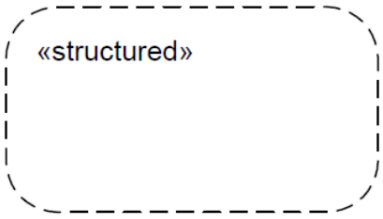
- Atrybuty:
  - `isAssured: Boolean = false` – czy zagwarantowane jest, że zostanie wykonany przynajmniej jeden blok.
  - `isDeterminate: Boolean = false` – czy co najwyżej jeden blok zostanie wybrany.
- Zawiera:
  - Jedną lub więcej `Clause`, z których każdy ma warunek (`test`), blok działań (`body`) oraz wskazanie zależności między klauzulami (`predecessorClause` oraz `successorClause`).
  - `OutputPin` – zbiera wynik działania wybranego kodu.

c) `LoopNode`:
Reprezentuje strukturę pętli.
- Atrybuty:
  - `isTestedFirst: Boolean = false` – jeśli `true`, warunek (`test`) jest sprawdzany przed wykonaniem ciała pętli (czyli pętla typu `while`); jeśli `false` – pętla typu `do-while`.
- Składa się z:
  - `setupPart` – kod inicjalizujący.
  - `test` – warunek zakończenia.
  - `bodyPart` – ciało pętli.
  - `decider` – `OutputPin` decydujący o kontynuacji pętli.
  - `loopVariable` – zmienne pętli (`InputPin` i `OutputPin`).
  - `result` – wynik końcowy pętli.

d) `SequenceNode`:
Reprezentuje sekwencję węzłów wykonywanych w określonej kolejności. Jest prostym sposobem grupowania działań w liniową kolejność bez dodatkowej logiki sterowania.
- Zawiera:
  - `ExecutableNode` – kolejne węzły wykonywane w ustalonej kolejności.

Notacja w UML

`StructuredActivityNode` jest oznaczany za pomocą przerywanego prostokąta z zaokrąglonymi rogami, który otacza węzły i krawędzie, należące do tego węzła. Na górze tego prostokąta umieszcza się stereotyp `<<structured>>`.

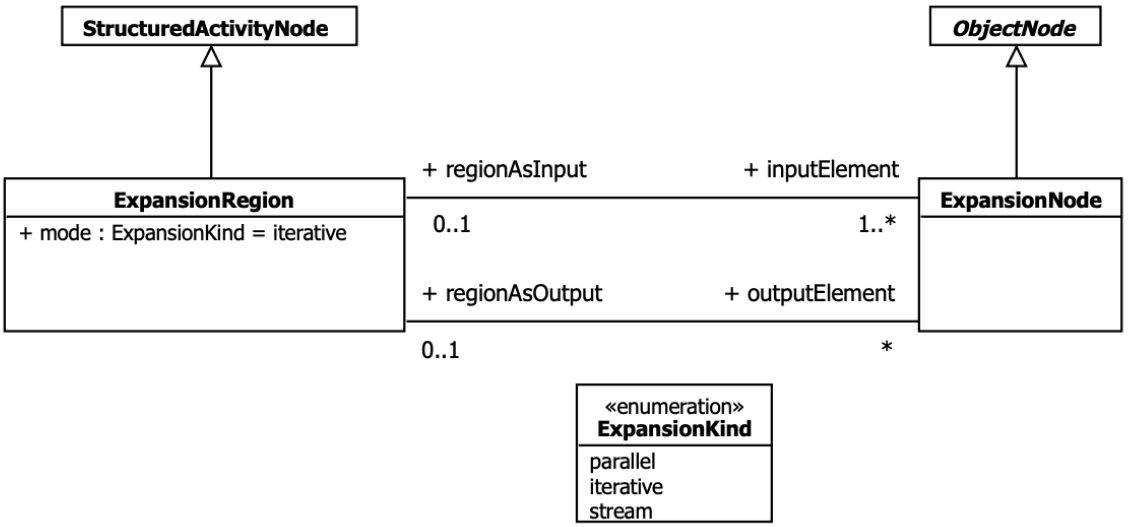


Dla `ConditionalNode`, `LoopNode` i `SequenceNode` UML nie definiuje żadnej szczególnej notacji.

3.11 Akcja: obszary rozszerzenia (*Expansion Regions*)

`ExpansionRegion` to `StructuredActivityNode`, który wykonuje zawarte w sobie element wielokrotnie, odpowiadająco do elementów kolekcji wejściowej. Automatycznie przetwarza wszystkie elementy z kolekcji przekazanej przez `ExpansionNode` jako wejście, stosując te same działania na każdym elemencie i opcjonalnie zwracając wyniki przez węzeł wyjściowy. Może służyć, np. do filtrowania, przeliczania czy konwersji zbiorów danych.

Składnia abstrakcyjna



`ExpansionRegion` dziedziczy po `StructuredActivityNode`, więc zawiera w sobie węzły aktywności i krawędzie. Posiada atrybut `mode: ExpansionKind = iterative`, który określa sposób, w jaki region przetwarza dane wejściowe.

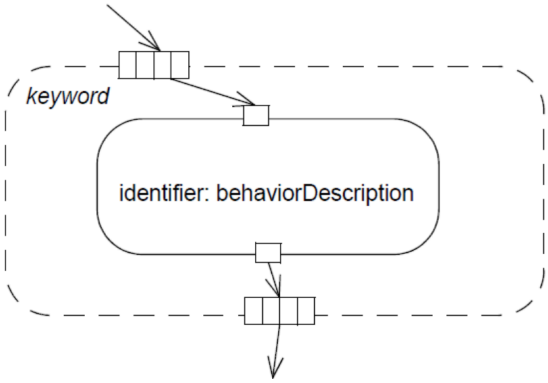
a) `ExpansionKind` (enumeracja) – typ przetwarzania kolekcji danych:
   - `parallel` – elementy są przetwarzane jednocześnie (równolegle).
   - `iterative` – elementy są przetwarzane jeden po drugim (iteracyjnie).
   - `stream` – elementy są przetwarzane jako strumień danych (ciągły dopływ i odpływ danych).

b) `ExpansionNode` – dziedziczy po `ObjectNode`. Służy jako:
   - `inputElement [1..*]` – reprezentuje wejściową kolekcję danych, z której każdy element zostanie przetworzony.
   - `outputElement [*]` – przechowuje wyniki przetwarzania każdego elementu.

c) Relacje:
   - `regionAsInput [0..1]` – wskazuje, że `ExpansionRegion` może mieć własny węzeł wejściowy.
   - `regionAsOutput [*]` – może mieć również własny węzeł wyjściowy.

Notacja w UML

`ExpansionRegion` jest przedstawiany jako przerywana ramka o zaokrąglonych rogach z jednym ze stereotypów: `<<parallel>>`. `<<iterative>>` lub `<<stream>>` w lewym górnym rogu. Węzły rozszerzenia są rysowane jako małe prostokąty podzielone pionowymi kreskami na małe komórki (symbol ten ma sugerować listę elementów).
Symbole `ExpansionNode` są umieszczane na granicy przerywanej ramki. Zazwyczaj strzałki krawędzi aktywności wewnątrz i na zewnątrz `ExpansionRegion` pozwalają rozróżnić węzły wejściowe i wyjściowe. Jeśli jednak nie są one wystarczająco czytelne, można zastosować małą strzałkę, tak jak w przypadku `Pins`.



Pełna forma notacji obszaru rozszerzenia zawierającego jeden węzeł.

4. Ograniczenia i reguły biznesowe w modelowaniu systemów w UML

C12 4.1-4.3

4.1 Reguły biznesowe i ich kategorie (definicja terminów biznesowych; fakty pokazujące relacje między terminami; ograniczenia; wnioskowania)

W UML reguły biznesowe nie są formalnie definiowane jako elementy składni języka, ale są częścią semantyki i logiki domeny, którą modelujemy. Mimo, że UML sam w sobie nie zawiera specjalnego „bloku" do modelowania reguł biznesowych, to reguły te są wyrażane przez elementy modelu (klasy, atrybuty, asocjacje, ograniczenia, stereotypy, itp.) oraz są dokumentowane jako tekst często z użyciem języków formalnych takich jak OCL (*Object Constraint Language*).

a) Definicje terminów biznesowych - są to podstawowe pojęcia domenowe, które opisują elementy rzeczywistości biznesowej. Definiują one słownictwo, które musi być spójne i jednoznaczne we wszystkich modelach.

   Przykład:
   - Klient - osoba fizyczna lub prawna dokonująca zakupu produktów.
   - Produkt - przedmiot oferowany w sprzedaży, posiadający cenę, nazwę i kategorię.

   W UML często reprezentowane są przez klasy w diagramie klas, często z użyciem komentarzy lub notatek (*notes*) oraz słowników pojęć tworzonych jako dokumentacja pomocnicza.

b) Fakty pokazujące relacje między terminami - są to logiczne zależności i relacje pomiędzy terminami biznesowymi. Odpowiadają na pytania „jak są powiązane pojęcia?".

   Przykład:
   - Klient może złożyć wiele zamówień.
   - Każde zamówienie zawiera co najmniej jeden produkt.

   W UML wyrażane są jako asocjacje między klasami, kardynalności a czasem także dodatkowe komentarze lub ograniczenia OCL dla dokładnego sprecyzowania logiki.

c) Ograniczenia (*business constraints*) - są to reguły, które określają dopuszczalne wartości, ilości, stany lub zachowania w ramach faktów i definicji.

Przykład:
- Komentarz recenzji musi mieć co najmniej 20 znaków.
- Użytkownik może wystawić tylko jedną opinię na dany produkt.
- Wiek klienta musi być większy niż 18 lat.

W UML modelowane są przy użyciu:
- OCL jako formalnego języka opisu ograniczeń.
- Stereotypów: `<<invariant>>`, `<<precondition>>`, `<<postcondition>>`.
- Adnotacji tekstowych na diagramach.

Przykład OCL:
```
context Review
inv: self.comment.size() >= 20
```

d) Wnioskowanie (*business derivations/inferences*) - są to reguły, które pozwalają wywnioskować dane na podstawie innych danych, tzn. dane pochodne lub logiczne implikacje.

Przykład:
- Jeśli klient kupił produkt A i B, może otrzymać rabat.
- Liczba zamówień > 10 → przyznanie statusu „Stały klient”.

W UML modelowane są jako:
- Atrybuty pochodne (oznaczane np. jako `/totalPrice`).
- OCL do wyliczeń (np. suma, średnia).
- Reguły obliczeniowe umieszczane w dokumentacji operacji.

Przykład w OCL:
```
context Order
def: totalPrice: Real = self.positions->collect(price)->sum()
```

4.2 Ograniczenia na elementy modelu systemu (niezmienniki dla atrybutów klasy; niezmienniki dla asocjacji klasy; określenia warunków początkowych i końcowych realizacji operacji; ograniczenia dla zbiorów generalizacji)

W języku UML ograniczenia na elementy modelu systemu służą do precyzyjnego określenia reguł, które muszą być przestrzegane w odniesieniu do klas, ich atrybutów, asocjacji czy

hierarchii dziedziczenia. Choć UML jest głównie językiem graficznym, ograniczenia można zapisywać formalnie w języku OCL, albo nieformalnie w postaci adnotacji tekstowych.

a) Niezmienniki dla atrybutów klasy
Niezmienniki (*invariants*) definiują warunki, które muszą być spełnione przez instancję klasy.

Przykład:
- Dla klasy `Review` można określić niezmiennik, że `grade >= 1 and grade <= 5`.
- Dla `Comment.text`: długość tekstu recenzji musi być większa niż 20 znaków (`text.size() >= 20`).

Taki warunek można zapisać w notacji UML za pomocą stereotypu `<<invariant>>`, np. jako tekstowy blok OCL umieszczony obok klasy lub w osobnym diagramie ograniczeń.

b) Niezmienniki dla asocjacji klasy
UML pozwala również definiować ograniczenia na relacje między obiektami, np. asocjacje. Typowe ograniczenia to:
- Liczność, np. `1..1`, `0..*`, itp.
- Ograniczenie unikalności, np. każdy użytkownik może wystawić jedną recenzję dla danego produktu (można zapisać jako OCL lub komentarz).
- Warunki logiczne, np. „jeśli klient złożył zamówienie, to jego koszyk powinien być pusty”.

Przykład w OCL:
```
context Client
inv OneReviewPerProduct:
    self.review->forAll(r1, r2 | r1 <> r2 implies
r1.product <> r2.product)
```

c) Warunki początkowe i końcowe operacji
Operacje w UML mogą mieć przypisane:
- Warunki początkowe (*preconditions*), które muszą być spełnione przed wykonaniem operacji.
- Warunki końcowe (*postconditions*), które muszą być spełnione po jej zakończeniu.

Przykład dla operacji `addReview()`:
- *precondition*: użytkownik musi wcześniej zakupić produkt (`self.purchasedProducts->includes(product)`).

- *postcondition*: liczba recenzji zwiększa się o 1 (`self.reviews->size() = self.reviews@pre.size() + 1`).

d) Ograniczenia dla zbiorów generalizacji
W UML można także definiować ograniczenia na dziedziczenie:
   - `disjoint` - podklasy są rozłączne, tzn. instancja może być tylko instancją jednej z nich.
   - `overlapping` - instancja może należeć do więcej niż jednej podklasy.
   - `complete` - zbiór podklas jest pełny, tzn. każda instancja musi należeć do co najmniej jednej podklasy.
   - `incomplete` - podklasy nie wyczerpują wszystkich możliwości.

   Przykład:
   Dla klasy `User` można mieć podklasy `Client` i `Admin` i ograniczenia `{disjoint, complete}`, co oznacza, że każdy `User` jest albo instancją klasy `Client`, albo `Admin`, i że nie może być jednocześnie jednym i drugim.

4.3 Stereotypy związane z ograniczeniami w UML: <<invariant>>, <<precondition>>, <<postcondition>>

W UML stereotypy takie jak `<<invariant>>`, `<<precondition>>` i `<<postcondition>>` są używane do opisywania ograniczeń związanych z elementami modelu, przede wszystkim z klasami i operacjami. Umożliwiają one formalizację reguł, jakie muszą być spełnione w czasie życia systemu lub podczas wykonywania konkretnych operacji.

a) `<<invariant>>` - niezmiennik to warunek, który musi być zawsze spełniony dla obiektu danej klasy, zarówno przed, jak i po wykonaniu jakiejkolwiek operacji na tym obiekcie. Służy do zapewnienia spójności danych.

   Zastosowanie w UML:
   - Dołączany do klasy (np. jako adnotacja obok klasy).
   - Może być zapisany w języku naturalnym lub formalnym (najczęściej OCL).

   Przykład:
```
context Account
inv balanceNotNegative:
    self.balance >= 0
```

   W języku naturalnym: „saldo konta nie może być ujemne”.

b) `<<precondition>>` - jest to warunek, który musi być spełniony przed wykonaniem operacji. Jeśli warunek wstępny nie jest spełniony, operacja nie powinna być wykonywana - może zostać odrzucona lub zgłosić wyjątek.

Zastosowanie w UML:
- Dotyczy operacji/metody klasy.
- Opisuje wymagania wejściowe lub stan obiektu/systemu wymagany do poprawnego działania metody.
- Zapis można umieścić w dokumentacji operacji lub jako adnotację na diagramie klas.

Przykład:
```
context Order::confirm()
pre OrderHasProducts:
    self.products->notEmpty()
```

W języku naturalnym: „zamówienie musi zawierać przynajmniej jeden produkt przed jego zatwierdzeniem”.

c) `<<postcondition>>` - jest to warunek, który musi być spełniony po zakończeniu wykonania operacji, o ile operacja zakończyła się powodzeniem. Określa efekt działania danej funkcji.

Zastosowanie:
- Dotyczy również operacji/metody.
- Może opisywać zmiany w stanie obiektu, liczbę elementów, powiązania, komunikaty itp.

Przykład:
```
context User::addReview()
post ReviewAdded:
    self.reviews->size() = self.reviews@pre->size() + 1
```

W języku naturalnym: “liczba recenzji użytkownika zwiększyła się o 1”.

Gdzie i jak umieszcza się te stereotypy?
W UML można dodać je jako komentarze lub adnotacje przy klasach i operacjach, umieścić w osobnym diagramie ograniczeń, przedstawić za pomocą tekstu opisanego stereotypem lub też zapisać formalnie w języku OCL.

5. Wymagania

C13 5.1-5.8

5.1 Wymagania wg UML (diagram przypadków użycia) i wg sysML (diagram wymagań)

Modelowanie wymagań to jeden z pierwszych etapów inżynierii systemów, od którego zależy jakość całego projektu informatycznego. W zależności od przyjętego standardu modelowania (UML czy sysML), sposób prezentacji i formalizacji wymagań będzie inny, choć cele pozostają zbieżne.

a) Modelowanie przypadków użycia w UML (*Unified Modeling Language*)
W UML wymagania funkcjonalne są zwykle modelowane w postaci diagramów przypadków użycia.

Elementy diagramu przypadków użycia:
- Aktorzy/aktory - czyli zewnętrzne byty (użytkownicy, systemy, urządzenia) wchodzące w interakcję z systemem.
- Przypadki użycia - czyli funkcjonalności systemu widziane oczami aktora.
- Relacje między przypadkami użycia:
  - `<<include>>` - obowiązkowy podprzypadek wykonywany zawsze, gdy wykonywany jest nadrzędny przypadek użycia.
  - `<<extend>>` - opcjonalne zachowanie, warunkowo włączane do przypadku głównego.
  - Generalizacja aktorów lub przypadków (dziedziczenie funkcjonalności).

Kwestia ograniczeń:
Diagram przypadków użycia nie jest modelem wymagań w sensie formalnym, jest to raczej narzędzie do identyfikacji i wizualizacji interakcji funkcjonalnych. Nie umożliwia on wyrażania niefunkcjonalnych wymagań, priorytetów, statusu wymagań czy ich identyfikatorów.

b) Diagram wymagań (*Requirements Diagram*) w sysML (*Systems Modeling Language*)
sysML to rozszerzenie UML, opracowane z myślą o inżynierii systemów. Jednym z jego głównych elementów jest formalny model wymagań, przedstawiony w postaci diagramu wymagań.

Jak wygląda diagram wymagań:
- Każde wymaganie to element typu `requirement`.

- Reprezentowane jako prostokąt z etykietą `<<requirement>>` zawierającą:
  - `id` - unikalny identyfikator.
  - `text` - opis wymagania.

Typowe relacje między wymaganiami:

- `<<deriveRetq>>` - wymaganie pochodne, wywiedzione z innego.
- `<<refine>>` - uszczegółowienie wymagania.
- `<<satisfy>>` - powiązanie elementu projektowego, który spełnia wymaganie.
- `<<verify>>` - wskazuje test lub przypadek weryfikacji.
- `<<trace>>` - ogólne śledzenie powiązania między wymaganiami i innymi elementami modelu.

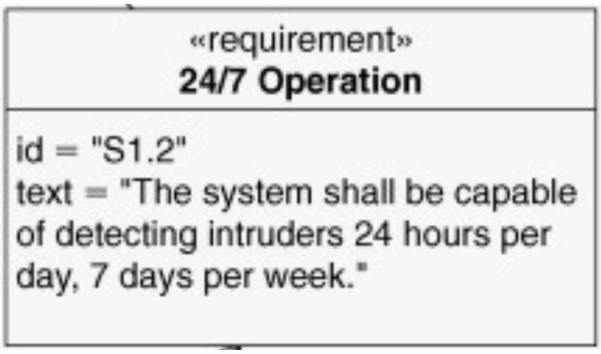
Dodatkowe możliwości:
sysML umożliwia podział na wymagania funkcjonalne, niefunkcjonalne, techniczne, interfejsowe itd. Można tworzyć hierarchię wymagań, analizować wpływ zmian, czy przypisywać atrybuty (np. priorytet, status, ryzyko).

5.2 Elementy składni konkretnej diagramu wymagań wg sysML

Diagram wymagań w języku sysML służy do modelowania, zarządzania i śledzenia wymagań funkcjonalnych, niefunkcjonalnych i projektowych systemu. sysML, jako rozszerzenie UML, wprowadza dedykowane elementy graficzne (składnię konkretną), które umożliwiają wyrażenie nie tylko treści wymagań, ale także ich zależności i realizacji.

a) Element podstawowy `<<requirement>>`
Notacja:
   - Prostokąt ze stereotypem `<<requirement>>` w nagłówku.
   - Wewnątrz znajdują się `id` oraz `text`.

Przykład:



b) Rodzaje relacji pomiędzy wymaganiami i innymi elementami modelu
sysML nie tylko reprezentuje wymagania, ale również określa ich związki z innymi wymaganiami i modelami projektowymi. Główne relacje:
   - `<<deriveRetq>>`:
     - Używane, gdy jedno wymaganie wynika logicznie z innego.
     - Notacja: strzałka z przerywaną linią i stereotypem.
   - `<<refine>>`:
     - Służy do doprecyzowania, np. z wysokopoziomowego na operacyjne.
     - Może łączyć wymaganie z przypadkiem użycia, klasą itp.
     - Notacja: przerywana linia ze stereotypem.
   - `<<satisfy>>`:
     - Łączy wymaganie z elementem modelu, który je realizuje (np. klasą, komponentem).
     - Notacja: pełna linia z pustym trójkątem i stereotypem.
   - `<<verify>>`:
     - Pokazuje, który test weryfikuje dane wymaganie.
     - Notacja: pełna linia z pustym rombem i stereotypem.
   - `<<trace>>`:
     - Relacja służąca do ogólnego śledzenia (np. między wymaganiami a przypadkami użycia).
     - Notacja: przerywana linia i stereotyp.

c) Elementy wspierające: `TestCase`, `Block`, `UseCase`
Diagram wymagań może odnosić się do innych elementów sysML:
   - `TestCase` - przypadek testowy, powiązany przez `<<verify>>`.
   - `Block` - element architektury systemu, powiązany przez `<<satisfy>>`.
   - `UseCase` - przypadek użycia, powiązany przez `<<refine>>`.

   Elementy te korzystają z klasycznej notacji UML, ale w diagramie wymagań pokazują, jak wymaganie jest realizowane lub weryfikowane.

d) Grupowanie i zarządzania wymaganiami
sysML umożliwia tworzenie hierarchii wymagań, np. wymagania ogólne → szczegółowe a także dodawanie tagów i właściwości (np. `priority`, `risk`, `status`, `source`, `type`) oraz organizację w pakiety (`Functional`, `Safety`, `Performance`).

5.3 Wymagania wg RUP

Wymagania według RUP (*Rational Unified Process*) to jedno z fundamentalnych pojęć tego iteracyjnego procesu tworzenia oprogramowania. RUP kładzie ogromny nacisk na właściwe

zarządzanie wymaganiami, ponieważ błędy i nieścisłości w tym zakresie mogą mieć ogromny wpływ na jakość i sukces projektu.
Samo RUP to metodyka wytwarzania oprogramowania opracowana przez firmę Rational (obecnie IBM). Opiera się na iteracyjnym, przyrostowym podejściu, w którym rozwój systemu jest podzielony na fazy i cykle, a wymagania są analizowane i zarządzane przez cały czas trwania projektu.

Wymagania RUP są jednym z kluczowych obszarów procesowych. RUP definiuje specjalną dziedzinę zwaną przepływem wymagań (*Requirements Flow*), która odpowiada za:
- Identyfikację potrzeb interesariuszy.
- Opis wymagań funkcjonalnych i niefunkcjonalnych.
- Dokumentowanie i modelowanie wymagań.
- Zarządzanie zmianami wymagań w czasie.

RUP wyróżnia różne typy wymagań, w tym:
a) Wymagania interesariuszy (*Stakeholder Requirements*):
    - Opisują oczekiwania użytkowników i sponsorów systemu.
    - Zbierane, np. podczas wywiadów lub warsztatów.

b) Wymagania systemowe (*System Requirements*):
    - Funkcjonalne - co system ma robić (np. logowanie, przetwarzanie płatności)
    - Niefunkcjonalne - wymagania jakościowe (wydajność, bezpieczeństwo, dostępność).
    - Ograniczenia projektowe - np. technologia, platforma, język programowania.

RUP definiuje konkretne artefakty (czyli dokumenty i modele) wspierające pracę z wymaganiami:
- *Vision* - dokument wizji systemu (cele, główne funkcje, interesariusze).
- SRS (*Software Requirements Specification*) - szczegółowa specyfikacja wymagań systemowych.
- `UseCase` *Model* - model przypadków użycia (główne źródło wymagań funkcjonalnych).
- *Supplementary Specification* - dodatkowe wymagania (np. niefunkcjonalne, interfejsowe).
- *Glossary* - słownik pojęć dziedzinowych.

W RUP zarządzanie wymaganiami to ciągły, iteracyjny proces, który obejmuje:
- Pozyskiwanie wymagań (*elicitation*) - komunikacja z użytkownikami i analiza źródeł.
- Analizę wymagań (*analysis*) - ich priorytetyzacja, grupowanie, wykrywanie sprzeczności.
- Specyfikację (*specification*) - formalny opis (często z użyciem przypadków użycia).
- Weryfikację i walidację (*validation & verification*) - sprawdzanie poprawności wymagań.

– Zarządzanie zmianą (*requirements management*) - kontrola wersji, śledzenie wpływu zmian.

W RUP cały cykl życia projektu dzieli się na cztery fazy. W każdej z nich wymagania odgrywają inną rolę:

a) Incepcja - identyfikacja najważniejszych wymagań, utworzenie dokumentu wizji i początkowych przypadków użycia.
b) Elaboracja - analiza i szczegółowa specyfikacja kluczowych wymagań, tworzenie modelu przypadków użycia.
c) Konstrukcja - implementacja funkcji zgodnie z wymaganiami, weryfikacja ich realizacji.
d) Upewnienie się, że system spełnia wymagania interesariuszy, ostateczne testy.

5.4 Specyfikowanie treści przypadków użycia wg RUP (przejście od dyscypliny *Requirements* do dyscypliny *Analisys & Design*)

W RUP specyfikowanie treści przypadków użycia jest centralnym mechanizmem przejścia od dyscypliny Wymagań (*Requirements*) do dyscypliny Analizy i Projektowania (*Analisys & Design*). To przejście stanowi kluczowy etap w zrozumieniu, jak wymagania użytkowników są przekładane na strukturę i zachowanie systemu. *Requirements* określa co system ma robić, a *Analisys & Design* określa jak system ma to robić. Przypadki użycia pełnią funkcję pomostu między tymi dwiema dyscyplinami.

Rola przypadków użycia w *Requirements*:
W dyscyplinie Wymagań przypadki użycia służą do:

– Identyfikowania aktorów i ich celów względem systemu.
– Opisu funkcjonalności w postaci interakcji (scenariuszy).
– Dokumentowanie co ma zostać osiągnięte, bez opisywania szczegółów technicznych.

Struktura przypadku użycia zawiera:

– Nazwę i opis celu.
– Aktorów.
– Warunki początkowe i końcowe.
– Główny przebieg i przebiegi alternatywne (rozszerzenia, wyjątki).
– Wymagania niefunkcjonalne powiązane z tym przypadkiem.

Przejście do *Analisys & Design*:
W momencie przejścia do dyscypliny Analizy i Projektowania, przypadki użycia stają się źródłem wejściowym do:

– Modelowania interakcji i klas analitycznych.
– Wyodrębnienia granic systemu (*System Boundary*).
– Identyfikacji klas kontrolnych, encji i interfejsów użytkownika.
– Tworzenia diagramów sekwencji, aktywności i klas.

Zasady przejścia według RUP

a) Zidentyfikuj jednostki analityczne na podstawie przypadków użycia:
   - Każdy przypadek użycia może prowadzić do powstania jednej lub więcej klas kontrolnych (*Control classes*).
   - Aktorzy z przypadków użycia stają się granicznymi klasami interfejsu (*Boundary classes*).
   - Dane operowane w przypadku użycia prowadzą do encji (*Entity classes*).

b) Określ scenariusze realizacji przypadków:
   Scenariusze z dokumentu przypadku użycia przekształcane są w diagramy interakcji, np.:
   - Diagramy sekwencji (*Sequence Diagrams*) - pokazują interakcje obiektów dla konkretnego scenariusza.
   - Diagramy aktywności (*Activity Diagrams*) - modelują przebieg działania z podziałem na role/obiekty.

c) Ustal odpowiedzialności klas:
   RUP zaleca użycie wzorców takich jak GRASP (*General Responsibility Assignment Software Patterns*) do przypisywania odpowiedzialności:
   - `Controller` - logika przypadków użycia.
   - `Boundary` - komunikacja z aktorem.
   - `Entity` - reprezentacja danych i reguł biznesowych.

d) Ustal operacje i atrybuty klas:
   Na podstawie kroków przypadku użycia i wymagań identyfikujesz:
   - Metody klas kontrolnych i encji.
   - Wymagane dane do przechowywania (atrybuty).

Przykład przejścia „Złóż zamówienie”:

Wymagania:
- Aktor: Klient
- Cel: Złożyć zamówienie
- Główny przebieg: wybór produktu, koszyk, płatność, potwierdzenie.

Przejście do analizy:

Element z *Requirements*	**Odpowiednik w *Analisys & Design***
Klient	Klasa graniczna: `CustomerUI`
Krok: Dodaj do koszyka	Operacja w klasie `CartControl`
Krok: Wybierz płatność	Klasa `PaymentService` (encja lub zewnętrzny serwis)
Scenariusz składania zamówienia	Diagram sekwencji: `PlaceOrder()`

5.6 Klasyfikacja FURPS+ wg RUP w odniesieniu do wymagań

FURPS+ to akronim pochodzący z metodyki Hewlett-Packard, który został zaadaptowany w RUP do klasyfikowania wymagań. Kolejne litery oznaczają:

- F - *Functional requirements* - wymagania funkcjonalne,
- U - *Usability* - użyteczność,
- R - *Reliability* - niezawodność,
- P - *Performance* - wydajność,
- S - *Supportability* - możliwość wsparcia, konserwacji.

Symbol „+" obejmuje wszystkie inne wymagania jakościowe, architektoniczne, prawne, techniczne i projektowe, które nie mieszczą się w głównych kategoriach.

FURPS+ a wymagania RUP

W RUP wymagania są pierwszoplanowym produktem dyscypliny *Requirements*, dlatego istotna jest ich jednoznaczna klasyfikacja.

FURPS+ pomaga:

- Wyodrębnić wymagania funkcjonalne jako przypadki użycia.
- Zidentyfikować wymagania niefunkcjonalne jako ograniczenia systemu.
- Przypisać odpowiednie kategorie wymagań do dokumentów projektowych i testowych.

a) ***Functional Requirements*** - opisują co system powinien robić - funkcje, reguły biznesowe i przetwarzanie danych. W RUP najczęściej modelowane są jako przypadki użycia.

   Przykład:
   Użytkownik może dodać produkt do koszyka.

b) ***Usability*** - obejmuje łatwość obsługi, ergonomię, dostępność, lokalizację. W RUP często zapisywane jako wymagania niefunkcjonalne towarzyszące przypadkom użycia.

   Przykład:
   System będzie dostępny w trzech językach.
   Użytkownik końcowy będzie potrzebował nie więcej niż 2 godzin szkolenia.

c) ***Reliability*** - dotyczy niezawodności systemu, odporności na błędy, dostępności i bezpieczeństwa danych.

   Przykład:
   System będzie dostępny przez 99.9% czasu w miesiącu.
   Dane transakcyjne muszą być zachowane w przypadku awarii.

d) ***Performance*** - związane z czasem odpowiedzi, przepustowością czy czasem przetwarzania.
Przykład:
Czas odpowiedzi systemu nie przekroczy 2 sekund dla 95% zapytań.

e) ***Supportability*** - obejmuje możliwość utrzymania, aktualizacji, migracji i testowania.

Przykład:
System musi być kompatybilny z narzędziem monitorującym X.
Kod ma być udokumentowany zgodnie z normą ISO XYZ.

f) **Pozostałe wymagania** - skład tej ostatniej grupy wchodzą grupy takie jak:
   - Architektoniczne, np. zastosowanie określonej architektury.
   - Techniczne, np. użycie konkretnego frameworka.
   - Prawne i regulacyjne, np. zgodność z RODO, ISO.
   - Interfejsowe, np. komunikacja z API zewnętrznych usług.
   - Ograniczenia projektowe, np. realizacja na istniejącej bazie danych.

5.7 Klasyfikacja FURPS+ w odniesieniu do testów (przez analogię do klasyfikacji wymagań wg RUP)

FURPS+ to klasyfikacja stosowana pierwotnie do porządkowania wymagań systemowych - funkcjonalnych i niefunkcjonalnych. W RUP ta klasyfikacja nie tylko pomaga w identyfikacji pełnych wymagań, ale również w organizacji testowania.
Każda kategoria FURPS+ może być bezpośrednio powiązana z odpowiednim typem testów, co umożliwia pełne pokrycie testowe systemu i systematyczne planowanie testów.

a) **Testowanie funkcjonalne** - obejmuje funkcje systemu opisane przypadkami użycia i wymaganiami funkcjonalnymi. Celem tego testowania jest upewnienie się, że system realizuje określone funkcje zgodnie ze specyfikacją. Dotyczy następujących typów testów:
   - Testy przypadków użycia.
   - Testy scenariuszy użytkownika.
   - Testy regresyjne.
   - Testy interfejsów użytkownika.

b) **Testowanie użyteczności** - sprawdzenie jakości interakcji użytkownika z systemem - ergonomia, dostępność, intuicyjność. Celem tego testowania jest zbadanie, czy system jest łatwy w użyciu, zrozumiały i przyjazny dla użytkownika. Dotyczy następujących typów testów:
   - Testy UX/UI.
   - Testy dostępności, np. WCAG (*Web Content Accessibility Guidelines*).
   - Testy heurystyczne.

- Testy A/B.

c) **Testowanie niezawodności** - sprawdza odporność na błędy, stabilność i dostępność. Celem tego testowania jest zapewnienie ciągłości działania systemu mimo błędów lub nieprzewidzianych sytuacji. Dotyczy następujących typów testów:
   - Testy obciążeniowe.
   - Testy awaryjne (*failure testing*).
   - Testy stabilności.
   - Testy backupu i odtwarzania (*recovery testing*).

d) **Testowanie wydajności** - sprawdza czas odpowiedzi, przepustowość i skalowalność. Celem tego testowania jest ocena działania systemu pod różnym obciążeniem i sprawdzenie, czy spełnia wymagania SLA (*Service Level Agreement*). Dotyczy następujących typów testów:
   - Testy wydajnościowe.
   - Testy przeciążeniowe (*stress testing*).
   - Testy skalowalności.
   - Benchmarking.

e) **Testowanie wsparcia technicznego i utrzymania** - sprawdza możliwość monitorowania, debugowania, konserwacji i logowania błędów. Celem tego testowania jest zapewnienie, że system można łatwo wspierać, diagnozować i rozwijać. Dotyczy następujących typów testów:
   - Testy logowania zdarzeń.
   - Testy zgodności z systemami monitoringu.
   - Testy automatyzacji wdrożeń (CI/CD *pipelines*).

f) **Testy rozszerzone** - sprawdzają wymagania techniczne, architektoniczne, prawne i bezpieczeństwa. Ich celem jest pokrycie wymagań niefunkcjonalnych niestanowiących części UX ani wydajności. Dotyczą następujących typów testów:
   - Testy integracyjne, np. API.
   - Testy bezpieczeństwa (penetracyjne, szyfrowania).
   - Testy zgodności (z przepisami i normami).
   - Testy lokalizacji i internacjonalizacji.

5.8 Wybrane klasyfikacje i metodyki opisu wymagań

a) Klasyfikacje wymagań
   Wymagania dzieli się na:
   - Funkcjonalne (FR) - określają co system ma robić, np. logowanie, przetwarzanie płatności, zarządzanie kontem.
   - Niefunkcjonalne (NFR) - mówią jak dobrze system ma działać, obejmując m.in. wydajność, bezpieczeństwo, dostępność.

Do priorytetyzacjia wymagań stosuje się **MoSCoW** (*Must Have*, *Should Have*, *Could Have*, *Won't Have* (*this time*)).

Z perspektywy użytkownika - **Model Kano**:
- *Must-be* (oczywiste),
- *Performance* (im więcej, tym lepiej),
- *Delighters* (zaskakujące, pozytywne).

b) Metodyki i style opisu wymagań
Różne style opisu pozwalają dostosować wymagania do specyfiki projektu:
- Język naturalny (ustrukturyzowany) - zrozumiały dla interesariuszy, np. „System umożliwia...".
- Przypadki użycia (UML/RUP) - opis przypadków z udziałem aktorów, zawiera przebiegi główne i alternatywne.
- Historie użytkownika (Scrum/Agile) - krótkie opisy potrzeb użytkownika, np. „Jako użytkownik chcę funkcji, aby uzyskać wartość". Często zawierają kryteria akceptacji w formacie GWT (*Given-When-Then*).
- Diagram wymagań (SysML) - graficzne przedstawienie relacji między wymaganiami, np. *satisfy*, *verify*.
- Specyfikacje formalne, np. OCL, LTA+, Z dla systemów krytycznych, wymagających jednoznaczności.

c) Standardy i notacje
Profesjonalne opisy wymagań opierają się na uznanych standardach:
- IEEE 830 / ISO / IEC / IEEE 29148 - określają strukturę i treść dokumentów specyfikacji wymagań.
- BABOK (*Business Analisys Body of Knowledge*) - zbiór praktyk analizy biznesowej, w tym zbierania i dokumentowania wymagań.
- Volere - szablon dokumentacji wymagań zawierający pełną strukturę z pytaniami pomocniczymi.

d) Praktyki wspomagające
Aby zapewnić jakość i spójność wymagań, stosuje się techniki pomocnicze:
- Śledzenie wymagań (*traceability*) - od wymagań przez projekt do implementacji i testów.
- Prototypowanie - tworzenie UI lub makiet w celu weryfikacji wymagań użytkowników.
- Modelowanie domeny, np. diagramy klas, diagramy przypadków użycia, diagramy kontekstu.
- Walidacja i weryfikacja - przegląd, testy i analizy służące sprawdzeniu poprawności i zgodności wymagań.